SISTEMAS OPERATIVOS

UNIDAD I

Contenidos

Definición de Sistema Operativo y de los conceptos fundamentales a ellos..... 5
El sistema operativo como una máquina extendida. 6
El sistema operativo como administrador de recursos..... 6
Conceptos de los sistemas operativos. 6
Procedimientos y funciones. 6
Llamada a procedimiento..... 7
Llamadas al sistema..... 7
Procesos. 9
Espacio de direcciones. 10
Archivos. 10
Entrada/salida. 12
Protección. 12
El shell..... 12
Referencias y bibliografía de la sección. 13
Evolución histórica. 13
La primera generación (1945 a 1955): tubos al vacío. 13
Tubos de vacío..... 15
La segunda generación (1955 a 1965): transistores y sistemas de procesamiento por lotes. 16
Transistor..... 17
La tercera generación (1965 a 1980): circuitos integrados y multiprogramación. 18
UNIX..... 19
La cuarta generación (1980 a la fecha): las computadoras personales. 19
Sistemas operativos de red y distribuidos 21
Referencias y bibliografía de la sección. 21
El lenguaje C. 21
Referencias y bibliografía de la sección. 24
Clasificación de los Sistemas Operativos..... 24
Según el manejo de usuarios, procesadores y procesos..... 24

Sistemas operativos de un solo usuario. 24
Sistemas operativos multiusuario. 24
Sistemas operativos monotarea o monoprogramación. 24
Sistemas operativos multitarea o multiprogramación 25
Sistemas operativos monoprocesador. 26
Sistemas operativos multiprocesador. 26
Según su propósito o destino. 27
Sistemas operativos de mainframe. 27
Sistemas operativos de servidores. 28
Sistemas operativos de computadoras personales 29
Sistemas operativos integrados. 29
Sistemas operativos móviles. 30
Sistemas operativos en tiempo real 31
Referencias y bibliografía de la sección. 31
Revisión del hardware de una computadora. 32
Procesadores 32
Chips con multihilamiento y multinúcleo 33
Memoria 35
Dispositivos de almacenamiento 38
Discos 39
Almacenamiento de estado solido 40
Dispositivos de E/S. 42
Buses 46
Arranque de la computadora. 48
Referencias y bibliografía de la sección. 50
Subsistemas principales. 51
Gestor de procesos. 51
Gestor de memoria. 51
Gestor de archivos. 52
Gestión de archivos. 52
Gestión de almacenamiento masivo. 52
Almacenamiento en cache. 52
Gestor de E/S. 52
Seguridad 53
Referencias y bibliografía de la sección. 53
Servicios del sistema operativo. 53

Referencias y bibliografía de la sección. 55
Estructura de un sistema operativo. 56
Estructura simple. 56
Estructura en niveles. 57
Referencias y bibliografía de la sección. 60
Otras estructuras del núcleo. 60
Sistemas monolíticos. 60
Sistemas de capas. 61
Microkernels. 64
MINIX 3. 66
Modelo cliente-servidor. 67
Máquinas virtuales. 69
Exokernels. 70
Exokernel del MIT. 72
Referencias y bibliografía de la sección. 72
Más sobre el modelo de capas. 72
Referencias y bibliografía de la sección. 76
Definición de modo núcleo y usuario 76
Referencias y bibliografía de la sección. 78
Llamadas al sistema. 78
Referencias y bibliografía de la sección. 84
Clasificación de llamadas al sistema. 84
Llamadas al sistema para la administración de procesos. 85
Llamadas al sistema para la administración de archivos. 87
Llamadas al sistema para la administración de directorios. 87
Más llamadas al sistema 88
La API Win32 de Windows 88
Referencias y bibliografía de la sección. 90
Interfaz con las aplicaciones. 90
Libc y glibc. 92
Wrappers. 94
POSIX. 94
Referencias y bibliografía de la sección. 95
Interfaz con la maquina real. 95
E/S. 97
Referencias y bibliografía de la sección. 98

El Intérprete de mandatos. 98
Referencias y bibliografía de la sección. 100
La interfaz gráfica de usuario. 100
Referencias y bibliografía de la sección. 104

Definición de Sistema Operativo y de los conceptos fundamentales a ellos.

No existe una definición de Sistema Operativo universalmente aceptada. Por lo tanto, podríamos comenzar por comprender el concepto de ***Sistema Informático Global*** para luego entender el lugar que ocupa el Sistema Operativo.

Un sistema informático global está compuesto por el ***hardware*** (físico), un ***Sistema Operativo***, programas de ***aplicación*** y al menos un ***usuario***.



El hardware (CPU, memoria, etc.) proporciona capacidad de cómputo. Las aplicaciones como browsers o procesadores de texto definen como se aprovecha el poder de cómputo por parte de los usuarios. El sistema operativo controla y coordina (explota) el hardware para darle poder de cómputo a las aplicaciones que son utilizadas por los usuarios.

Una primera definición de sistema operativo, entonces, sería la de ***asignador de recursos*** (tiempo de CPU, espacio en memoria, espacio de almacenamiento, etc.).

También podríamos ver al sistema informático global como hardware, software y datos. Donde el sistema operativo brinda los medios necesarios para hacer un buen uso de estos recursos.

Desde este último punto de vista, también se entiende al sistema operativo como un asignador de recursos que no realiza una tarea útil de forma directa, pero ***proporciona un entorno*** en el que otros programas pueden realizar trabajos de utilidad.

Otro enfoque sugiere que el sistema operativo es el programa que se ejecuta continuamente en la computadora (***kernel***[1]), todo lo demás serán programas del sistema o programas de aplicación. Esta forma de entender la OS predomina en los sistemas tipo Linux.

Para terminar, partiendo de la idea de computadora moderna, que en general, es un sistema complejo, motivo por el cual está equipada con una capa de software cuyo trabajo es proporcionar a los programas de usuario un modelo de computadora más simple y pulcro, así como encargarse de la administración de todos los recursos antes mencionados. Esta capa de software es el sistema operativo.

El sistema operativo como una máquina extendida[2].

La arquitectura (conjunto de instrucciones, organización de memoria, E/S y estructura de bus) de la mayoría de las computadoras a nivel de lenguaje de máquina es primitiva y compleja de programar, en especial para la entrada/salida.

Una de las principales tareas del sistema operativo es ocultar el hardware y presentar a los programas (y a sus programadores) abstracciones agradables, elegantes, simples y consistentes con las que puedan trabajar.

En otras palabras, el sistema operativo abstrae al usuario de la complejidad de administrar directamente el hardware y le presenta un conjunto de directivas que permiten tener una visión más agradable y sencilla del sistema.

El sistema operativo como administrador de recursos.

El sistema operativo está presente para administrar todas las piezas de un sistema complejo como lo es una computadora. Estas constan de procesadores, memorias, temporizadores, discos, teclados, interfaces de red, impresoras y una amplia variedad de otros dispositivos.

El trabajo del sistema operativo es proporcionar una asignación ordenada y controlada de los recursos de hardware entre los diversos programas que compiten por estos recursos.

Conceptos de los sistemas operativos.

La mayoría de los sistemas operativos comparten ciertos conceptos básicos y abstracciones que son fundamentales para comprender su funcionamiento.

Procedimientos y funciones.

El procedimiento en ámbito de la programación es un código independiente que contiene instrucciones. Un procedimiento le ordena a la computadora que ejecute un programa específico o realice ciertas tareas.

---

[1] Núcleo o kernel es un software que constituye una parte fundamental del sistema operativo, y se define como la parte que se ejecuta en modo privilegiado (conocido también como modo núcleo).
[2] Máquina extendida o máquina virtual es el sistema operativo que funciona sobre el hardware como intérprete de ordenes programadas. Es un software que emula a una computadora y puede ejecutar programas como si fuese una computadora real.

Sin entrar en detalles, es importante entender que un ***procedimiento*** se diferencia de una ***función***, una rutina o un método, que se utilizan en varios lenguajes de programación, aunque a veces se usen como sinónimos en el lenguaje cotidiano. Es decir, en la práctica moderna es posible encontrar casos en los que los términos ***procedimiento*** y ***función*** se utilicen indistintamente.

A pesar de que cumplan un papel similar al permitir a un programador llamarlo varias veces dentro de un programa, la principal diferencia es que una función está diseñada para realizar una tarea y devolver un valor al código que la llamó. Mientras que un procedimiento es una serie de pasos que se ejecutan para realizar una tarea o acción, pero no devuelve un valor al programa principal.

De todas formas, llamando a un procedimiento o función con diferentes valores, podemos hacer que un programa se ejecute de manera diferente, utilizando un conjunto separado de parámetros y datos.

Los procedimientos y funciones pueden ser creados a medida dentro del programa o en una encapsulación de código. Sin embargo, en algunos casos, los procedimientos y funciones se incluyen dentro de bibliotecas externas.

Llamada a procedimiento.

Una llamada a procedimiento es una instrucción dentro de un programa que transfiere el control a una sección de código específica para ejecutar una tarea, y al finalizar, devuelve el control al lugar donde fue llamada.

Es posible pasar una lista de parámetros junto con la llamada al procedimiento. Después de una ejecución exitosa, el control regresa al procedimiento inicial invocado y el programa continúa su ejecución.

La llamada a procedimiento local (LPC), también conocida como comunicación local entre procesos, es la función que tienen los subprocesos para sincronizarse y comunicarse cuando no se comparte memoria. En LPC, podemos intercambiar mensajes en modos de comunicación sincrónicos y asincrónicos a través de puertos. El sistema operativo proporciona herramientas integradas para el intercambio de datos y llamadas a procedimientos para las aplicaciones.

La llamada a procedimiento remoto (RPC) permite que un programa ejecute código en otra máquina remota, como si se tratara de un proceso local. En otras palabras, protocolo RPC permite a los usuarios trabajar con procedimientos remotos como si los procedimientos fueran locales.

Tanto ***LPC*** como ***RPC*** son evoluciones del concepto de "llamada a procedimiento" que vimos antes, pero aplicadas a la comunicación entre diferentes procesos o incluso diferentes computadoras.

Llamadas al sistema.

Todo procesador moderno posee al menos dos modos de funcionamiento distintos: el ***modo kernel*** (Protected Mode) y el ***modo usuario*** (System Management Mode – SMM).

- En modo kernel se encuentran disponibles todas las instrucciones y funcionalidades que la arquitectura del procesador es capaz de brindar, sin ningún tipo de restricciones.
- En modo usuario tan sólo un subconjunto de las instrucciones y funcionalidades que la arquitectura del procesador ofrece se encuentran disponibles. En este modo se ejecutan los procesos de los usuarios del sistema.

Los programas y procesos que no pertenecen al sistema operativo no tienen acceso directo a la CPU y a la memoria, sino que dependen de las ***llamadas al sistema***.

Una llamada al sistema o system call es un método utilizado por los programas de aplicación para comunicarse con el núcleo del sistema. Esto es necesario cuando una aplicación o proceso de usuario necesita transmitir a o leer información del hardware, de otros procesos o del propio núcleo. De este modo, la llamada es el punto de enlace entre el modo de usuario y el modo de núcleo.

Antes de que la llamada al sistema termine de procesarse y se transmitan o reciban los datos correspondientes, el núcleo del sistema toma el control del programa o proceso. La ejecución se interrumpe durante este período. Una vez realizada la acción solicitada por una llamada al sistema, el núcleo renuncia al control y el proceso continúa desde el punto en el que se había detenido previamente.

Tipo de system call	Función	Linux	Windows
Control del proceso	Crear proceso	fork()	CreateProcess()
Control del proceso	Terminar proceso	exit()	ExitProcess()
Gestión de archivos	Crear/abrir archivo	open()	CreateFile()
Gestión de archivos	Leer archivo	read()	ReadFile()
Gestión de archivos	Editar archivo	write()	WriteFile()
Gestión de archivos	Cerrar archivo	close()	CloseHandle()
Gestión de dispositivos	Abrir dispositivo	read()	ReadConsole()
Gestión de dispositivos	Cerrar dispositivo	close()	CloseConsole()
Gestión de la información	Definición de un intervalo de tiempo específico	alarm()	SetTimer()
Gestión de la información	Pausa (por ejemplo, de un proceso)	sleep()	Sleep()
Comunicación	Crear Pipe (memoria intermedia para el flujo de datos entre dos procesos)	pipe()	CreatePipe()
Comunicación	Creación de una memoria compartida (Shared memory)	shmget()	CreateFileMapping()

Ilustración 1 - Llamadas al sistema

Procesos.

Un proceso es en esencia un programa en ejecución. También podríamos entenderlo como un recipiente que guarda toda la información necesaria para ejecutar un programa.

A modo de ejemplo, podemos imaginar a un usuario de un sistema de multiprogramación que ha iniciado un programa de edición de video para trabajar en un proyecto mientras navega en la Web. Al mismo tiempo, un proceso en segundo plano que despierta en forma periódica para comprobar los mensajes de correo entrantes puede haber empezado a ejecutarse. Así, tenemos (al menos) tres procesos activos: el editor de video, el navegador Web y el lector de correo electrónico. Cada cierto tiempo, el sistema operativo decide detener la ejecución de un proceso y empezar a ejecutar otro; por ejemplo, debido a que el primero ha utilizado más tiempo del que le correspondía de la CPU en el último segundo.

Cuando un proceso se suspende en forma temporal, debe reiniciarse después en el mismo estado que tenía cuando se detuvo. Esto significa que toda la información acerca del proceso debe guardarse en forma explícita en alguna parte durante la suspensión. Por ejemplo, el proceso puede tener varios archivos abiertos para leerlos al mismo tiempo.

En muchos sistemas operativos, toda la información acerca de cada proceso se almacena en una tabla del sistema, conocida como la ***tabla de procesos***, la cual es un arreglo[3] de estructuras, una para cada proceso que se encuentre en existencia.

Las llamadas al sistema de administración de procesos son las que se encargan de la creación y la terminación de los procesos. Por ejemplo, un proceso llamado intérprete de comandos o shell lee comandos de una terminal. El usuario acaba de escribir un comando, solicitando la compilación de un programa. El shell debe entonces crear un proceso para ejecutar el compilador. Cuando ese proceso ha terminado la compilación, ejecuta una llamada al sistema para terminarse a sí mismo.

Si un proceso puede crear uno o más procesos aparte (conocidos como procesos hijos) y estos procesos a su vez pueden crear procesos hijos, llegamos rápidamente la ***estructura de árbol de procesos***.

Los procesos relacionados que cooperan para realizar un cierto trabajo a menudo necesitan comunicarse entre sí y sincronizar sus actividades. A esta relación se le conoce como ***comunicación entre procesos***.

Hay otras llamadas al sistema de procesos disponibles para solicitar más memoria (o liberar la memoria sin utilizar), esperar a que termine un proceso hijo y superponer su programa con uno distinto.

En muchos sistemas operativos cada persona autorizada para utilizar un sistema recibe una ***UID*** (User Identification, Identificación de usuario) que el administrador del sistema le asigna. Cada proceso iniciado tiene el UID de la persona que lo inició. Un proceso hijo tiene el mismo UID que su padre. Los usuarios pueden ser miembros de grupos, cada uno de los cuales tiene una GID (Group Identification, Identificación de grupo).

[3] Un arreglo (array o matriz) es una colección ordenada de datos (tanto primitivos como objetos, dependiendo del lenguaje). Los arreglos se emplean para almacenar múltiples valores en una sola variable.

Una UID conocida como superusuario (superuser en UNIX) tiene poder especial y puede violar muchas de las reglas de protección.

Espacio de direcciones.

Cada computadora tiene cierta memoria principal[4] que utiliza para mantener los programas en ejecución.

En un sistema operativo muy simple sólo hay un programa a la vez en la memoria. Para ejecutar un segundo programa se tiene que quitar el primero y colocar el segundo en la memoria.

Los sistemas operativos más sofisticados permiten colocar varios programas en memoria al mismo tiempo. En este contexto, la administración del espacio de direcciones de los procesos es una actividad de gran importancia.

El ***espacio de direcciones*** de un proceso es el conjunto de direcciones de memoria que este puede utilizar. Cada proceso tiene su propio espacio de direcciones, que es privado.

Por lo general, este espacio de direcciones asignado, va desde 0 hasta cierto valor máximo. En el caso más simple, la máxima cantidad de espacio de direcciones que tiene un proceso es menor que la memoria principal. De esta forma, un proceso puede llenar su espacio de direcciones y aun así habrá suficiente espacio en la memoria principal para contener todo lo necesario.

Sin embargo, en muchas computadoras las direcciones son de 32 o 64 bits, lo que significa que se pueden referenciar 232 direcciones (o 4 gigabytes de memoria RAM) y 264 direcciones (equivalente a 16 exabytes de RAM) respectivamente. Es decir, sería posible que el espacio de direcciones necesario supere en tamaño a la memoria principal.

Cuando un proceso tiene más espacio de direcciones que la memoria principal de la computadora, y desea usarlo todo se utiliza una técnica llamada ***memoria virtual***. El sistema operativo mantiene una parte del espacio de direcciones en memoria principal y otra parte en la unidad de almacenamiento, moviendo pedazos de un lugar a otro según sea necesario. En esencia, el sistema operativo crea la abstracción de un espacio de direcciones como el conjunto de direcciones al que puede hacer referencia un proceso.

El espacio de direcciones se desacopla de la memoria física de la máquina, pudiendo ser mayor o menor que la memoria física. La administración de los espacios de direcciones y la memoria física forman una parte importante de lo que hace un sistema operativo.

Archivos.

Un archivo es un contenedor de información. La mayoría de los archivos que se utilizan contienen información en un ***formato*** determinado: un documento, una hoja de cálculo, un gráfico. El formato es la disposición de los datos dentro del archivo.

[4] La memoria principal, memoria central o memoria interna es la memoria de la computadora donde se almacenan temporalmente tanto los datos como los programas que la CPU está procesando o va a procesar en un determinado momento.

Una de las funciones principales del sistema operativo es ocultar las peculiaridades de los discos y demás dispositivos de E/S, presentando al programador un modelo abstracto limpio y agradable de archivos independiente del dispositivo.

Sin duda, se requiere de las llamadas al sistema para crear los archivos, eliminarlos, leer y escribir en ellos.

Para proveer un lugar lógico en donde se puedan mantener los archivos, la mayoría de los sistemas operativos utilizan el concepto de un ***directorio*** como una manera de agrupar archivos. Serán necesarias llamadas al sistema para crear y eliminar directorios.

También se proporcionan llamadas para poner un archivo existente en un directorio y para eliminar un archivo de un directorio. Las entradas de directorio pueden ser archivos u otros directorios.

Este modelo también da surgimiento a una jerarquía (el ***sistema de archivos***).

Las jerarquías de procesos y de archivos están organizadas en forma de ***árboles***, pero la similitud se detiene ahí. Por lo general, las jerarquías de procesos no son muy profundas (más de tres niveles es algo inusual), mientras que las jerarquías de archivos son comúnmente de cuatro, cinco o incluso más niveles de profundidad.

Es común que las jerarquías de procesos tengan un tiempo de vida corto, por lo general de minutos a lo más, mientras que la jerarquía de directorios puede existir por años.

La propiedad y la protección también difieren para los procesos y los archivos. Por lo general, sólo un proceso padre puede controlar o incluso acceder a un proceso hijo, pero casi siempre existen mecanismos para permitir que los archivos y directorios sean leídos por un grupo aparte del propietario.

Para especificar cada archivo dentro de la jerarquía de directorio se utiliza su nombre de ruta. Dichos nombres consisten de la lista de directorios que deben recorrerse desde el directorio raíz para llegar al archivo (normalmente se utilizan barras diagonales para separar los componentes).

Antes de poder leer o escribir en un archivo se debe abrir y en ese momento se comprueban los permisos. Si está permitido el acceso, el sistema devuelve un pequeño entero conocido como descriptor de archivo para usarlo en las siguientes operaciones. Si el acceso está prohibido, se devuelve un código de error.

En UNIX, un concepto importante es el sistema de ***archivos montados***. Casi todas las computadoras personales tienen una o más unidades de almacenamiento. Casi siempre tienen puertos USB, a los que se pueden conectar memorias USB. Para ofrecer una manera elegante de lidiar con estos medios, UNIX permite adjuntar el sistema de archivos de cualquier unidad al árbol principal.

Otro concepto importante en UNIX es el ***archivo especial***. Los archivos especiales se proporcionan para poder hacer que los dispositivos de E/S se vean como archivos. De esta forma se puede leer y escribir en ellos utilizando las mismas llamadas al sistema que se utilizan para leer y escribir en archivos.

Existen dos tipos de archivos especiales: archivos ***especiales de bloque*** y archivos ***especiales de carácter***.

Los archivos especiales de bloque se utilizan para modelar dispositivos que consisten en una colección de bloques direccionables al azar, tales como los discos.

De manera similar, los archivos especiales de carácter se utilizan para modelar impresoras, módems y otros dispositivos que aceptan o producen como salida un flujo de caracteres.

Por convención, los archivos especiales se mantienen en el directorio ***/dev***.

Por último, un ***canal*** (pipe) es un tipo de pseudoarchivo que puede utilizarse para conectar dos procesos.

La implementación de un canal es muy parecida a la de un archivo. Un proceso puede leer los datos a través del canal, como si fuera un archivo de entrada. Por ende, la comunicación entre procesos en UNIX tiene una apariencia muy similar a las operaciones comunes de lectura y escritura en los archivos.

La única manera en que un proceso puede descubrir que el archivo de salida en el que está escribiendo no es en realidad un archivo sino un canal, es mediante una llamada al sistema especial.

Entrada/salida.

Todas las computadoras tienen dispositivos físicos para adquirir entrada y producir salida. Existen muchos tipos de dispositivos de entrada y de salida, incluyendo teclados, monitores, impresoras, etc. Es responsabilidad del sistema operativo administrar estos dispositivos.

En consecuencia, cada sistema operativo tiene un subsistema de E/S para administrar sus dispositivos. Parte del software de E/S es independiente de los dispositivos, es decir, se aplica a muchos o a todos los dispositivos de E/S por igual. Otras partes del software, como los drivers de dispositivos, son específicas para ciertos dispositivos de E/S.

Protección.

Las computadoras contienen grandes cantidades de información que los usuarios comúnmente desean proteger y mantener de manera confidencial. Es responsabilidad del sistema operativo administrar la seguridad del sistema de manera que los archivos, por ejemplo, sólo sean accesibles para los usuarios autorizados.

Además de la protección de archivos, existen muchas otras cuestiones de seguridad. Una de ellas es proteger al sistema de los intrusos no deseados, tanto humanos como no humanos (por ejemplo, virus).

El shell.

El ***shell*** o intérprete de comandos es el programa que provee una interfaz de usuario para acceder a los servicios del sistema operativo.

Aunque el shell de UNIX no forma parte del sistema operativo, utiliza con frecuencia muchas características del mismo y, por ende, sirve como un buen ejemplo de la forma en que se pueden utilizar las llamadas al sistema. También es la interfaz principal entre un usuario

sentado en su terminal y el sistema operativo, a menos que el usuario esté usando una interfaz gráfica de usuario.

En los sistemas UNIX existen muchos shells, incluyendo csh, ksh y bash. Todos derivan del shell original (sh).

Cuando cualquier usuario inicia sesión, se activa un shell. El shell tiene la terminal como entrada y salida estándar.

El indicador de comandos (***prompt***), un carácter tal como un signo de dólar, que indica al usuario que el shell está esperando aceptar un comando.

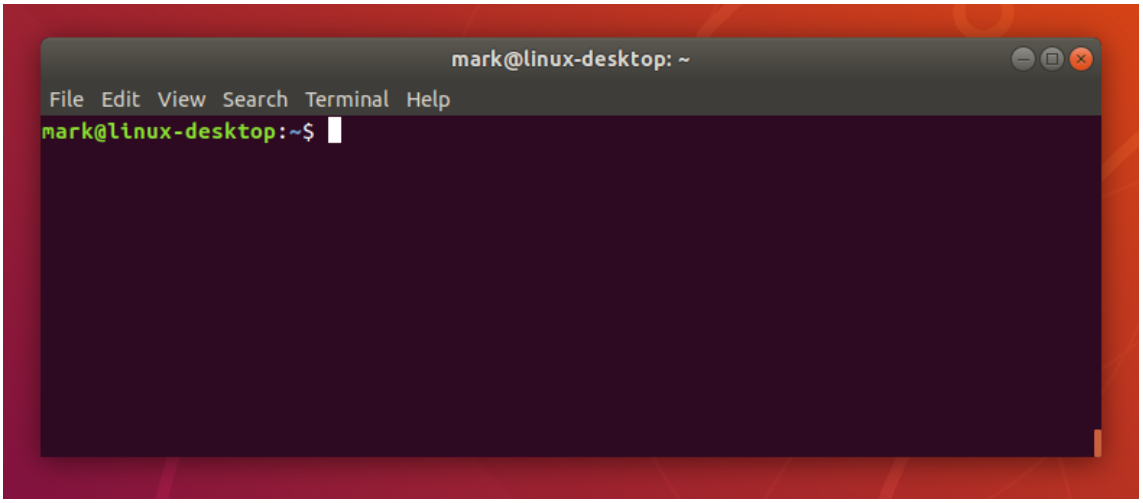


Ilustración 2 – Terminal Ubuntu

Referencias y bibliografía de la sección.

- Tanenbaum 1-1 / 1-5.
- Silberschatz 1-1 / 2-1.
- IONOS Digital Guide - System calls: ¿qué son y para qué se emplean?
- Baeldung - Remote Procedure Calls.
- IBM - Llamada a procedimiento remoto.

Evolución histórica.

Como los sistemas operativos han estado estrechamente relacionados a través de la historia con la arquitectura de las computadoras en las que se ejecutan, analizaremos generaciones sucesivas de computadoras para ver cómo eran sus sistemas operativos.

La primera generación (1945 a 1955): tubos al vacío.

Las computadoras de esta época eran muy primitivas y tardaban segundos en realizar incluso hasta el cálculo más simple. Toda la programación se realizaba exclusivamente en *lenguaje*

máquina[5] o, peor aún, creando circuitos eléctricos mediante la conexión de miles de cables a tableros de conexiones (***plugboards***[6]) para controlar las funciones básicas de la máquina. Los lenguajes de programación eran desconocidos (incluso se desconocía el lenguaje ensamblador). Los sistemas operativos también se desconocían.

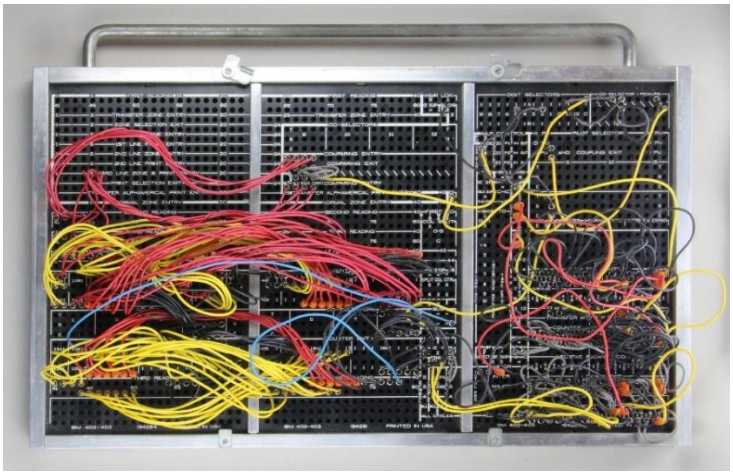


Ilustración 3 - Plugboard.

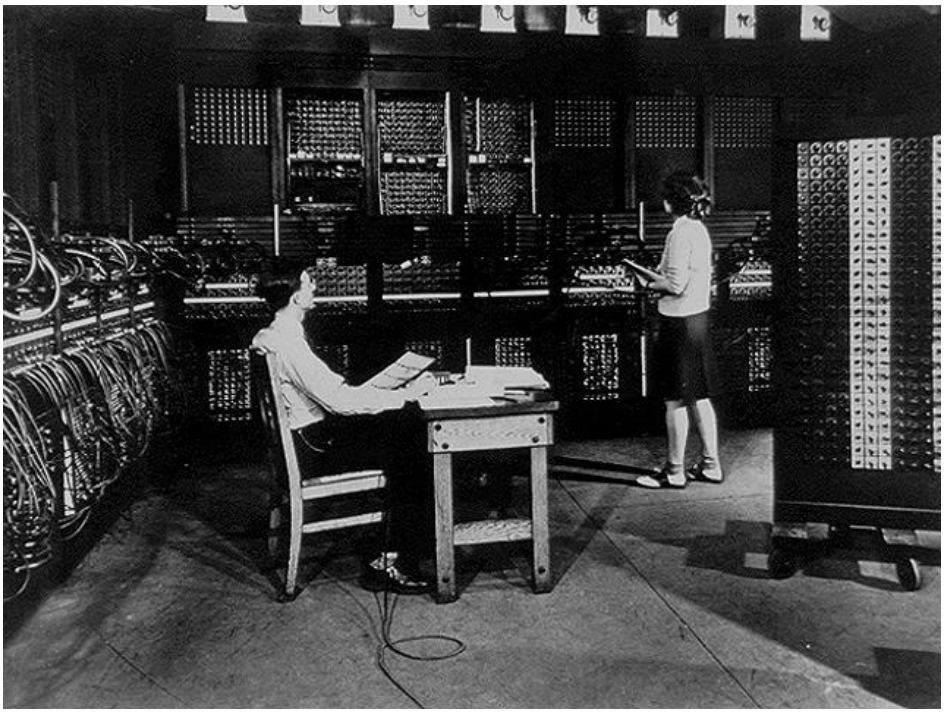


Ilustración 4 - Tableros de la ENIAC, una de las primeras computadoras de propósito general.

[5] E lenguaje de maquina está compuesto por instrucciones binarias que son directamente interpretadas por un circuito microprogramable, como el microprocesador de una computadora o el microcontrolador. Hay que tener en cuenta que el lenguaje de máquina es específico de cada máquina o arquitectura.

[6] Un panel de conexiones está formado por tomas de corriente en las que los cables se pueden insertar para completar un circuito eléctrico.

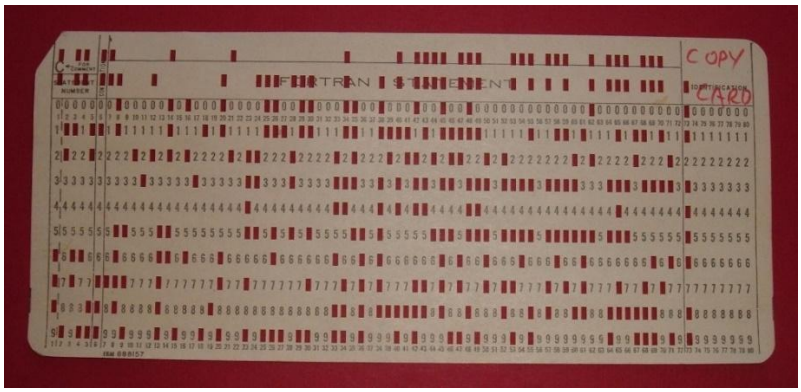


Ilustración 5 - Tarjeta perforada binaria.

Tubos de vacío.

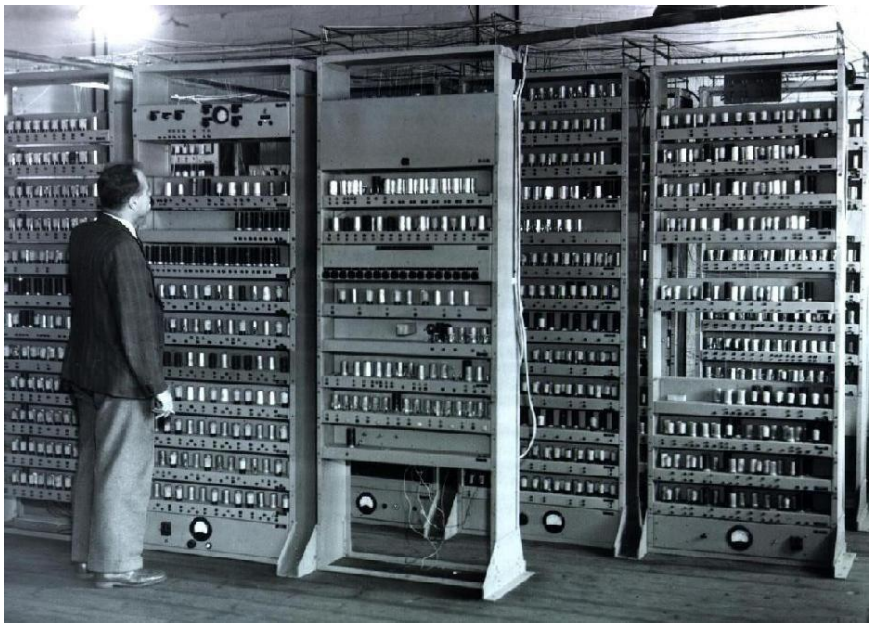
También llamada válvula electrónica o válvula de vacío es un componente electrónico utilizado para amplificar, conmutar, o modificar una señal eléctrica mediante el control del movimiento de los electrones en un espacio vacío a muy baja presión, o en presencia de gases especialmente seleccionados.

Los tubos de vacío posibilitaron el desarrollo de la electrónica durante la primera mitad del siglo XX, incluyendo la expansión de la radiodifusión, televisión, audio, redes telefónicas, computadoras analógicas y digitales.

El ocaso de esta tecnología comenzó con la invención del transistor y el posterior desarrollo de componentes de estado sólido que eran mucho más pequeños, baratos y fiables.



Ilustración 6 - Tubos de vacío.

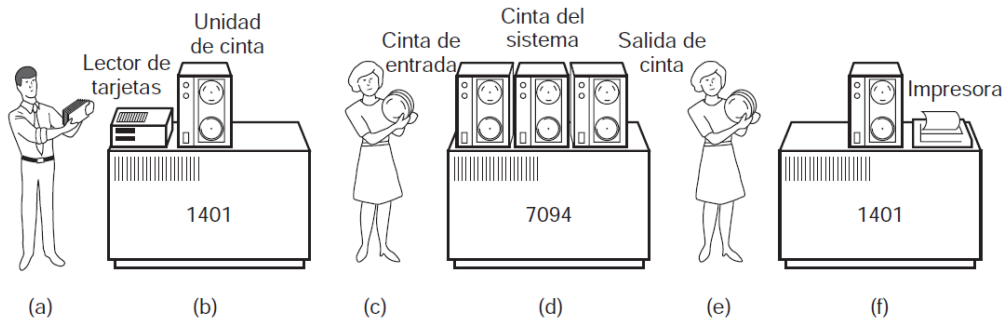


La segunda generación (1955 a 1965): transistores y sistemas de procesamiento por lotes.

La introducción del transistor a mediados de la década de 1950 cambió radicalmente el panorama. Las computadoras se volvieron lo bastante confiables como para poder fabricarlas y venderlas a clientes dispuestos a pagar por ellas.

Se utilizan lenguajes como FORTRAN o ensamblador. Pero el procedimiento de carga de los programas y obtención de los resultados sigue siendo tedioso y lento.

En esta generación aparece el ***sistema de procesamiento por lotes***.



Después de aproximadamente una hora de recolectar un lote de trabajos, las tarjetas se leían y se colocaban en una cinta magnética. Para estas tareas se utilizaban computadores manos potentes como la IBM 1401.

Luego, la cinta se llevaba al cuarto de máquinas, en donde se montaba en una unidad de cinta. Seguidamente, el operador cargaba un programa especial (el ancestro del sistema operativo de hoy en día), el cual leía sucesivamente cada trabajo y lo ejecutaba. Se utilizaban equipos

más potentes y costosos en esta etapa, como la IBM 7094[7]. Los resultados se escribían en una segunda cinta, en lugar de imprimirlos.

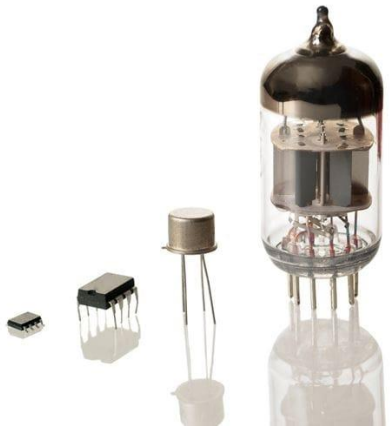
Cuando se terminaba de ejecutar todo el lote, el operador quitaba las cintas de entrada y de salida, reemplazaba la cinta de entrada con el siguiente lote y llevaba la cinta de salida a una IBM 1401 para imprimir fuera de línea (es decir, sin conexión con la computadora principal).



Transistor

El transistor es un dispositivo electrónico utilizado para amplificar o cambiar señales y potencias eléctricas. Es uno de los bloques básicos de la electrónica moderna.

Es el dispositivo electrónico básico que dio lugar a los circuitos integrados. Fueron desarrollados en 1947 por los físicos John Bardeen y Walter Brattain en los Laboratorios Bell.



[7] Estas computadoras utilizaban transistores en lugar de tubos de vacío, consiguiendo unas velocidades seis veces superiores que sus antecesoras.

La tercera generación (1965 a 1980): circuitos integrados y multiprogramación.

Se empezaron a utilizar ***circuitos integrados*** (ICs) (a pequeña escala), con lo cual se pudo ofrecer una mayor ventaja de precio/rendimiento en comparación con las máquinas de segunda generación, las cuales fueron construidas a partir de transistores individuales.

El sistema operativo más popular de aquella era fue el ***OS/360***[8], enorme y lleno de problemas, pero que pesar de todo dejaba razonablemente satisfechos a la mayoría de sus clientes.



Ilustración 7 - Computadora IBM S/360.

En esta generación aparece la ***multiprogramación*** en escena. Antes de la multiprogramación, cuando el trabajo actual se detenía para esperar a que se completara una operación con cinta u otro dispositivo de E/S, la CPU simplemente permanecía inactiva hasta terminar la operación de E/S. Con los cálculos científicos que requieren un uso intensivo de la CPU, la E/S no es frecuente, por lo que este tiempo desperdiciado no es considerable. Con el procesamiento de datos comerciales, el tiempo de espera de las operaciones de E/S puede ser a menudo de 80 a 90 por ciento del tiempo total, por lo que debía hacerse algo para evitar que la (costosa) CPU esté inactiva por mucho tiempo.

La solución que surgió fue particionar la memoria en varias piezas, con un trabajo distinto en cada partición. Mientras que un trabajo esperaba a que se completara una operación de E/S, otro podía estar usando la CPU. Si pudieran contenerse suficientes trabajos en memoria principal al mismo tiempo, la CPU podía estar ocupada casi 100 por ciento del tiempo. Para tener varios trabajos de forma segura en memoria a la vez, se requiere hardware especial para proteger cada trabajo y evitar que los otros se entrometan y lo malogren; el 360 y los demás sistemas de tercera generación estaban equipados con este hardware.

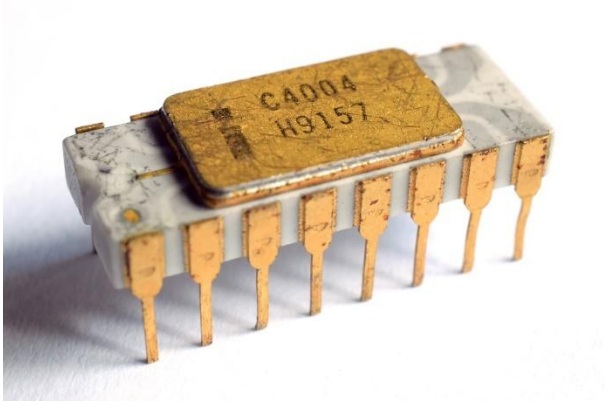
Aparece el ***Spooling,*** que es la capacidad para leer trabajos en tarjetas y colocarlos en el disco tan pronto como se llevaban al cuarto de computadoras. Así, cada vez que terminaba un trabajo en ejecución, el sistema operativo podía cargar un nuevo trabajo del disco en la partición que entonces estaba vacía y lo ejecutaba. También se utilizó para las operaciones de

[8] El OS/360 fue un sistema operativo producido por IBM entre 1965 y 1972.

salida. Con el spooling, las máquinas IBM 1401 no eran ya necesarias y desapareció la mayor parte del trabajo de transportar las cintas.

Tiempo compartido es una variante de la multiprogramación donde cada usuario tenía una terminal en línea. En un sistema de tiempo compartido, si 20 usuarios están conectados y 17 de ellos están pensando en dar un paseo o tomar café, la CPU se puede asignar por turno a los tres trabajos que desean ser atendidos.

Durante esta generación aparece en escena el sistema operativo ***UNIX***.

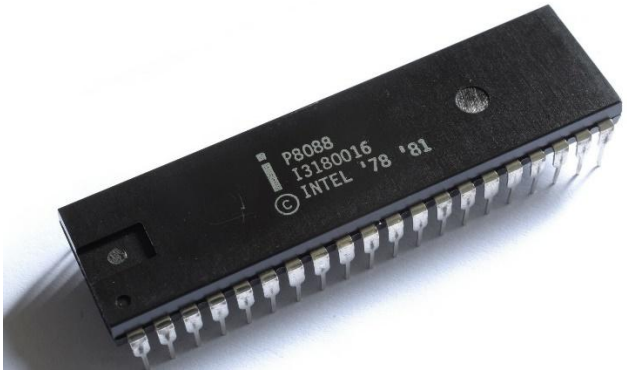


UNIX

Fue un sistema operativo que revolucionó el mercado tecnológico de la década de 1970. Unix fue desarrollado en 1969 por un grupo de empleados de los laboratorios Bell de AT&T. Sus características técnicas principales son su portabilidad, su capacidad multiusuario y multitarea, su eficiencia, su alta seguridad y su buen desempeño en tareas de red.

La cuarta generación (1980 a la fecha): las computadoras personales.

Con el chip, nació la era de la computadora personal. El chip microprocesador logró que un individuo tuviera su propia computadora personal.



Durante estos años aparece el sistema operativo ***CP/M***[9] y domina por un tiempo corto.

[9] CP/M (Control Program for Microcomputers) fue un sistema operativo monousuario y monotarea desarrollado por Gary Kildall para el microprocesador Intel 8080.

A principios de la década de 1980, IBM diseñó la ***IBM PC***[10] y buscó software para ejecutar en ella.



El principal sistema operativo para computadoras personales compatible con IBM PC en la década de 1980 fue ***MS-DOS*** (Microsoft Disk Operating System) que rápidamente llegó a dominar el mercado. Un factor clave fue la decisión de vender MS-DOS a las empresas de computadoras para que lo incluyeran con su hardware.

Otro hecho importante en esta generación es la invención de la Interfaz Gráfica de Usuario ***GUI***[11].

Se comercializa la primera ***Apple Macintosh***, estaba diseñada para los usuarios que no sólo no sabían nada acerca de las computadoras, sino que además no tenían ninguna intención de aprender.

Aparece el sistema operativo ***Windows*** de Microsoft. De 1985 a 1995, Windows fue sólo un entorno gráfico encima de MS-DOS. Sin embargo, a partir de 1995 se liberó una versión independiente de Windows, conocida como Windows 95, que incorporaba muchas características de los sistemas operativos y utilizaba el sistema MS-DOS subyacente sólo para iniciar y ejecutar programas de MS-DOS antiguos.

Surgen los derivados de UNIX, entre los que se destaca ***Linux***. Es un sistema operativo de código abierto que se utiliza en una gran variedad de entornos. Fue creado por Linus Torvalds en 1991 como un pasatiempo.

[10] El IBM Personal Computer fue la versión original y el progenitor de la plataforma de hardware compatible IBM PC.
[11] GUI es la abreviatura de graphical user interface (interfaz gráfica de usuario). Es un programa informático que permite a los usuarios interactuar con dispositivos electrónicos mediante imágenes y objetos gráficos.

Sistemas operativos de red y distribuidos

Un interesante desarrollo que surge a mediados de la década de 1980 es el crecimiento de las redes de computadoras personales que ejecutan sistemas operativos de red y sistemas operativos distribuidos.

Un sistema operativo de red permite la interconexión de computadoras para poder acceder a los servicios y recursos, hardware y software creando redes entre ellas.

Un sistema operativo distribuido utiliza los recursos computacionales en varios nodos de cálculo distintos para lograr un objetivo compartido común.

Referencias y bibliografía de la sección.

- Tanenbaum 1-2.

El lenguaje C.

Por lo general los sistemas operativos son extensos programas en ***C*** que consisten de muchas piezas escritas por muchos programadores. El entorno que se utiliza para desarrollar sistemas operativos es muy distinto a lo que están acostumbrados quienes desarrollan pequeños programas en lenguajes más modernos como Java[12].

El lenguaje C nació en los Laboratorios Bell de AT and T. Su historia está íntimamente ligada a la del sistema operativo UNIX, que empezó a desarrollarse a finales de los 60s usando ensamblador, un lenguaje de bajo nivel.

Para desarrollar aplicaciones para la nueva plataforma, se la dotó de un compilador de un lenguaje de programación de alto nivel, una versión sencilla del lenguaje ***BCPL***[13], a la que se denominó simplemente ***B***. De esta manera, se pudo contar con un compilador que traducía los programas al código máquina de un procesador específico, para facilitar la portabilidad de un sistema a otro.

El problema con B era que resultaba lento. Por este motivo a principios de la década de 1970, Dennis Ritchie, un miembro del equipo de desarrollo de UNIX, mejoró a B para que incluyera nuevas funcionalidades, como un tipo de dato de texto, y llamó a esta versión «nuevo B» o ***NB***.

NB comenzó a ser utilizado para programar UNIX en lugar de ensamblador. Este hecho impulsó el desarrollo del lenguaje y se añadieron nuevos tipos de datos, punteros a memoria

[12] Java es un lenguaje de programación orientado a objetos que se utiliza para desarrollar aplicaciones web, móviles y empresariales. Es multiplataforma, lo que significa que el código escrito en Java se puede ejecutar en diferentes sistemas sin necesidad de modificaciones.

[13] BCPL (Basic Combined Programming Language, Lenguaje de Programación Básico Combinado). Fue diseñado por Martin Richards de la Universidad de Cambridge en 1966 debido a las dificultades experimentadas con el lenguaje de programación CPL durante los años 60.

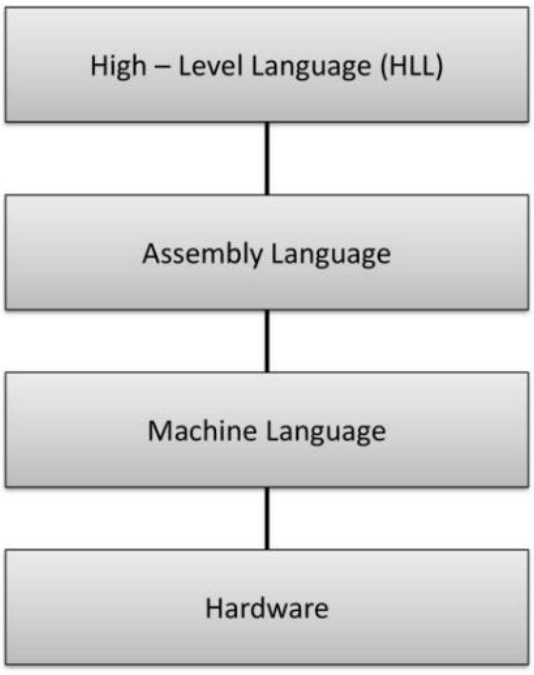
y arrays de muchos tipos. Como consecuencia, se creó un nuevo compilador y lenguaje se renombró como ***C***.

La versión 2 de UNIX, ya incluía el compilador de C y algunas utilidades. La 4, lanzada en 1973, fue reimplementada casi por completo en C. Este hecho aumento mucho la portabilidad del sistema operativo. La creación de compiladores C compatibles con otras maquinas harían de UNIX un sistema más fácil de adaptar.

A mediados de la década de 1980 ya había en el mercado numerosos compiladores C y muchas aplicaciones habían sido reescritas para aprovechar sus ventajas. Durante este periodo se introducen mejoras en el lenguaje, las cuales son recogidas por un comité de estandarización ANSI[14] y se establecen las especificaciones de lo que se conoce hoy en día como ***ANSI C***.

C es uno de los lenguajes de programación más aceptados por los programadores. Combina características de un lenguaje de alto nivel y lenguaje máquina. Podríamos decir que C se encuentra en la jerarquía de lenguajes en un nivel intermedio entre Pascal[15] y el Ensamblador. Pretende ser un lenguaje de alto nivel con la versatilidad del bajo nivel.

Históricamente se ha considerado a C, dentro de la jerarquía de los lenguajes, como un "lenguaje de más bajo nivel entre los de alto nivel". Mientras que lenguajes como Python o Java crean una capa de protección (máquina virtual o intérprete) entre el código y el procesador, C se mantiene "más cerca" del hardware.



[14] ANSI (American National Standards Institute, Instituto Nacional Estadounidense de Estándares) es una organización privada sin fines de lucro que se encarga de desarrollar y administrar normas y evaluaciones de la conformidad en Estados Unidos.

[15] Pascal es un lenguaje de programación creado por el profesor suizo Niklaus Wirth entre los años 1968 y 1969, y publicado en 1970. Su objetivo era crear un lenguaje que facilitara el aprendizaje de programación a sus alumnos, utilizando la programación estructurada y estructuración de datos.

La potencia del lenguaje C no admite discusión, es ampliamente usado en dispositivos integrados, y alimenta muchos servidores de internet, los cuales están construidos, mayormente, usando Linux.

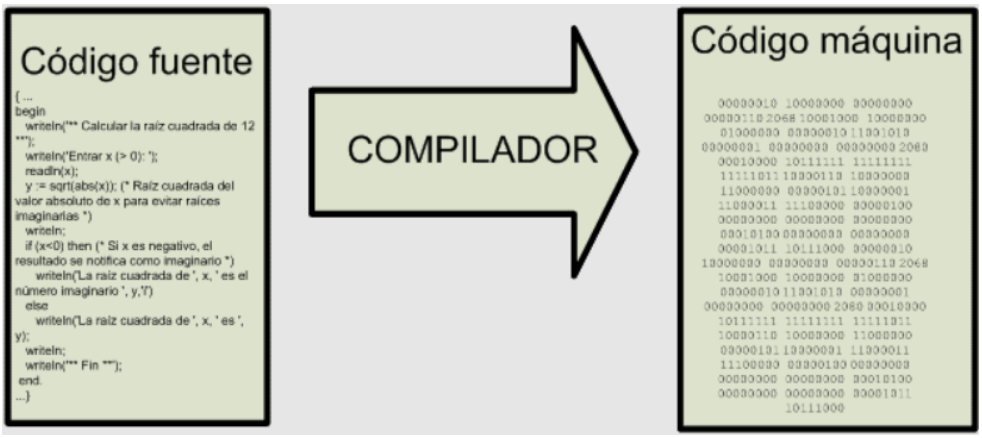
El Kernel de Linux está construido usando C, esto significa también que alimenta el núcleo de todos los dispositivos Android.

Cuando fue creado, C se consideraba un lenguaje de alto nivel, porque era portable entre máquinas. Ahora damos por sentado que podemos ejecutar un programa escrito en una Mac, Windows o Linux, usando tal vez Java o Python. Hace algún tiempo, esto simplemente no era el caso. Lo que C aportó en su momento fue un lenguaje simple de implementar y que tenía un compilador que podía ser fácilmente portado a diferentes máquinas.

C es un lenguaje compilado, como Go o Java. Otros lenguajes de programación populares como Python, PHP o JavaScript son interpretados.

La diferencia consiste entre un lenguaje compilable y otro interpretable es que el primero genera un archivo binario que puede ser directamente ejecutado y distribuido.

Cuando un programa en C es compilado, el resultado es un archivo ***binario*** que el procesador entiende de forma nativa. No hay traducción/interpretación mientras el programa corre. Por este motivo es que se suele decir que C es “rápido”.



C es un lenguaje muy utilizado por estudiantes para aprender programación. Sin embargo, no es un lenguaje académico. De hecho, no es una herramienta sencilla de manejar.

La velocidad de C no es gratuita; se paga con responsabilidad. Como este lenguaje no cuenta con las mismas capas de protección de otros lenguajes de más alto nivel, es muy fácil cometer errores que generen fallas de seguridad (como los famosos ***buffer overflows***[16]).

C no cuenta con un sistema de recolección de basura. Esto significa que tenemos que manejar la memoria por nuestra cuenta. Es una tarea compleja, y una que requiere mucha de nuestra

[16] Un buffer overflow (desbordamiento de búfer) es un fallo de seguridad que ocurre cuando un programa escribe más datos en un área de memoria (búfer) de los que esta puede almacenar.

atención para prevenir fallas. Sin embargo, esto también hace que C sea ideal para escribir programas para dispositivos integrados como Arduino.

C no esconde la complejidad y capacidades de la máquina subyacente como hacen otros lenguajes. Esto tiene sus beneficios evidentemente, aunque también plantea dificultades.

Con el tiempo, se adicionaron las características de la programación orientada a objetos y la denominación cambio a la de C++.

Referencias y bibliografía de la sección.

- Tanenbaum 1-8.
- FreecodeCamp - The C Beginner's Handbook.

Clasificación de los Sistemas Operativos.

Según el manejo de usuarios, procesadores y procesos.

Podríamos diferenciar a los sistemas operativos por el número de usuarios, el número de procesos o tareas y el número de procesadores con los que trabaja.

Sistemas operativos de un solo usuario.

Un sistema operativo de un solo usuario está diseñado para ejecutarse en una computadora que solo admite un usuario a la vez. Esto significa que solo un usuario puede iniciar sesión y acceder a los recursos y aplicaciones del sistema. Un sistema operativo de un solo usuario suele tener una interfaz sencilla y fácil de usar, como una interfaz gráfica de usuario o una interfaz de línea de comandos (CLI). Algunos ejemplos de sistemas operativos de un solo usuario son Windows, macOS y Linux (cuando se utiliza como sistema de escritorio).

Sistemas operativos multiusuario.

Un sistema operativo multiusuario está diseñado para ejecutarse en un servidor o una red que admite varios usuarios y tareas simultáneamente. Esto significa que varios usuarios pueden iniciar sesión y acceder a los recursos y aplicaciones del sistema, ya sea de forma local o remota. Un sistema operativo multiusuario suele tener una interfaz compleja y potente, como un shell. Algunos ejemplos de sistemas operativos multiusuario son Unix, Linux (cuando se utiliza como sistema de servidor) y Windows Server.

Sistemas operativos monotarea o monoprogramación.

En estos sistemas operativos solamente se puede ejecutar un proceso a la vez. Es una característica común en los sistemas operativos más antiguos. Ejemplo de este tipo de sistema son MS-DOS o Freedos.

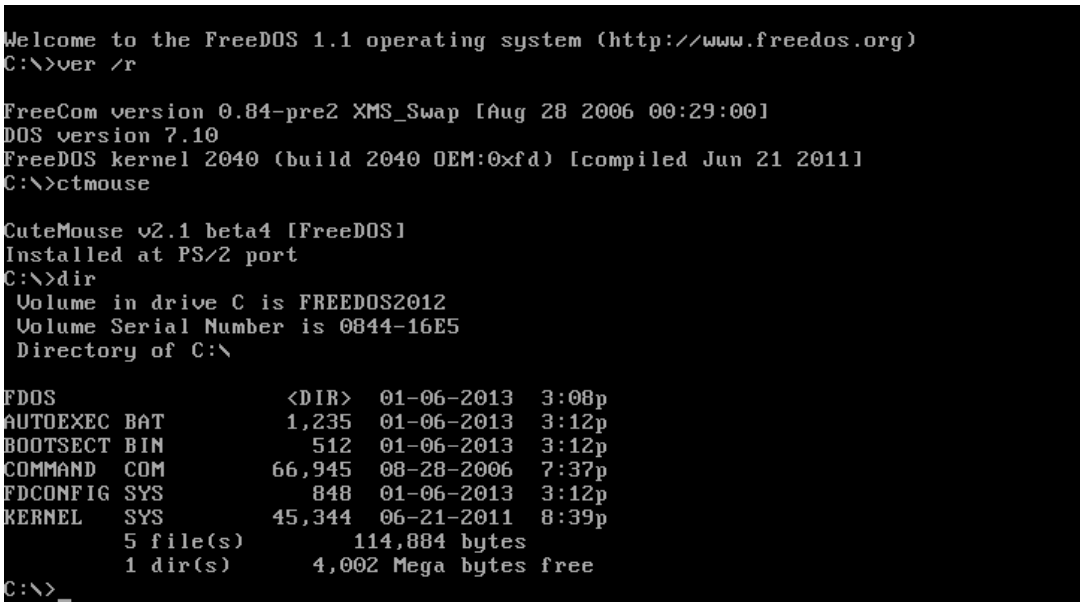


Ilustración 8 - El sistema operativo Freedos, podría considerarse un clon de MS-DOS.

Sistemas operativos multitarea o multiprogramación.

En estos sistemas varios procesos se ejecutan a la vez.

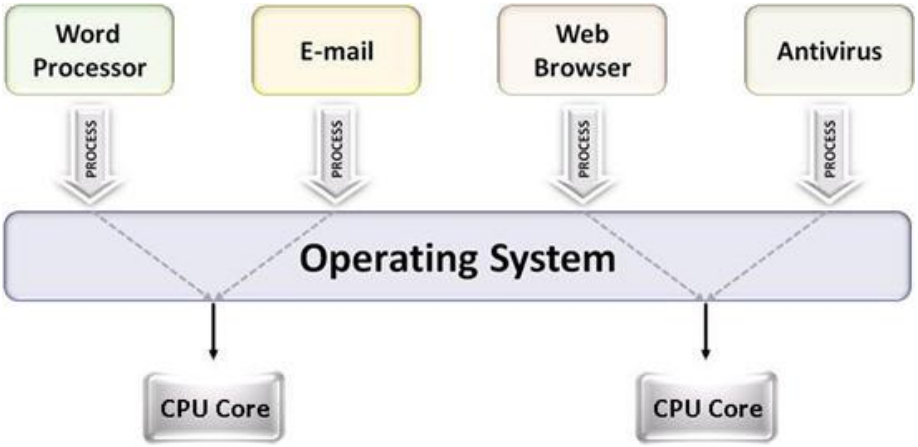
En los primeros tiempos de la computación los sistemas operativos solo podían manejar una tarea a la vez, lo que limitaba drásticamente la productividad. Con el avance de la tecnología la multitarea se ha convertido en una característica esencial. La mayoría de los sistemas operativos actuales son de este tipo.

UNIX, implementó una multitarea apropiativa[17], permitiendo que el sistema operativo interrumpiera y cambiara entre tareas de manera más eficiente. Este avance permitió una mayor estabilidad y rendimiento del sistema.

Windows, con su transición de Windows 3.x a Windows 95, incorporó multitarea y una mejor gestión de recursos, marcando un hito importante en la evolución de los sistemas operativos personales.

Algunos ejemplos de sistema operativos multitarea son Unix, Linux, MacOS, o Windows.

[17] La multitarea apropiativa (del inglés “preemptive multitasking”) es una manera en que los sistemas operativos pueden proveer multitarea. Con la multitarea apropiativa, el sistema operativo asigna tiempos de CPU a los procesos que se están ejecutando. Por cada núcleo de la CPU, se puede ejecutar a lo sumo un proceso en cada momento. Al proceso actual, el sistema le asigna un intervalo de tiempo para ejecutarse; una vez acabado el tiempo, el proceso queda pausado y se destina el siguiente intervalo de tiempo a un proceso distinto.



Sistemas operativos monoprocesador.

Los sistemas operativos monoprocesador son comunes en dispositivos más antiguos o en sistemas embebidos con recursos limitados. Aunque pueden no ser tan potentes como sus contrapartes multiprocesador, siguen siendo fundamentales para el funcionamiento de una amplia variedad de dispositivos y aplicaciones.

Uno de los principales beneficios de un sistema operativo monoprocesador es su simplicidad. Esto lo hace adecuado para dispositivos con requerimientos básicos de procesamiento, como electrodomésticos inteligentes, sistemas de control industrial.

Los sistemas operativos basados en DOS suelen ser monoprocesador.

Sistemas operativos multiprocesador.

Hay varios procesadores en la computadora. Estos sistemas operativos son capaces de usarlos a todos para distribuir la carga de trabajo. Generalmente trabajan de dos formas: simétrica o asimétricamente.

En el multiprocesamiento simétrico (SMP) los procesadores son idénticos y comparten una memoria principal. Cada procesador tiene acceso a la memoria y a los dispositivos de entrada/salida (I/O). Además, los procesos se distribuyen de forma simétrica entre los procesadores consiguiendo que la carga computacional se divida de manera uniforme entre todos los recursos. La mayoría de los sistemas multiprocesador actuales utilizan esta arquitectura.

En el multiprocesamiento asimétrico uno o más procesadores se dedican a tareas específicas, como ejecutar el sistema operativo, mientras que los procesadores restantes se dedican a otras tareas, como las aplicaciones del usuario. Este tipo de sistema se suele utilizar en sistemas embebidos.

Algunos ejemplos de sistemas que utilizan multiprocesamiento asimétrico son:

- Robótica automatizada.
- Dispositivos de Internet de las cosas (***IoT***).
- Dispositivos de red, como enrutadores y conmutadores.
- Consolas de juegos.

Según su propósito o destino.

Los sistemas operativos han estado entre nosotros durante más de medio siglo. Durante este tiempo se ha desarrollado una variedad bastante extensa de ellos, no todos se conocen ampliamente.

Sistemas operativos de mainframe.

Los sistemas operativos para las mainframes funcionan en computadoras del tamaño de un cuarto completo. Aún se los puede encontrar en centros de datos corporativos.

Se caracterizan por su seguridad, escalabilidad y capacidad de recuperación.



Podemos encontrar estos sistemas operativos en ámbitos computacionales como servidores Web de alto rendimiento o servidores para sitios de comercio electrónico a gran escala.

Como ejemplo podemos mencionar a IBM z/OS. Es un sistema operativo para mainframes, adecuado para operaciones continuas y de gran volumen con gran seguridad y estabilidad.

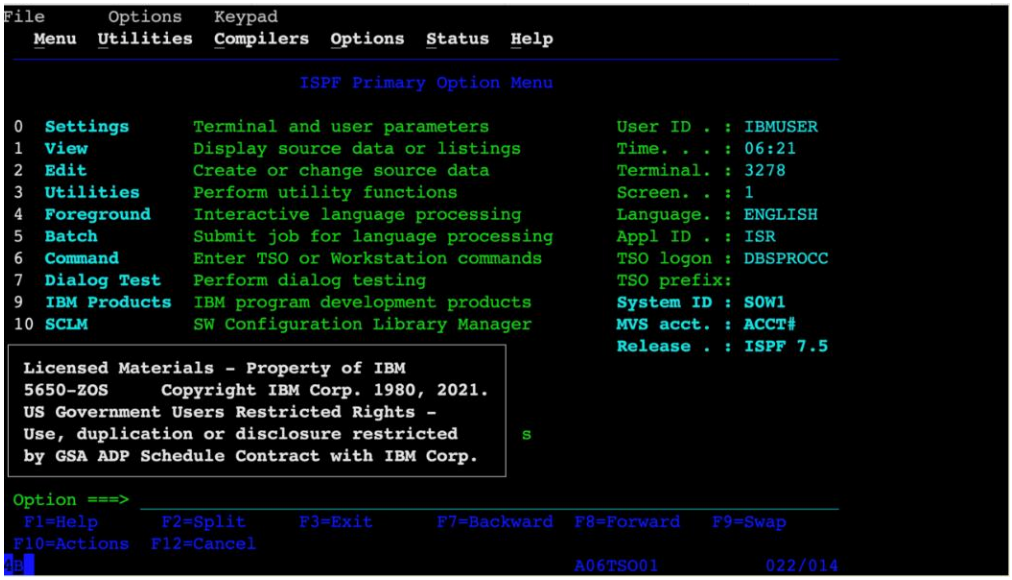


Ilustración 9 - Apariencia de la interfaz del IBM z/OZ.

Sistemas operativos de servidores.

Se ejecutan en servidores, que solían ser computadoras personales muy potentes o incluso mainframes.

Dan servicio a varios usuarios a la vez a través de una red y les permiten compartir los recursos de hardware y de software.

Estos sistemas están optimizados para ofrecer características y funciones más avanzadas para las arquitecturas cliente-servidor o los entornos informáticos empresariales. Además, están orientados a la red y no a un solo usuario, por lo que pueden gestionar múltiples escritorios, facilitar la administración y reducir el tiempo de inactividad.



Varias distribuciones de Linux ofrecen versiones especialmente optimizadas para servidores. Microsoft por su parte desarrolla Windows Server como sistema operativo pensado para estos fines.

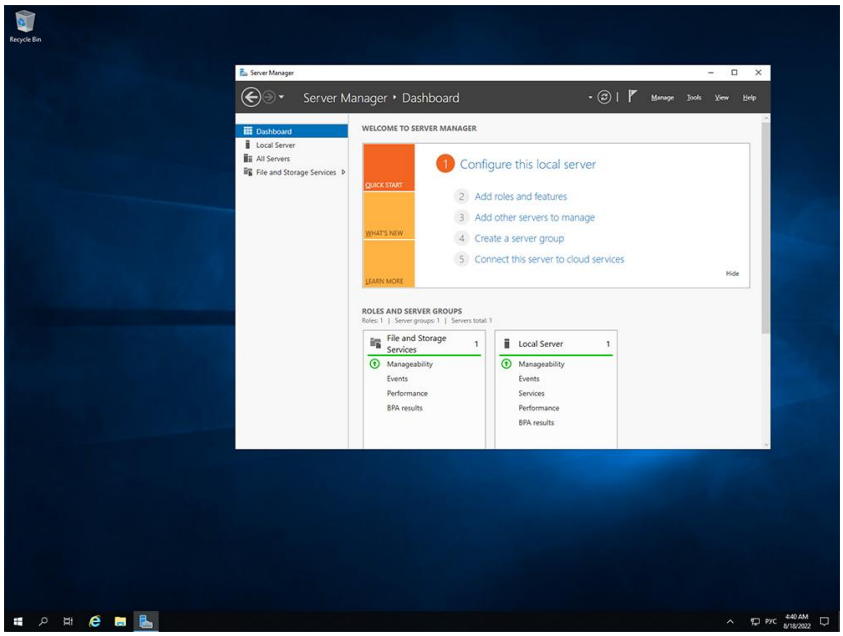


Ilustración 10 - Windows Sever.

Sistemas operativos de computadoras personales.

Diseñados para ser instalados en computadoras personales. Se utilizan ampliamente para el procesamiento de texto, las hojas de cálculo y el acceso a Internet. Son los sistemas operativos que resultan más familiares para el usuario promedio. Algunos ejemplos comunes son Linux, FreeBSD, Windows.

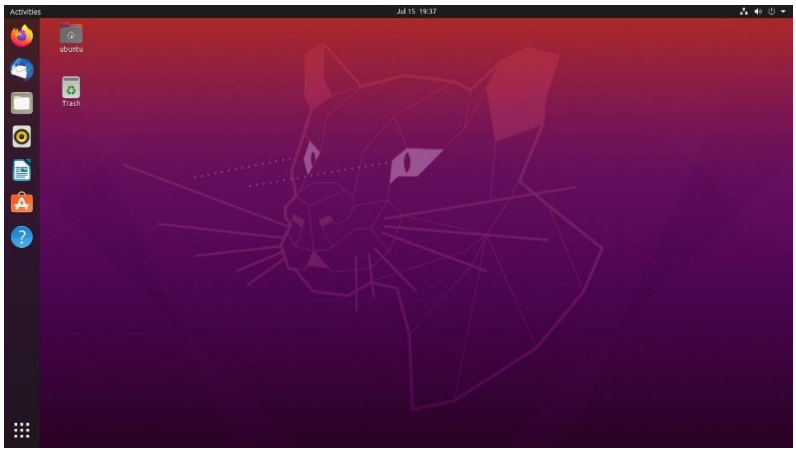


Ilustración 11 - Escritorio de Ubuntu 20.04.

Sistemas operativos integrados.

Los sistemas integrados (embedded), que también se conocen como incrustados o embebidos, operan en dispositivos dedicados, como:

- Automóviles.
- Semáforos.
- Televisores digitales.

- Cajeros automáticos.
- Controles de aviones.
- Terminales de punto de venta.
- Cámaras digitales.
- Sistemas de navegación GPS.
- Ascensores.

Los sistemas operativos integrados suelen tener aplicaciones limitadas y están diseñados para ejecutarse rápidamente, no fallar y manejar errores.

Algunos electrodomésticos modernos cuentan con este tipo de sistemas. Todo el software se encuentra en ROM y están basados en un microprocesador o un microcontrolador diseñado para realizar algunas pocas funciones específicas.

VxWorks[18] es un buen ejemplo de sistema operativo integrado.

En algunos casos, los sistemas instalados en los teléfonos móviles inteligentes son considerados sistemas operativos integrados. Actualmente podríamos considerar a los sistemas diseñados para funcionar en smartphones como una categoría aparte.

Sistemas operativos móviles.

Al igual que los computadores que utilizan Windows, Linux o MacOS, los dispositivos móviles tienen sus sistemas operativos. Los sistemas operativos móviles son mucho más simples, al menos por ahora. Están más orientados a la conectividad inalámbrica y los formatos multimedia.

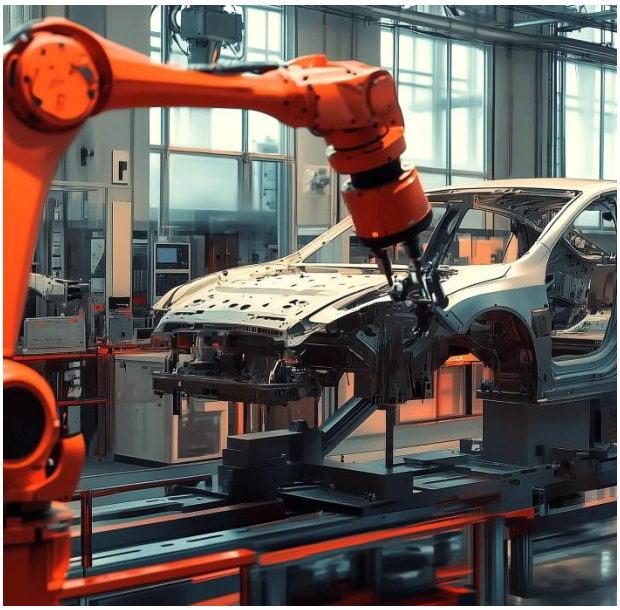
Android y iOS son los sistemas operativos móviles más populares en la actualizad.



[18] VxWorks es un sistema operativo de tiempo real desarrollado como software propietario por Wind River Systems. Es utilizado para sistemas embebidos que habitualmente necesitan una respuesta rápida del orden de microsegundos ante interrupciones en su funcionamiento, una reconocida estabilidad y una seguridad certificada.

Sistemas operativos en tiempo real

Estos sistemas se caracterizan por tener el tiempo como un parámetro clave. Por ejemplo, en los sistemas de control de procesos industriales, las computadoras en tiempo real tienen que recolectar datos acerca del proceso de producción y utilizarlos para controlar las máquinas en la fábrica. A menudo hay tiempos de entrega estrictos que se deben cumplir.



Un sistema operativo en tiempo real debe garantizar que las tareas se ejecuten dentro de un lapso de tiempo determinado.

QNX es un ejemplo de sistema operativo que ha sido desarrollado para aplicaciones de tiempo real.

Referencias y bibliografía de la sección.

- Tanenbaum 1-4.

Revisión del hardware de una computadora.

Un sistema operativo está íntimamente relacionado con el hardware de la computadora sobre la que se ejecuta. Extiende el conjunto de instrucciones de la computadora[19] y administra sus recursos.

Conceptualmente, una computadora personal simple se puede abstraer mediante un modelo en el que la CPU, la memoria y los dispositivos de E/S están conectados mediante un bus del sistema y se comunican entre sí a través de este bus. Vale aclarar. Que las computadoras personales modernas tienen una estructura más complicada en la que intervienen varios buses.

Procesadores

La CPU obtiene las instrucciones de la memoria y las ejecuta. El ciclo básico de toda CPU es obtener la primera instrucción de memoria, decodificarla para determinar su tipo y operandos[20], ejecutarla y después obtener, decodificar y ejecutar las instrucciones subsiguientes. El ciclo se repite hasta que el programa termina. De esta forma se ejecutan los programas.

Cada CPU tiene un conjunto específico de instrucciones que puede ejecutar. Así, un Pentium no puede ejecutar programas de SPARC[21] y un SPARC no puede ejecutar programas de Pentium.

Para mejorar el rendimiento, los diseñadores de CPUs abandonaron desde hace mucho tiempo el modelo de obtener, decodificar y ejecutar una instrucción a la vez. Muchas CPUs modernas cuentan con medios para ejecutar más de una instrucción al mismo tiempo. Por ejemplo, una CPU podría tener unidades separadas de obtención, decodificación y ejecución, de manera que mientras se encuentra ejecutando la instrucción n, también podría estar decodificando la instrucción n 1 y obteniendo la instrucción n 2. A dicha organización se le conoce como canalización (***pipeline***). El uso de canalizaciones más grandes es común.

Las canalizaciones producen grandes dolores de cabeza a los programadores de compiladores y de sistemas operativos, ya que quedan al descubierto las complejidades de la máquina subyacente.

Aún más avanzada que el diseño de una canalización es la CPU ***superescalar***. En este diseño hay varias unidades de ejecución; por ejemplo, una para la aritmética de enteros, una para la aritmética de punto flotante y otra para las operaciones Booleanas.

---

[19] El conjunto de instrucciones de la computadora son las instrucciones que una unidad central de procesamiento puede entender y ejecutar, o el conjunto de todos los comandos implementados por un diseño particular de una CPU.

[20] Los operandos son datos que se utilizan en una operación aritmética de la CPU.

[21] SPARC (del inglés Scalable Processor ARChitecture) es una arquitectura RISC big-endian. Es decir, una arquitectura con un conjunto de instrucciones reducidas.

Dos o más instrucciones se obtienen a la vez, se decodifican y se vacían en un búfer[22] de contención hasta que puedan ejecutarse. Tan pronto como una unidad de ejecución se encuentre libre, busca en el búfer de contención para ver si hay una instrucción que pueda manejar; de ser así, saca la instrucción del búfer y la ejecuta. Una consecuencia de este diseño es que con frecuencia las instrucciones del programa se ejecutan en forma desordenada. En gran parte, es responsabilidad del hardware asegurarse de que el resultado producido sea el mismo que hubiera producido una implementación secuencial, pero una cantidad molesta de complejidad es impuesta al sistema operativo.



Como ya vimos, la mayoría de las CPU, con excepción de las extremadamente simples que se utilizan en los sistemas integrados[23], tienen dos modos: modo kernel y modo usuario. Al operar en modo kernel, la CPU puede ejecutar cualquier instrucción de su conjunto de instrucciones y utilizar todas las características del hardware. Cuando el sistema operativo opera en modo kernel, tiene acceso al hardware completo.

En contraste, los programas de usuario operan en modo de usuario, el cual les permite ejecutar sólo un subconjunto de las instrucciones y les da acceso sólo a un subconjunto de las características.

Para obtener servicios del sistema operativo, un programa usuario debe lanzar una llamada al sistema (system call), la cual se atrapa en el kernel e invoca al sistema operativo.

Chips con multihilamiento y multinúcleo

La ley de Moore[24] establecía que el número de transistores en un chip se duplicaría cada 18 meses.

Las arquitecturas superescalares, con múltiples unidades funcionales son una forma de enfocar el desafío del incremento de transistores en los procesadores.

Otra alternativa es colocar cachés más grandes en el chip de la CPU, pero en cierto momento se llega al punto de rendimiento decreciente.

El siguiente paso obvio es multiplicar no sólo las unidades funcionales, sino también parte de la lógica de control. Algunas CPU cuenta con una propiedad conocida como multihilamiento (multithreading) o hiperhilamiento (hyperthreading).

---

[22] Un búfer (del inglés, buffer) es un espacio de memoria, en el que se almacenan datos de manera temporal, normalmente para un único uso.

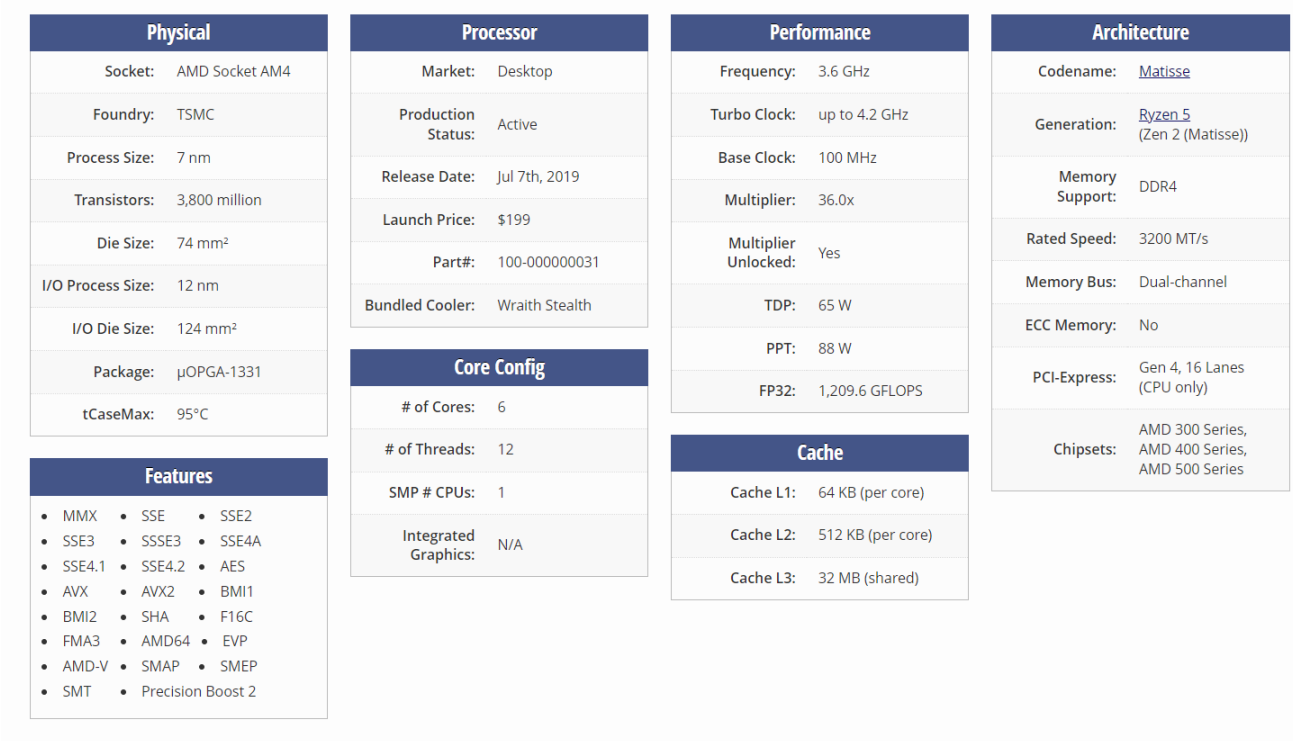
[23] Un sistema embebido (también, sistema embarcado o sistema empotrado, sistema integrado, sistema incrustado) es un sistema de computación basado en un microprocesador o un microcontrolador diseñado para realizar una o algunas pocas funciones dedicadas.

[24] Se trata de una ley empírica, formulada por el cofundador de Intel, Gordon E. Moore, el 19 de abril de 1965, cuyo cumplimiento se ha podido constatar hasta mediados de la década de 2010.

El multihilamiento permite que la CPU contenga el estado de dos hilos[25] de ejecución (threads) distintos y luego alterne entre uno y otro con una escala de tiempo en nanosegundos. Por ejemplo, si uno de los procesos necesita leer una palabra de memoria (que requiere muchos ciclos de reloj), una CPU con multihilamiento puede cambiar a otro hilo. El multihilamiento no ofrece un verdadero paralelismo. Sólo hay un proceso en ejecución a la vez, pero el tiempo de cambio entre un hilo y otro se reduce al orden de un nanosegundo.

El multihilamiento tiene consecuencias para el sistema operativo, debido a que cada hilo aparece para el sistema operativo como una CPU separada. Considere un sistema con dos CPU reales, cada una con dos hilos. El sistema operativo verá esto como si hubiera cuatro CPU. Si hay suficiente trabajo sólo para mantener ocupadas dos CPU en cierto punto en el tiempo, podría planificar de manera inadvertida dos hilos en la misma CPU, mientras que la otra CPU estaría completamente inactiva. Esta elección es mucho menos eficiente que utilizar un hilo en cada CPU.

Más allá del multihilamiento, tenemos chips de CPU con dos, cuatro o más procesadores completos, o núcleos (cores) en su interior. Cada uno con su propia CPU independiente. Para hacer uso de dicho chip multinúcleo se requiere en definitiva un sistema operativo multiprocesador.



Physical

Socket:	AMD Socket AM4
Foundry:	TSMC
Process Size:	7 nm
Transistors:	3,800 million
Die Size:	74 mm²
I/O Process Size:	12 nm
I/O Die Size:	124 mm²
Package:	µOPGA-1331
tCaseMax:	95°C

Features

- MMX • SSE • SSE2
- SSE3 • SSSE3 • SSE4A
- SSE4.1 • SSE4.2 • AES
- AVX • AVX2 • BMI1
- BMI2 • SHA • F16C
- FMA3 • AMD64 • EVP
- AMD-V • SMAP • SMEP
- SMT • Precision Boost 2

Processor

Market:	Desktop
Production Status:	Active
Release Date:	Jul 7th, 2019
Launch Price:	$199
Part#:	100-000000031
Bundled Cooler:	Wraith Stealth

Core Config

# of Cores:	6
# of Threads:	12
SMP # CPUs:	1
Integrated Graphics:	N/A

Performance

Frequency:	3.6 GHz
Turbo Clock:	up to 4.2 GHz
Base Clock:	100 MHz
Multiplier:	36.0x
Multiplier Unlocked:	Yes
TDP:	65 W
PPT:	88 W
FP32:	1,209.6 GFLOPS

Cache

Cache L1:	64 KB (per core)
Cache L2:	512 KB (per core)
Cache L3:	32 MB (shared)

Architecture

Codename:	Matisse
Generation:	Ryzen 5 (Zen 2 (Matisse))
Memory Support:	DDR4
Rated Speed:	3200 MT/s
Memory Bus:	Dual-channel
ECC Memory:	No
PCI-Express:	Gen 4, 16 Lanes (CPU only)
Chipsets:	AMD 300 Series, AMD 400 Series, AMD 500 Series

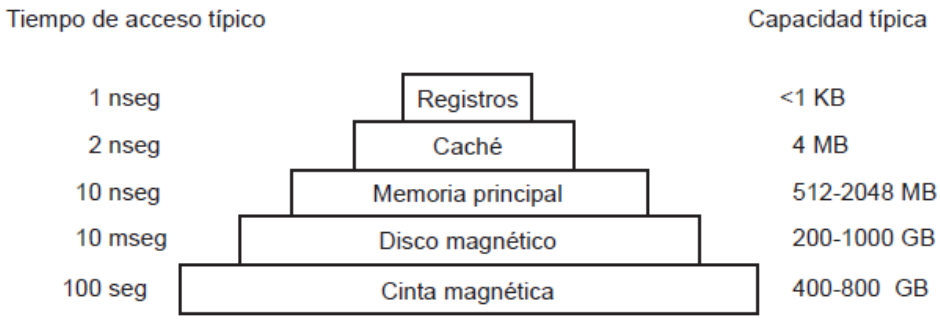
Ilustración 12 - Especificaciones AMD Ryzen 5 3600

[25] Un hilo o hebra (del inglés thread), proceso ligero o subproceso es una secuencia de tareas encadenadas muy pequeña que puede ser ejecutada por un sistema operativo.

Memoria

En teoría, una memoria debe ser en extremo rápida (más rápida que la velocidad de ejecución de una instrucción, de manera que la memoria no detenga a la CPU), de gran tamaño y muy económica. Ninguna tecnología en la actualidad cumple con todos estos objetivos, por lo que se adopta una solución distinta. El sistema de memoria está construido como una jerarquía de capas.

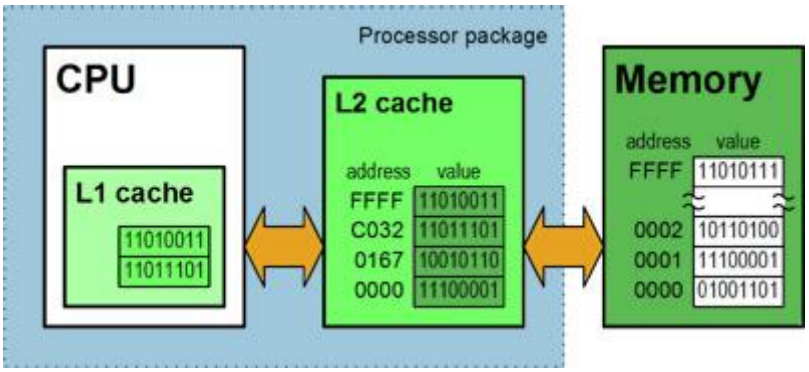
Las capas superiores tienen mayor velocidad, menor capacidad y mayor costo por bit que las capas inferiores.



La capa superior consiste en los registros internos de la CPU. Están compuestos del mismo material que la CPU y, por ende, tienen la misma rapidez. En consecuencia, no hay retraso a la hora de utilizarlos. Los programas deben administrar los registros (es decir, decidir qué deben guardar en ellos) por su cuenta, en el software.

El siguiente nivel es la memoria caché, que el hardware controla de manera parcial. Las líneas de caché que se utilizan con más frecuencia se mantienen en una caché de alta velocidad, ubicada dentro o muy cerca de la CPU. Cuando el programa necesita leer una palabra de memoria[26], el hardware de la caché comprueba si la línea que se requiere se encuentra en la caché. Si es así (a lo cual se le conoce como acierto de caché), la petición de la caché se cumple y no se envía una petición de memoria a través del bus hacia la memoria principal. Los fallos de caché, en cambio, deben realizar una petición de memoria, con un castigo considerable de tiempo.

[26] Una palabra es una cadena finita de bits que son manejados como un conjunto por la máquina. El tamaño o longitud de una palabra hace referencia al número de bits contenidos en ella, y es un aspecto muy importante al momento de diseñar una arquitectura de computadoras. Las computadoras modernas normalmente tienen un tamaño de palabra de 16, 32 o 64 bits.



La memoria caché está limitada en tamaño debido a su alto costo. Algunas máquinas tienen dos o incluso tres niveles de caché, cada una más lenta y más grande que la anterior.

El uso de cachés juega un papel importante en muchas áreas de las ciencias computacionales. Cada vez que hay un recurso extenso que se puede dividir en piezas, algunas de las cuales se utilizan con mucha más frecuencia que otras, a menudo se invoca a la caché para mejorar el rendimiento. Los sistemas operativos la utilizan todo el tiempo.

Por ejemplo, la mayoría de los sistemas operativos mantienen los archivos (o parte de ellos) que se utilizan con frecuencia en la memoria principal para evitar tener que obtenerlos la unidad de almacenamiento en forma repetida. De manera similar, los resultados de convertir nombres de rutas extensas a la dirección en disco donde se encuentra el archivo, se pueden colocar en la caché para evitar búsquedas repetidas.

Las cachés son una idea tan útil que las CPUs modernas tienen varias de ellas. Comúnmente la caché L1 o de primer nivel está siempre dentro de la CPU. La mayoría de los chips tienen una segunda caché L1 para las palabras de datos que se utilizan con frecuencia. Además, a menudo hay una segunda caché, conocida como caché L2, que contiene varios megabytes de palabras de memoria utilizadas recientemente. La diferencia entre las cachés L1 y L2 está en la velocidad. El acceso a la caché L1 se realiza sin ningún retraso, mientras que el acceso a la caché L2 requiere un retraso de uno o dos ciclos de reloj.

La memoria principal es el “caballo de batalla” del sistema de memoria. Por lo general a la memoria principal se le conoce como RAM (Random Access Memory, Memoria de Acceso Aleatorio).

En la actualidad, las memorias contienen desde cientos de megabytes hasta varios gigabytes y su tamaño aumenta con rapidez. Todas las peticiones de la CPU que no se puedan satisfacer desde la caché pasan a la memoria principal.

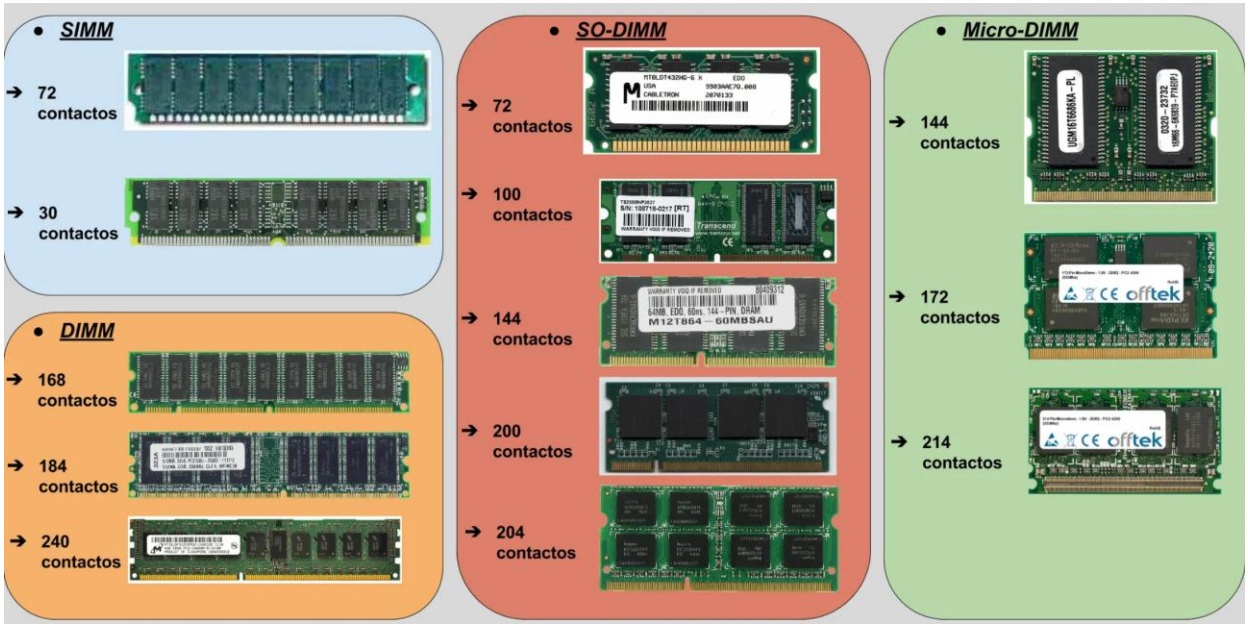


Ilustración 13 - Zócalos de memoria

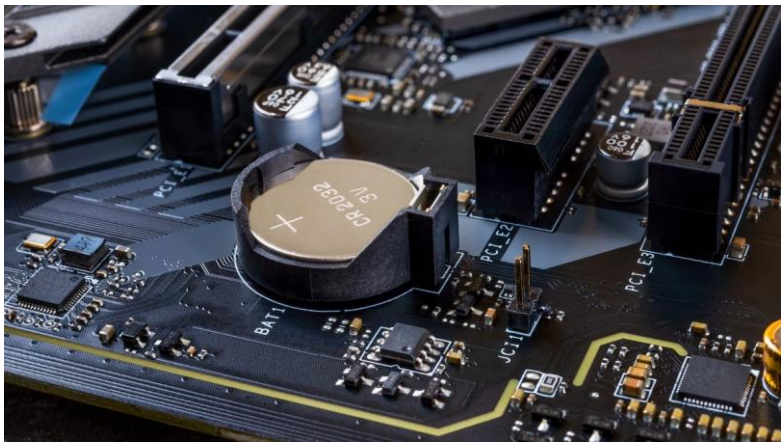
Además de la memoria principal, muchas computadoras tienen una pequeña cantidad de memoria de acceso aleatorio no volátil. A diferencia de la RAM, la memoria no volátil no pierde su contenido cuando se desconecta la energía. La ROM (Read Only Memory, Memoria de sólo lectura) se programa en la fábrica y no puede modificarse después. Es rápida y económica. En algunas computadoras, el cargador de arranque[27] (bootstrap loader) que se utiliza para iniciar la computadora está contenido en la ROM.



La EEPROM (Electrically Erasable PROM, PROM eléctricamente borrable) y la memoria flash son también no volátiles, pero en contraste con la ROM se pueden borrar y volver a escribir datos en ellas. Sin embargo, para escribir en este tipo de memorias se requiere mucho más tiempo que para escribir en la RAM, por lo cual se utilizan en la misma forma que la ROM, sólo con la característica adicional de que ahora es posible corregir los errores en los programas que contienen, mediante la acción de volver a escribir datos en ellas en el campo de trabajo.

[27] El cargador de arranque bootstrap es el programa que se ejecuta al encender o reiniciar una computadora. Se encarga de inicializar el hardware y cargar el sistema operativo.

Otro tipo más de memoria es CMOS, la cual es volátil. Muchas computadoras utilizan memoria CMOS para guardar la fecha y hora actuales. La memoria CMOS y el circuito de reloj que incrementa la hora en esta memoria están energizados por una pequeña batería, por lo que la hora se actualiza en forma correcta aun cuando la computadora se encuentre desconectada. La memoria CMOS también puede contener los parámetros de configuración, como el disco del cual se debe iniciar el sistema. Se utiliza CMOS debido a que consume tan poca energía que la batería instalada en la fábrica dura varios años.



Dispositivos de almacenamiento

El almacenamiento de datos se realiza en dispositivos de hardware que disponen de características como el rendimiento de lectura/escritura, la capacidad de almacenamiento, la fiabilidad, etc. Estas características definen cual es el uso más adecuado para cada tipo de dispositivos de almacenamiento.

- Capacidad: Mide la cantidad de datos que puede almacenar el sistema de almacenamiento, y es medida en bytes.
- Rendimiento: Velocidad y eficiencia del sistema de almacenamiento de datos.
- Fiabilidad: Es la disponibilidad de los datos cuando son solicitados, así como el hecho de disponer de una baja tasa de errores o fallos (por ejemplo, utilizando una configuración RAID[28]).
- Recuperabilidad: Capacidad del sistema para recuperar datos tras una pérdida, borrado, corrupción o cualquier otro incidente que impida el acceso a los mismos.

Si bien existen muchos tipos de dispositivos de almacenamiento como cintas magnéticas y discos flexibles, nos centraremos en los discos duros mecánicos (HDD) y en unides de almacenamiento de estado sólido.

[28] RAID es una tecnología de almacenamiento de datos que combina varios discos duros en una sola unidad lógica. El acrónimo RAID proviene de las siglas en inglés de Redundant Array of Independent Disks (matriz redundante de discos independientes).



Discos

El almacenamiento en disco es más económico que la RAM, y casi siempre de mayor tamaño. El único problema es que el tiempo para acceder en forma aleatoria a los datos es mucho mayor. Esta baja velocidad se debe al hecho de que un disco es un dispositivo mecánico.



Un disco consiste en uno o más platos que giran generalmente a 5400, 7200 o 10,800 rpm. Un brazo mecánico, con un punto de giro colocado en una esquina, se mueve sobre los platos de manera similar al brazo de la aguja en un viejo tocadiscos. La información se escribe en el disco en una serie de círculos concéntricos. En cualquier posición dada del brazo, cada una de las cabezas puede leer una región anular conocida como pista (track). En conjunto, todas las pistas para una posición dada del brazo forman un cilindro (cylinder).

Cada pista se divide en cierto número de sectores. En los discos modernos, los cilindros exteriores contienen más sectores que los interiores.

Una vez que el brazo se encuentra en la pista correcta, la unidad debe esperar a que el sector necesario se encuentre debajo del cabezal. Una vez que el sector está bajo el cabezal, se realiza la lectura o escritura.

Muchas computadoras presentan un esquema conocido como memoria virtual (virtual memory). Este esquema hace posible la ejecución de programas más grandes que la memoria física al colocarlos en el disco y utilizar la memoria principal como un tipo de caché para las partes que se ejecutan con más frecuencia. Este esquema requiere la reasignación de direcciones de memoria al instante, para convertir la dirección que el programa generó en la dirección física en la RAM en donde se encuentra la palabra. Esta asignación se realiza mediante una parte de la CPU conocida como ***MMU*** (Memory Management Unit, Unidad de Administración de Memoria).

La presencia de la caché y la MMU pueden tener un gran impacto en el rendimiento. En un sistema de multiprogramación, al cambiar de un programa a otro (lo que se conoce comúnmente como cambio de contexto o context switch), puede ser necesario vaciar todos los bloques modificados de la caché y alterar los registros de asignación en la MMU. Ambas operaciones son costosas y los programadores se esfuerzan bastante por evitarlas.

Almacenamiento de estado solido

El almacenamiento de estado sólido es un método de almacenamiento de datos creado mediante dispositivos de circuitos integrados para almacenar datos en lugar de utilizar soportes ópticos o magnéticos móviles. Suele ser no volátil y puede tomar varias formas como una tarjeta de estado sólido o una unidad de 2.5 pulgadas. Además, se incluyen opciones de interfaz[29] PATA, SATA, SAS o PCIe.



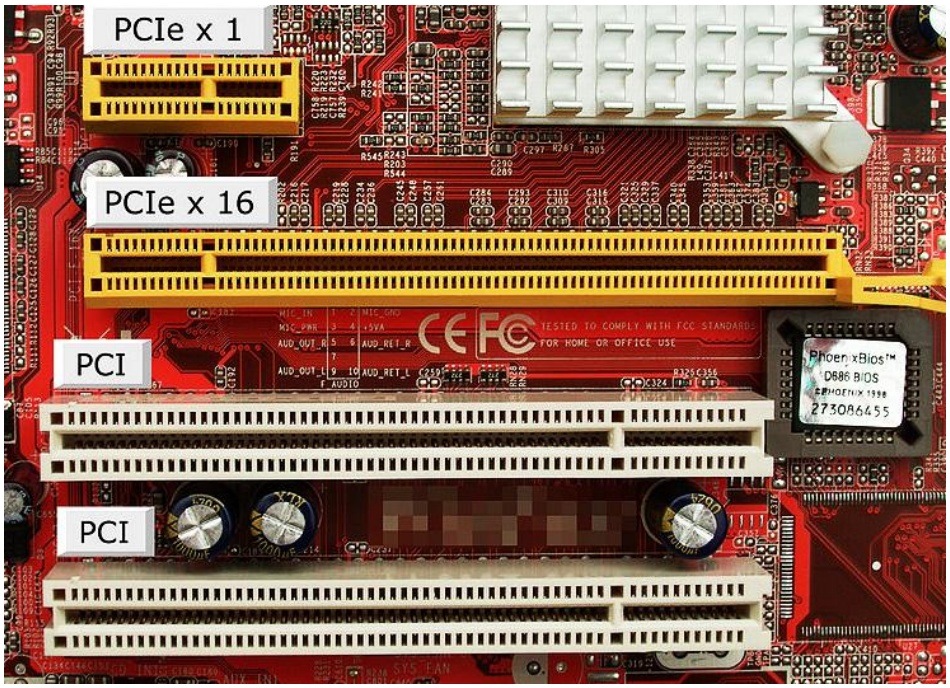
Las unidades de estado sólido (SSD) son dispositivos de almacenamiento de datos que utilizan tecnología de memoria flash estática en lugar de discos magnéticos giratorios o soportes ópticos. Las unidades SSD son compatibles con las interfaces de disco duro tradicionales, como SATA o SAS, y tienen un formato de disco duro familiar, como 3,5; 2,5 o 1,8 pulgadas y por lo general, utilizan memoria flash NAND.

[29] La interfaz de almacenamiento es el punto de conexión entre un dispositivo de almacenamiento y la placa base de una computadora. Se encarga de definir cómo se transfieren los datos entre ambos componentes.

La memoria flash es una memoria no volátil regrabable. Al contrario que la DRAM, es necesario borrar bloques de datos antes de poder escribir en ellos, lo que hace que el rendimiento de escritura sea menor que el de lectura. La memoria flash sólo admite un número finito de escrituras y el número varía según la tecnología utilizada.

Los SSD M.2 ***SATA***[30] emplean la interfaz SATA con una velocidad máxima de transferencia de datos de 6 Gbps. Los SSD basados en SATA son los SSD de menor categoría en cuanto a rendimiento, y utilizan la misma interfaz que los discos duros. Con todo, los SSD basados en SATA tienen entre tres y cuatro veces el ancho de banda en comparación con los discos duros giratorios. En muchas ocasiones, las delgadas computadoras portátiles (incluyendo los ultrabooks) no tienen espacio para los cables SATA, motivo por el cual utilizan el factor de forma M.2.

Los SSD M.2 NVMe utilizan el protocolo ***NVMe***, diseñado especialmente para las unidades SSD. Combinado con el bus ***PCIe***[31], un SSD NVMe se caracteriza por mayores niveles de rendimiento y velocidad. Los SSD NVMe se comunican directamente con la CPU del sistema utilizando los conectores PCIe.

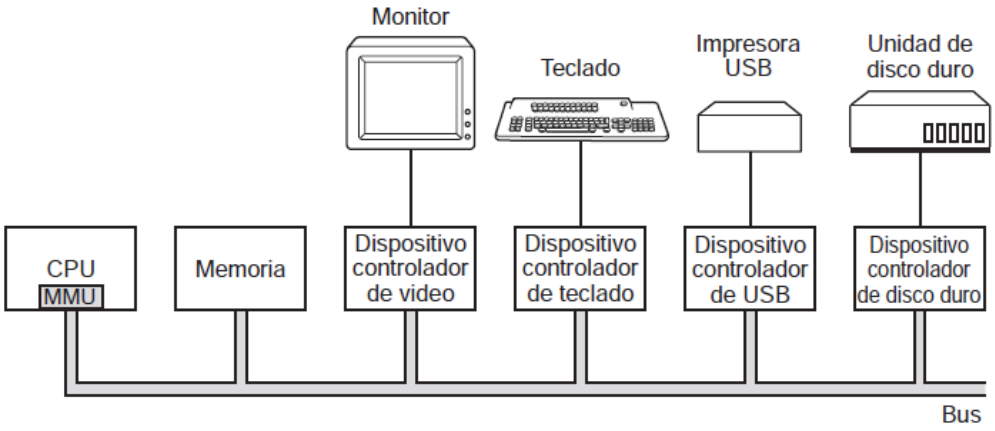


[30] Serial ATA, S-ATA o SATA (Serial Advanced Technology Attachment) es una interfaz de bus de computadoras para la transferencia de datos entre la placa base y algunos dispositivos de almacenamiento, como la unidad de disco duro, lectora y grabadora de discos ópticos, unidad de estado sólido u otros dispositivos.

[31] PCIe (Peripheral Component Interconnect Express) es un estándar de bus de expansión de alta velocidad para computadoras. Se utiliza para conectar tarjetas gráficas, tarjetas de sonido, tarjetas de red, discos duros y otros dispositivos.

Dispositivos de E/S.

La CPU y la memoria no son los únicos recursos que el sistema operativo debe administrar. Los dispositivos de E/S también interactúan mucho con el sistema operativo. Generalmente constan de dos partes: un dispositivo controlador y el dispositivo en sí. El dispositivo controlador es un chip o conjunto de chips que controla físicamente el dispositivo. Por ejemplo, acepta los comandos del sistema operativo para leer datos del dispositivo y los lleva a cabo.



En muchos casos, el control del dispositivo es muy complicado y detallado, por lo que el trabajo del chip o los chips del dispositivo controlador es presentar una interfaz más simple al sistema operativo (pero de todas formas sigue siendo muy complejo). Por ejemplo, un controlador de disco podría aceptar un comando para leer el sector 11,206 del disco 2; después, tiene que convertir este número de sector lineal en un cilindro, sector y cabeza. Esta conversión se puede complicar por el hecho de que los cilindros exteriores tienen más sectores que los interiores y que algunos sectores defectuosos se han reasignado a otros. Posteriormente, el dispositivo controlador tiene que determinar en cuál cilindro se encuentra el brazo y darle una secuencia de pulsos para desplazarse hacia dentro o hacia fuera el número requerido de cilindros; tiene que esperar hasta que el sector apropiado haya girado bajo la cabeza, y después empieza a leer y almacenar los bits a medida que van saliendo de la unidad, eliminando el preámbulo y calculando la suma de verificación. Por último, tiene que ensamblar los bits entrantes en palabras y almacenarlos en la memoria. Para hacer todo este trabajo, a menudo los dispositivos controladores consisten en pequeñas computadoras incrustadas que se programan para realizar su trabajo.

La otra pieza es el dispositivo en sí. Los dispositivos tienen interfaces bastante simples, debido a que no pueden hacer mucho y también para estandarizarlas. Esto último es necesario de manera que cualquier dispositivo controlador de disco IDE pueda manejar cualquier disco IDE, por ejemplo. IDE (Integrated Drive Electronics) significa Electrónica de unidades integradas y es el tipo estándar de disco en muchas computadoras. Como la interfaz real del dispositivo está oculta detrás del dispositivo controlador, todo lo que el sistema operativo ve es la interfaz para el dispositivo controlador, que puede ser bastante distinta de la interfaz para el dispositivo.

Como cada tipo de dispositivo controlador es distinto, se requiere software diferente para controlar cada uno de ellos. El software que se comunica con un dispositivo controlador, que le proporciona comandos y acepta respuestas, se conoce como ***driver*** (controlador). Cada fabricante de dispositivos controladores tiene que suministrar un driver específico para cada sistema operativo en que pueda funcionar. Así, un escáner puede venir, por ejemplo, con drivers para Windows y Linux.

Para utilizar el driver, se tiene que colocar en el sistema operativo de manera que pueda ejecutarse en ***modo kernel***. En realidad, los drivers se pueden ejecutar fuera del kernel, pero sólo unos cuantos sistemas actuales admiten esta posibilidad debido a que se requiere la capacidad para permitir que un driver en espacio de usuario pueda acceder al dispositivo de una manera controlada, una característica que raras veces se admite. Hay tres formas en que el driver se pueda colocar en el kernel: la primera es volver a enlazar el kernel con el nuevo driver y después reiniciar el sistema (muchos sistemas UNIX antiguos trabajan de esta manera); la segunda es crear una entrada en un archivo del sistema operativo que le indique que necesita el driver y después reinicie el sistema, para que en el momento del arranque, el sistema operativo busque los drivers necesarios y los cargue (Windows funciona de esta manera); la tercera forma es que el sistema operativo acepte nuevos drivers mientras los ejecuta e instala al instante, sin necesidad de reiniciar. Esta última forma solía ser rara, pero ahora es mucho más común.

Todo dispositivo controlador tiene un número pequeño de registros[32] que sirven para comunicarse con él. Por ejemplo, un dispositivo controlador de disco con las mínimas características podría tener registros para especificar la dirección de disco, dirección de memoria, número de sectores e instrucción (lectura o escritura). Para activar el dispositivo controlador, el driver recibe un comando del sistema operativo y después lo traduce en los valores apropiados para escribirlos en los registros del dispositivo.

Las operaciones de entrada y salida se pueden realizar de tres maneras distintas. En el método más simple, un programa de usuario emite una llamada al sistema, que el kernel posteriormente traduce en una llamada al procedimiento[33] para el driver apropiado. Después el driver inicia la E/S y permanece en un ciclo estrecho, sondeando en forma continua al dispositivo para ver si ha terminado (por lo general hay un bit que indica si el dispositivo sigue ocupado). Una vez terminada la E/S, el driver coloca los datos (si los hay) en donde se necesitan. Después el sistema operativo devuelve el control al programa de usuario llamador. A este método se le conoce como espera ocupada y tiene la desventaja de que mantiene ocupada la CPU sondeando al dispositivo hasta que éste termina.

El segundo método consiste en que el driver inicie el dispositivo y le pida generar una ***interrupción*** cuando termine. En este punto el driver regresa. Luego, el sistema operativo bloquea el programa llamador si es necesario y busca otro trabajo por hacer. Cuando el dispositivo controlador detecta el final de la transferencia, genera una interrupción para indicar que la operación se ha completado.

Una interrupción es una suspensión temporal de la ejecución de un proceso, para pasar a ejecutar una subrutina de servicio de interrupción, la cual, por lo general, no forma parte del

[32] En arquitectura de computadoras, un registro es una memoria de alta velocidad y poca capacidad que permite guardar transitoriamente y acceder a valores muy usados, generalmente en operaciones matemáticas.
[33] Una llamada a procedimiento es un proceso que invoca un procedimiento en un programa.

programa, sino que pertenece al sistema operativo. Una vez finalizada dicha subrutina, se reanuda la ejecución del programa.

Las interrupciones surgen de la necesidad que tienen los dispositivos periféricos de enviar información al procesador y son muy importantes en los sistemas operativos.

Si tomamos como ejemplo el proceso de escritura/lectura de un disco duro, los pasos típicos que se siguen cuando se produce una interrupción son los siguientes: en el paso 1, el driver indica al dispositivo controlador de disco lo que debe hacer, al escribir datos en sus registros de dispositivo. Después el dispositivo controlador inicia el dispositivo, cuando ha terminado de leer o escribir el número de bytes que debe transferir, alerta al chip controlador de interrupciones[34] mediante el uso de ciertas líneas de bus en el paso 2. Si el controlador de interrupciones está preparado para aceptar la interrupción (lo cual podría no ser cierto si está ocupado con una de mayor prioridad), utiliza un pin en el chip de CPU para informarlo, en el paso 3. En el paso 4, el controlador de interrupciones coloca el número del dispositivo en el bus, para que la CPU pueda leerlo y sepa cuál dispositivo acaba de terminar (puede haber muchos dispositivos funcionando al mismo tiempo).

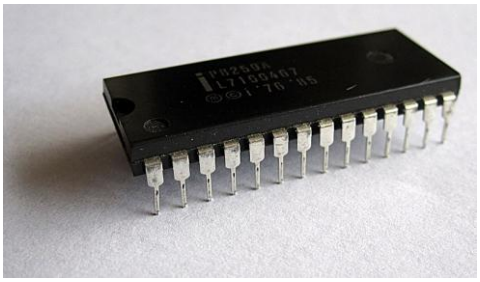


Ilustración 14 - Controlador programable de interrupciones Intel 8259.

Una vez que la CPU ha decidido tomar la interrupción, los datos del contador de programa[35] y del PSW[36] son típicamente agregados (pushed) en la pila actual y la CPU cambia al modo kernel. El número de dispositivo se puede utilizar como un índice en parte de la memoria para encontrar la dirección del manejador (handler) de interrupciones para este dispositivo. Esta parte de la memoria se conoce como vector de interrupción. Una vez que el manejador de interrupciones (parte del driver para el dispositivo que está realizando la interrupción) ha iniciado, quita el contador de programa y el PSW de la pila y los guarda, para después consultar al dispositivo y conocer su estado. Cuando el manejador de interrupciones termina, regresa al programa de usuario que se estaba ejecutando previamente a la primera instrucción que no se había ejecutado todavía.

[34] Un controlador programable de interrupciones o PIC (Programmable Interrupt Controller) es un dispositivo usado para combinar varias fuentes de interrupciones sobre una o más líneas del CPU, mientras que permite que los niveles de prioridad sean asignados a sus salidas de interrupción.
[35] Contador de programa (PC) es un registro de la unidad central de procesamiento (CPU) de una computadora que indica la posición de la instrucción que se está ejecutando o la siguiente.
[36] El program status word (PSW) es un área de la memoria o registro que contiene información sobre el estado de un programa utilizado por el sistema operativo.

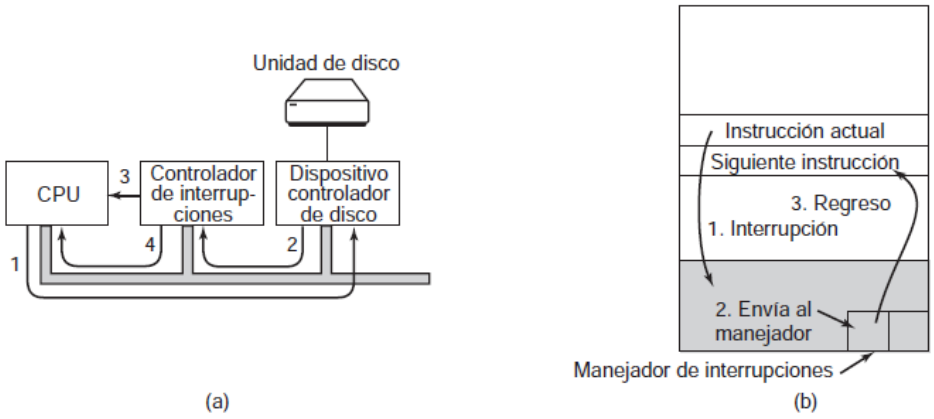


Ilustración 15 - (a) Los pasos para iniciar un dispositivo de E/S y obtener una interrupción. (b) El procesamiento de interrupciones involucra tomar la interrupción, ejecutar el manejador de interrupciones y regresar al programa de usuario.

El tercer método para realizar operaciones de E/S hace uso de un chip especial llamado ***DMA*** (Direct Memory Access; Acceso directo a memoria) que puede controlar el flujo de bits entre la memoria y un dispositivo controlador sin la intervención constante de la CPU. La CPU configura el chip DMA, le indica cuántos bytes debe transferir, las direcciones de dispositivo y de memoria involucradas, la instrucción y deja que haga su trabajo. Cuando el chip DMA termina genera una interrupción, la cual se maneja de la manera antes descrita.



Ilustración 16 -Clon de un controlador de acceso directo a memoria (DMA) Intel 8257 de NEC.

A menudo, las interrupciones pueden ocurrir en momentos muy inconvenientes, por ejemplo, mientras otro manejador de interrupciones se está ejecutando. Por esta razón, la CPU tiene una forma para deshabilitar las interrupciones y rehabilitarlas después. Mientras las interrupciones están deshabilitadas, cualquier dispositivo que termine continúa utilizando sus señales de interrupción, pero la CPU no se interrumpe sino hasta que se vuelven a habilitar las interrupciones. Si varios dispositivos terminan mientras las interrupciones están habilitadas, el controlador de interrupciones decide cuál debe dejar pasar primero, lo cual se basa generalmente en prioridades estáticas asignadas a cada dispositivo. El dispositivo de mayor prioridad gana.

Buses

El termino bus hace referencia a un conjunto de líneas o cables que se utilizan para transmitir datos, señales y energía entre diferentes componentes de un sistema informático o electrónico. Es un canal de comunicación y permite la transferencia de información entre los diferentes dispositivos conectados a él.

La organización que se utilizó en las computadoras como la IBM PC original, se basaba en un solo bus que manejaba todo el tráfico. Sin embargo, a medida que los procesadores y las memorias se hicieron más veloces, esta organización fue quedando obsoleta. Como resultado se agregaron más buses, tanto para dispositivos de E/S más rápidos como para el tráfico entre la CPU y la memoria.

A modo de ejemplo analizaremos una estructura un poco más moderna.

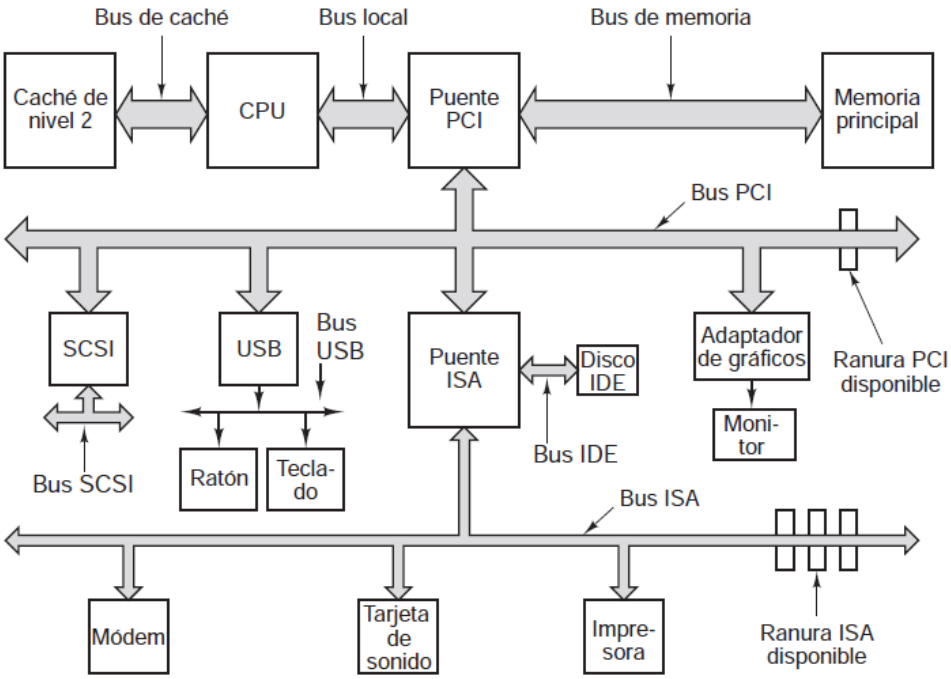


Ilustración 17 - La estructura de un sistema Pentium extenso.

El sistema tiene ocho buses (caché, local, memoria, PCI, SCSI, USB, IDE e ISA), cada uno con una velocidad de transferencia y función distintas. El sistema operativo debe estar al tanto de todos estos buses para su configuración y administración. Los dos buses principales son el bus ISA (Industry Standard Architecture, Arquitectura estándar de la industria) de la IBM PC original y su sucesor, el bus PCI (Peripheral Component Interconnect, Interconexión de componentes periféricos).

En sistemas más modernos probablemente no encontraríamos un bus PCI, sino alguna variante de su sucesor, el bus PCIE. El bus ISA ha dejado de ser utilizado masivamente hace ya muchos años.

Slot Width	PCIe 1.0 (2003)	PCIe 2.0 (2007)	PCIe 3.0 (2010)	PCIe 4.0 (2017)	PCIe 5.0 (2019)	PCIe 6.0 (2022)
x1	0.25GB/sec	0.5GB/sec	~1GB/sec	~2GB/sec	~4GB/sec	8GB/sec
x2	0.5GB/sec	1GB/sec	~2GB/sec	~4GB/sec	~8GB/sec	16GB/sec
x4	1GB/sec	2GB/sec	~4GB/sec	~8GB/sec	~16GB/sec	32GB/sec
x8	2GB/sec	4GB/sec	~8GB/sec	~16GB/sec	~32GB/sec	64GB/sec
x16	4GB/sec	8GB/sec	~16GB/sec	~32GB/sec	~64GB/sec	128GB/sec

Ilustración 18 - Ancho de banda de diferentes versiones de PCIE.

El USB (Universal Serial Bus; Bus serial universal) se inventó para conectar a la computadora todos los dispositivos de E/S lentos, como el teclado y el ratón. Utiliza un pequeño conector con cuatro cables, dos de los cuales suministran energía eléctrica a los dispositivos USB. El USB es un bus centralizado en el que un dispositivo raíz sondea los dispositivos de E/S para ver si tienen tráfico. Los dispositivos USB comparten un dispositivo controlador USB, lo que hace innecesario instalar un nuevo controlador para cada nuevo dispositivo USB. En consecuencia, pueden agregarse dispositivos USB a la computadora sin necesidad de reiniciar.

El sistema operativo tiene que saber qué dispositivos periféricos están conectados a la computadora y cómo configurarlos. Este requerimiento condujo a Intel y Microsoft a diseñar un sistema de PC conocido como plug and play basado en un concepto similar que se implementó por primera vez en la Apple Macintosh. Antes de plug and play, cada tarjeta de E/S tenía un nivel de petición de interrupción fijo y direcciones fijas para sus registros de E/S. Por ejemplo, el teclado tenía la interrupción 1 y utilizaba las direcciones de E/S 0x60 a 0x64, el dispositivo controlador de disco flexible tenía la interrupción 6 y utilizaba las direcciones de E/S 0x3F0 a 0x3F7, la impresora tenía la interrupción 7 y utilizaba las direcciones de E/S 0x378 a 0x37A, y así sucesivamente.

El problema llegó cuando el usuario compraba una tarjeta de sonido y una tarjeta de módem que utilizaban la misma interrupción, por ejemplo, la 4. Instaladas juntas serían incapaces de funcionar. La solución fue incluir interruptores DIP[37] o puentes (jumpers) en cada tarjeta de E/S e indicar al usuario que los configurara para seleccionar un nivel de interrupción y direcciones de dispositivos de E/S que no estuvieran en conflicto con las demás tarjetas en el sistema del usuario.

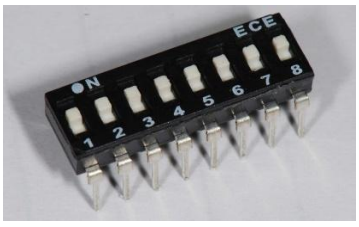
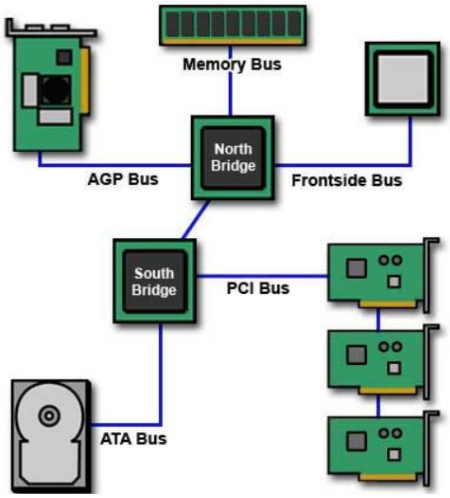


Ilustración 19 - Interruptor DIP.

[37] Un DIP se trata de un conjunto de interruptores eléctricos que se presenta en un formato encapsulado (en lo que se denomina Dual In-line Package).

La función de plug and play es permitir que el sistema recolecte automáticamente la información acerca de los dispositivos de E/S, asigne los niveles de interrupción y las direcciones de E/S de manera central, para que después indique a cada tarjeta cuáles son sus números. Este trabajo está íntimamente relacionado con el proceso de arranque de la computadora.



Arranque de la computadora.

Utilizaremos como ejemplo un sistema basado en Pentium, ya que ha sido un estándar durante muchos años. Cada Pentium contiene una tarjeta madre (motherboard). En la tarjeta madre hay un programa conocido como ***BIOS*** (Basic Input Output System, Sistema básico de entrada y salida) del sistema. El BIOS contiene software de E/S de bajo nivel[38], incluyendo procedimientos para leer el teclado, escribir en la pantalla y realizar operaciones de E/S de disco, entre otras cosas.

[38] Software de bajo nivel es aquel que ejerce un control directo sobre el hardware y está condicionado por la estructura física de las computadoras que lo soportan. El uso de la palabra bajo en su denominación no implica que sea menos potente que uno de alto nivel, sino que se refiere a la reducida abstracción entre el software y el hardware.

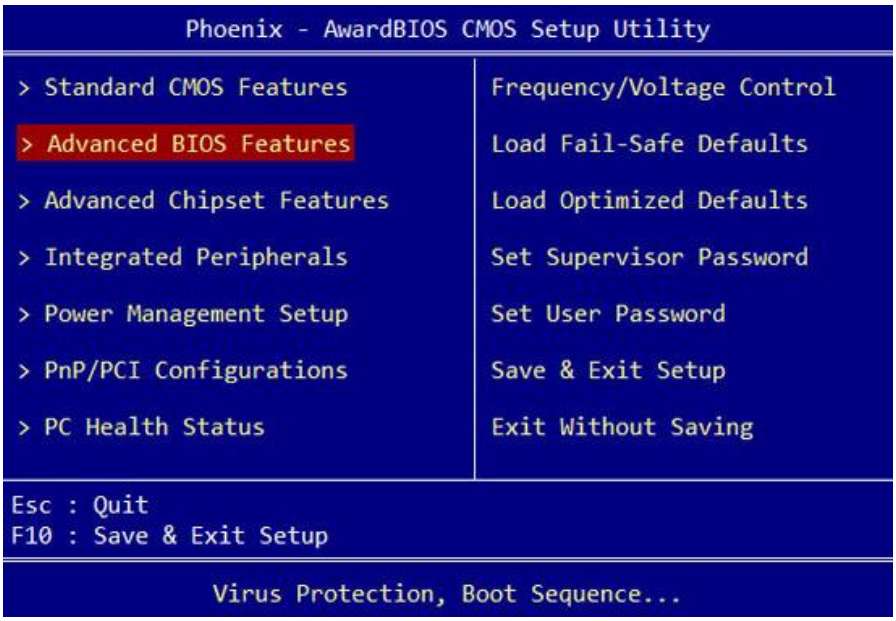


Ilustración 20 - Antigua interfaz de BIOS del fabricante Phoenix.

Cuando se arranca la computadora, el BIOS inicia su ejecución. Primero hace pruebas para ver cuánta RAM hay instalada y si el teclado junto con otros dispositivos básicos está instalado y responden en forma correcta. Empieza explorando los buses ISA y PCI para detectar todos los dispositivos conectados a ellos. Estos dispositivos se registran; y los dispositivos plug and play también. Si los dispositivos presentes son distintos de los que había cuando el sistema se inició por última vez, se configuran los nuevos dispositivos.

Después, el BIOS determina el dispositivo de arranque, para lo cual prueba una lista de dispositivos almacenada en la memoria CMOS. El usuario puede cambiar esta lista si entra a un programa de configuración del BIOS, justo después de iniciar el sistema.

El primer sector del dispositivo de arranque se lee y se coloca en la memoria, para luego ejecutarse. Este sector contiene un programa que por lo general examina la tabla de particiones al final del sector de arranque, para determinar qué partición está activa. Después se lee un cargador de arranque secundario de esa partición. Este cargador lee el sistema operativo de la partición activa y lo inicia.

Luego, el sistema operativo consulta al BIOS para obtener la información de configuración. Para cada dispositivo, comprueba si tiene el driver correspondiente. De no ser así, pide al usuario que lo provea. Una vez que tiene los drivers de todos los dispositivos, el sistema operativo los carga en el kernel. Después inicializa sus tablas, crea los procesos de segundo plano que se requieran, y arranca un programa de inicio de sesión o GUI.

Hay que tener en cuenta que el BIOS fue creado en la década de 1970, por lo tanto, podríamos decir que ha quedado obsoleto.

La Interfaz de Firmware Extensible Unificada o UEFI (Unified Extensible Firmware Interface) es el firmware[39] sucesor, y por lo general, está escrito en C.



Ilustración 21 - Interfaz UEFI

Básicamente, todo lo que hemos dicho antes que hace el BIOS lo hace también la UEFI. Pero también tiene otras funciones adicionales y mejoras sustanciales.

- La UEFI puede conectarse a Internet para actualizarse.
- La diferencia más notable para el usuario medio entre ambos firmwares está en el aspecto.
- Mientras que el BIOS se ejecuta en modo de 16 bits, limitando la velocidad de arranque, la UEFI puede funcionar en modo de 32 o 64 bits, permitiendo tiempos de arranque más rápidos.
- Compatibilidad con discos duros de mayor capacidad.
- Cuenta con arranque seguro, que verifica la validez del sistema operativo y previniendo la ejecución de código no autorizado durante el inicio.

Referencias y bibliografía de la sección.

- Tanenbaum 1-3.
- Seagate - Documento de tecnología - Almacenamiento de estado sólido.

[39] El firmware o soporte lógico inalterable es un programa informático que establece la lógica de más bajo nivel que controla los circuitos electrónicos de un dispositivo de cualquier tipo. Está fuertemente integrado con la electrónica del dispositivo, es el software que tiene directa interacción con el hardware, siendo así el encargado de controlarlo para ejecutar correctamente las instrucciones externas.

Subsistemas principales.

Gestor de procesos.

Un programa no hace nada a menos que una CPU ejecute sus instrucciones. Un programa en ejecución es un ***proceso***. Podríamos considerar al proceso como una unidad de trabajo en un sistema.

Un proceso necesita de ciertos recursos para llevar a cabo su tarea (tiempo de la CPU, memoria, espacio de almacenamiento, dispositivos de E/S, etc.). Además de estos recursos, a un proceso se le suministran entradas. Las entradas son datos o información que se suministran al proceso. Por ejemplo, a un programa que elimina un archivo, se le proporciona la entrada del nombre del archivo a eliminar. Cuando un proceso termina, se liberan los recursos utilizados.

Podemos diferenciar a los procesos basándonos en el código que ejecutan. De esta forma tendremos procesos del sistema cuando se ejecuta código del sistema operativo y procesos de usuario cuando se ejecuta código de programas del usuario.

El gestor de procesos de un sistema operativo se encarga de supervisar y regular los procesos que lleva a cabo. Se ocupa de:

- Crear y borrar los procesos.
- Sincronizar los procesos.
- La comunicación entre los procesos.
- Tratamientos de interbloqueos (se producen cuando todos los procesos aseguran algunos recursos, pero quedan a la espera de otros, los cuales han sido asignados a procesos diferentes).

Gestor de memoria.

La memoria principal es un repositorio de datos rápidamente accesible, es compartida por la CPU y los dispositivos de E/S. La CPU lee y escribe datos e instrucciones de la memoria principal.

Para que la CPU procese datos de una unidad de almacenamiento, primero deben copiarse a la memoria principal (mediante llamadas de E/S). Del mismo modo, cualquier instrucción debe estar primero en memoria para que la CPU pueda ejecutarla.

Entonces, para que un programa pueda ejecutarse debe estar previamente cargado en memoria. Una vez que un programa concluye su ejecución, la memoria que utilizaba se libera; y así puede ser utilizada para almacenar las instrucciones de otro programa.

Es común que un sistema operativo mantenga varios programas en memoria al mismo tiempo, esto incrementa la necesidad de mecanismos eficientes de gestión de memoria.

El gestor de memoria se encarga principalmente de:

- Controlar que partes de la memoria están en uso y por quién o qué.
- Decidir qué datos y procesos añadir o quitar de la memoria.

- Asignar y liberar el espacio en la memoria.

Gestor de archivos.

Las computadoras pueden guardar información en los diferentes medios físicos de almacenamiento. Él sistema operativo abstrae las propiedades físicas de estos medios físicos, y define una unidad de almacenamiento lógico, el ***archivo***.

Gestión de archivos.

El sistema operativo gestiona los medios de almacenamiento y los dispositivos que los controlan. Además, organiza los archivos en directorios para facilitar su uso.

Se encarga de:

- Creación y borrado de archivos.
- Creación y borrado de directorios.

Gestión de almacenamiento masivo.

La memoria principal es limitada y volátil. Por ese motivo los sistemas informáticos modernos utilizan diferentes soportes de almacenamiento secundario como unidades SDD o discos.

El gestor de almacenamiento masivo se encarga de:

- Gestión de espacio libre de los dispositivos de almacenamiento.
- Asignación de espacio de almacenamiento.
- Planificación de la unidad de almacenamiento.

Almacenamiento en cache.

Una caché o memoria intermedia es un componente de hardware o software que guarda datos para que las solicitudes futuras de esos datos se puedan atender con mayor rapidez.

Una memoria caché es una capa de almacenamiento de datos de alta velocidad que almacena un subconjunto de datos, normalmente transitorios, de modo que las solicitudes futuras de dichos datos se atenderán con mayor rapidez que si se accediera a estos datos desde la ubicación de almacenamiento principal. El almacenamiento en caché permite reutilizar de forma eficaz los datos recuperados o procesados anteriormente.

Gestor de E/S.

Todas las computadoras tienen dispositivos físicos para adquirir entradas y producir salidas (teclados, monitores, impresoras, etc.). De otra forma, los usuarios no podrían indicarles qué trabajo se debe realizar y tampoco se podrían obtener los resultados de dicho trabajo.

En consecuencia, cada sistema operativo tiene un subsistema de E/S para administrar sus dispositivos de E/S.

Consta de:

- Un componente de gestión de memoria (búfer, cache y gestión de colas).
- Una interfaz para controladores de dispositivos.
- Controladores para dispositivos de hardware específicos.

Seguridad.

Las computadoras contienen grandes cantidades de información que los usuarios comúnmente desean proteger y mantener de manera confidencial. Esta información puede incluir mensajes de correo electrónico, planes de negocios, declaraciones fiscales y mucho más. Es responsabilidad del sistema operativo administrar la seguridad del sistema de manera que los archivos, por ejemplo, sólo sean accesibles para los usuarios autorizados.

En un sistema operativo pueden trabajar al mismo tiempo múltiples usuarios. De la misma forma puede haber varios procesos ejecutándose concurrentemente.

Esta ensalada de usuarios y procesos puede generar conflictos o inconsistencias, por ese motivo es necesario regular el acceso a los datos.

Para esto, el sistema operativo utiliza mecanismos que procuran que solo accedan a los recursos los procesos que hayan obtenido la autorización correspondiente.

Hablamos de ***protección*** cuando nos referimos a cualquier mecanismo que controle el acceso de procesos y usuarios a los recursos.

Un sistema puede tener buenos mecanismos de protección y aun así permitir accesos inapropiados (por ejemplo, a un usuario le roban su password). Por este motivo es necesario contar con mecanismos de ***seguridad*** que se encarguen de defender al sistema de ataques internos o externos (virus, robos de identidad, etc.)

En algunos sistemas operativos la prevención ante estos ataques se considera una función del propia del sistema, en otros casos, la prevención se deja a cargo de programas adicionales.

Referencias y bibliografía de la sección.

- Silberschatz 1-6 / 1-9.

Servicios del sistema operativo.

Un sistema operativo proporciona un entorno para la ejecución de programas. El sistema presta ciertos servicios a los programas y a los usuarios de dichos programas. Por supuesto, los servicios específicos que se suministran difieren de un sistema operativo a otro, pero podemos identificar perfectamente una serie de clases comunes. Estos servicios del sistema operativo se proporcionan para comodidad del programador, con el fin de facilitar la tarea de desarrollo.

- **Interfaz de usuario.** Casi todos los sistemas operativos disponen de una interfaz de usuario (UI, user interface), que puede tomar diferentes formas. Uno de los tipos existentes es la interfaz de línea de comandos (CLI, command-line interface).

  Otro tipo destacable es la interfaz de proceso por lotes, en la que los comandos y las directivas para controlar dichos comandos se introducen en archivos, luego dichos archivos se ejecutan.

  Habitualmente, se utiliza una interfaz gráfica de usuario (GUI, graphical user interface).

Vale aclarar que, la interfaz de usuario (sea una línea de comandos o una interfaz gráfica/GUI) no es un servicio indispensable ni estrictamente obligatorio dentro del núcleo (kernel) de un sistema operativo.

- **Ejecución de programas.** El sistema tiene que poder cargar un programa en memoria y ejecutar dicho programa. Todo programa debe poder terminar su ejecución, de forma normal o anormal (indicando un error).
- **Operaciones de E/S.** Un programa en ejecución puede necesitar llevar a cabo operaciones de E/S, dirigidas a un archivo o a un dispositivo de E/S.

  Por cuestiones de eficiencia y protección, los usuarios no pueden, normalmente, controlar de modo directo los dispositivos de E/S; por tanto, el sistema operativo debe proporcionar medios para realizar la E /S.

- **Manipulación del sistema de archivos.** El sistema de archivos tiene una importancia especial. Obviamente, los programas necesitan leer y escribir en archivos y directorios. También necesitan crearlos y borrarlos usando su nombre, realizar búsquedas en un determinado archivo o presentar la información contenida en un archivo. Por último, algunos programas incluyen mecanismos de gestión de permisos para conceder o denegar el acceso a los archivos o directorios basándose en quién sea el propietario del archivo.
- **Comunicaciones**. Hay muchas circunstancias en las que un proceso necesita intercambiar información con otro. Dicha comunicación puede tener lugar entre procesos que estén ejecutándose en la misma computadora o entre procesos que se ejecuten en computadoras diferentes conectadas a través de una red.
- **Detección de errores.** El sistema operativo necesita detectar constantemente los posibles errores. Estos errores pueden producirse en el hardware del procesador y de memoria (como, por ejemplo, un error de memoria o un fallo de la alimentación) en un dispositivo de E/S (como un fallo de conexión en una red) o en los programas de usuario (como, por ejemplo, un desbordamiento aritmético). Para cada tipo de error, el sistema operativo debe llevar a cabo la acción apropiada para asegurar un funcionamiento correcto y coherente.

Hay disponible otro conjunto de funciones del sistema de operativo que no están pensadas para ayudar al usuario, sino para garantizar la eficiencia del propio sistema. Los sistemas con múltiples usuarios pueden ser más eficientes cuando se comparten los recursos del equipo entre los distintos usuarios.

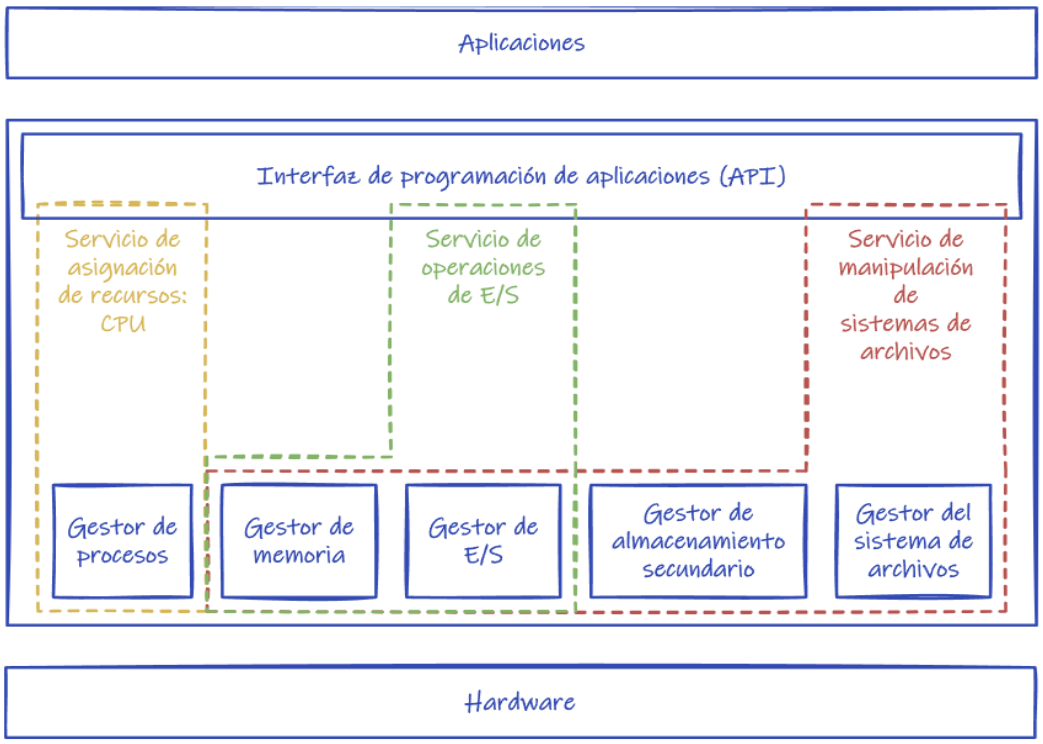
- **Asignación de recursos.** Cuando hay varios usuarios, o hay varios trabajos ejecutándose al mismo tiempo, deben asignarse a cada uno de ellos los recursos necesarios. El sistema operativo gestiona muchos tipos diferentes de recursos; algunos (como los ciclos de CPU, la memoria principal y el espacio de almacenamiento de archivos) pueden disponer de código software especial que gestione su asignación, mientras que otros (como los dispositivos de E/S) pueden tener código que gestione de forma mucho más general su solicitud y liberación.
- **Protección y seguridad.** Los propietarios de la información almacenada en un sistema de computadoras en red o multiusuario necesitan a menudo poder controlar el uso de

dicha información. Cuando se ejecutan de forma concurrente varios procesos distintos, no debe ser posible que un proceso interfiera con los demás procesos o con el propio sistema operativo.

La protección implica asegurar que todos los accesos a los recursos del sistema estén controlados.

También es importante garantizar la seguridad del sistema frente a posibles intrusos; dicha seguridad comienza por requerir que cada usuario se autentique ante el sistema, usualmente mediante una contraseña, para obtener acceso a los recursos del mismo.

Esto se extiende a la defensa de los dispositivos de E/S, entre los que se incluyen adaptadores de red, frente a intentos de acceso ilegales y el registro de dichas conexiones con el fin de detectar intrusiones.



Vale señalar que, como se muestra en la figura anterior, un servicio puede involucrar a más de un subsistema principal del sistema operativo.

Referencias y bibliografía de la sección.

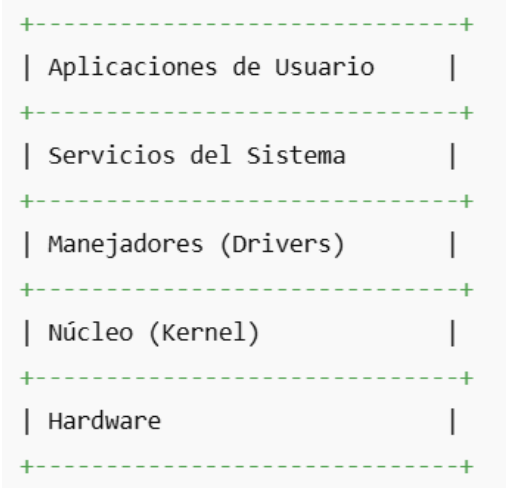
- Silberschatz 2.1.

Estructura de un sistema operativo.

La estructura de un sistema operativo se refiere a la forma en que está organizado internamente: cómo se dividen sus componentes y se comunican entre ellos, y cómo se relacionan con el hardware y las aplicaciones del usuario.

Hay distintos modelos de estructura, cada uno con sus ventajas y desventajas, usados según el diseño y los objetivos del sistema operativo.

Algunos de los componentes básicos de la estructura de un SO pueden ser el kernel con sus módulos/subsistemas principales, interfaces con el usuario y con las aplicaciones, y utilidades del sistema.



Estructura simple.

Muchos sistemas comerciales no tienen una estructura bien definida. Frecuentemente, tales sistemas operativos comienzan siendo sistemas pequeños, simples y limitados; luego crecen más allá de su ámbito original. MS-DOS fue un ejemplo de un sistema así.

Originalmente, MS-DOS fue diseñado e implementado por unas pocas personas que no tenían ni idea de que iba a terminar siendo tan popular. Fue escrito para proporcionar la máxima funcionalidad en el menor espacio posible, por lo que no fue dividido en módulos de forma cuidadosa.

En MS-DOS, las interfaces y niveles de funcionalidad no estaban separados. Por ejemplo, los programas de aplicación podían acceder a las rutinas básicas de E/S para escribir directamente en la pantalla y las unidades de disco. Tal libertad hacía que MS-DOS sea vulnerable a programas erróneos (o maliciosos), lo que facilitaba que el sistema completo fallase cuando los programas de usuario fallaban.

Como el 8088 de Intel para el que fue escrito no proporciona un modo dual (modo usuario / modo kernel) ni protección hardware, lo que impedía separar las aplicaciones del sistema operativo. Es decir, que los diseñadores de MS-DOS no tuvieron más opción que dejar accesible el hardware base.

Otro ejemplo de estructuración limitada es el sistema operativo UNIX original. UNIX es otro sistema que inicialmente estaba limitado por la funcionalidad hardware. Consta de dos partes separadas: el kernel y los programas del sistema.

El kernel se divide en una serie de interfaces y controladores de dispositivo, que se han ido añadiendo y ampliando a lo largo de los años, a medida que UNIX ha ido evolucionando.

Podemos entender el tradicional sistema operativo UNIX como una estructura de niveles. Todo lo que está por debajo de la interfaz de llamadas al sistema y por encima del hardware físico es el kernel.

El kernel proporciona el sistema de archivos, los mecanismos de planificación de la CPU, la funcionalidad de gestión de memoria y otras funciones del sistema operativo, a través de las llamadas al sistema.

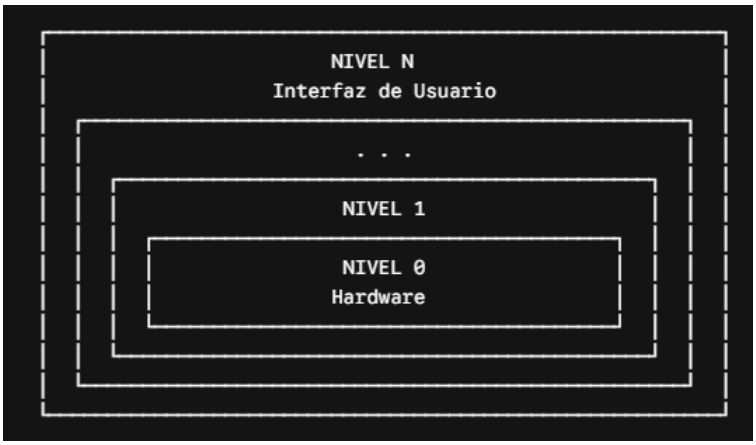
En resumen, es una enorme cantidad de funcionalidad que se combina en un sólo nivel. Esta estructura ***monolítica*** era difícil de implementar y de mantener.

Estructura en niveles.

Con el soporte hardware apropiado, los sistemas operativos puede dividirse en partes más pequeñas y más adecuadas que lo que permitían los sistemas originales MS-DOS o UNIX.

El sistema operativo puede entonces mantener un control mucho mayor sobre la computadora y sobre las aplicaciones que hacen uso de dicha computadora. Los implementadores tienen más libertad para cambiar el funcionamiento interno del sistema y crear sistemas operativos modulares.

Con el método de diseño arriba-abajo, se determinan las características y la funcionalidad globales y se separan en componentes. La ocultación de los detalles a ojos de los niveles superiores también es importante, dado que deja libres a los programadores para implementar las rutinas de bajo nivel como prefieran, siempre que la interfaz externa de la rutina permanezca invariable y la propia rutina realice la tarea anunciada.



Un sistema puede hacerse modular de muchas formas. Un posible método es mediante una estructura en ***niveles***, en el que el sistema operativo se divide en una serie de capas (niveles).

En los niveles de los extremos superiores e inferiores normalmente se encuentran los límites del sistema operativo. El nivel inferior, nivel 0, es el hardware; el nivel superior, nivel N, es la interfaz de usuario. Ninguno de estos niveles forma parte del sistema operativo.

La principal ventaja del método de niveles es la simplicidad de construcción y depuración. Los niveles se seleccionan de modo que cada uno usa funciones (operaciones) y servicios de los niveles inferiores. Este método simplifica la depuración y la verificación del sistema.

El primer nivel puede depurarse sin afectar al resto del sistema, dado que, por definición, sólo usa el hardware básico (que se supone correcto) para implementar sus funciones.

Una vez que el primer nivel se ha depurado, puede suponerse correcto su funcionamiento mientras se depura el segundo nivel, etc.

Si se encuentra un error durante la depuración de un determinado nivel, el error tendrá que estar localizado en dicho nivel, dado que los niveles inferiores a él ya se han depurado. Por tanto, el diseño e implementación del sistema se simplifican.

Cada nivel se implementa utilizando sólo las operaciones proporcionadas por los niveles inferiores. Un nivel no necesita saber cómo se implementan dichas operaciones; sólo necesita saber qué hacen esas operaciones.

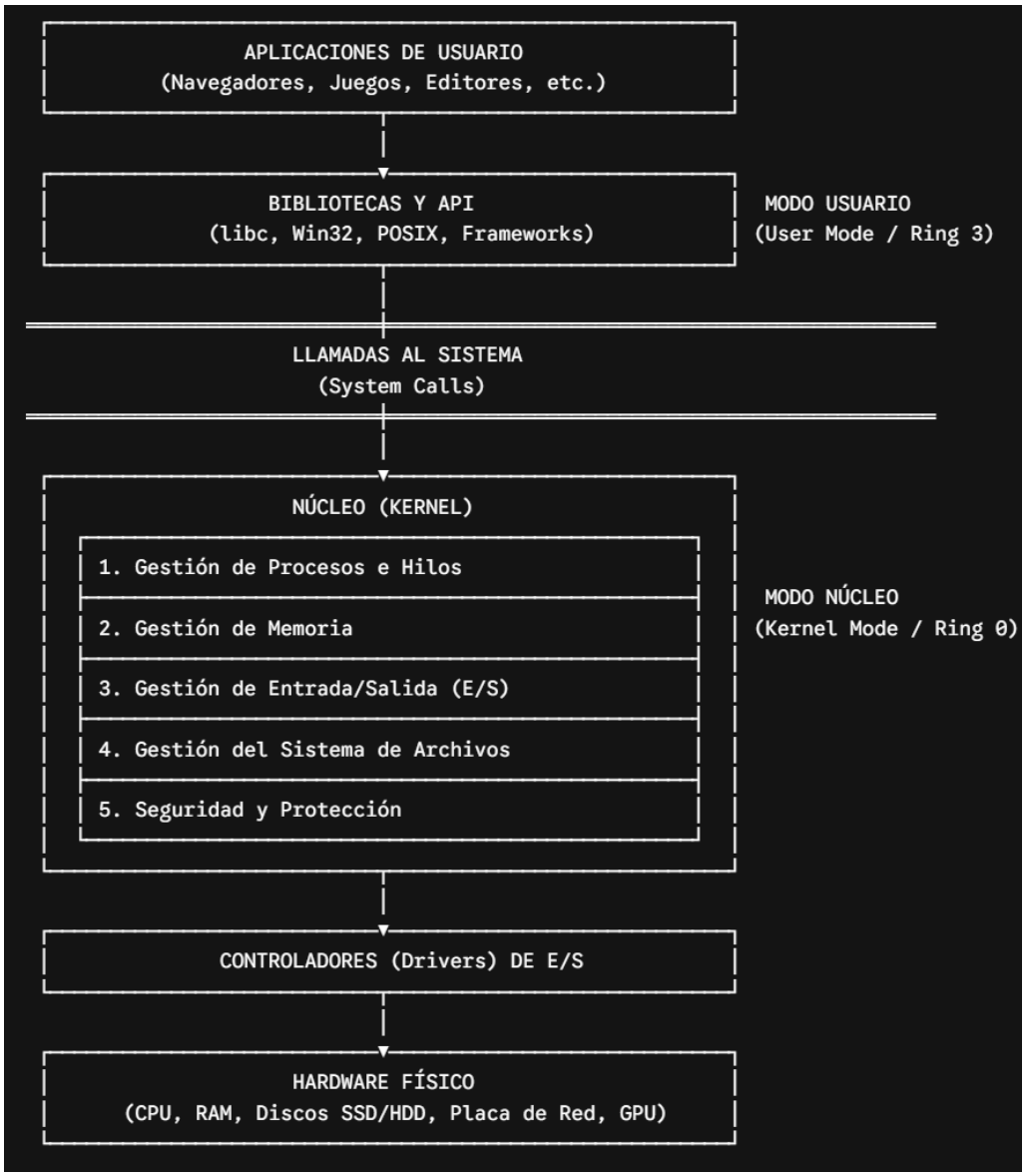
Por tanto, cada nivel oculta a los niveles superiores la existencia de determinadas estructuras de datos, operaciones y hardware.

La principal dificultad con el método de niveles es la de definir apropiadamente los diferentes niveles. Dado que un nivel sólo puede usar los servicios de los niveles inferiores, es necesario realizar una planificación cuidadosa.

Un último problema con las implementaciones por niveles es que tienden a ser menos eficientes que otros tipos de implementación. Por ejemplo, cuando un programa de usuario ejecuta una operación de E/S, realiza una llamada al sistema que será capturada por el nivel de E/S, el cual llamará al nivel de gestión de memoria, el cual a su vez llamará al nivel de planificación de la CPU, que pasará a continuación la llamada al hardware. En cada nivel, se pueden modificar los parámetros, puede ser necesario pasar datos, etc.

Cada nivel añade así una carga de trabajo adicional a la llamada al sistema; el resultado neto es una llamada al sistema que tarda más en ejecutarse que en un sistema sin niveles.

Estas limitaciones han hecho surgir una cierta reacción contra los sistemas basados en niveles. En otros diseños, se utiliza un menor número de niveles, con más funcionalidad por cada nivel, lo que proporciona muchas de las ventajas del código modular, a la vez que se evitan los problemas más difíciles relacionados con la definición e interacción de los niveles.



- **Espacio de Usuario (Modo Usuario / Ring 3):** Es el entorno de ejecución sin privilegios. Las instrucciones que se ejecutan aquí no pueden acceder directamente a la memoria de otros programas ni interactuar directamente con el hardware sin pasar por el sistema operativo.
    - **Aplicaciones de Usuario:** Programas de aplicación que ejecuta el usuario (navegadores web, juegos, editores de texto, etc.).
    - **Bibliotecas y API:** Conjunto de funciones estándar (como libc en UNIX/Linux, o la API Win32 en Windows) que abstraen y facilitan el uso de las llamadas al sistema. Ofrecen funciones como printf(), las cuales internamente solicitan servicios al núcleo.
- **Interfaz de Llamadas al Sistema (System Calls):** Es la frontera o mecanismo de transición entre el Modo Usuario y el Modo Núcleo. Cuando una aplicación necesita una operación privilegiada (como leer un archivo o reservar memoria), ejecuta una

instrucción especial (interrupción por software o trap) que conmuta la CPU a Modo Núcleo de forma segura.

- **Núcleo (Kernel / Modo Núcleo / Ring 0):** Es el conjunto de componentes ejecutados con privilegios totales sobre la CPU y el hardware. Los subsistemas principales habitan esta sección.
    - **Capa de Controladores (Drivers) de E/S:** Son módulos específicos de software encargados de la comunicación directa con cada periférico o controlador de dispositivo de hardware. Traducen las órdenes de alto nivel enviadas por el subsistema de E/S del núcleo a comandos de bajo nivel que entiende la electrónica del dispositivo particular.
- **Hardware:** Corresponde a la capa electrónica y de componentes físicos sobre la cual opera el sistema.
    - **CPU:** Ejecuta las instrucciones físicas y conmuta entre modos de privilegio.
    - **Memoria principal:** Almacena físicamente el código y los datos en ejecución.
    - **Dispositivos de almacenamiento y periféricos:** SSDs, HDDs, placas de red, GPUs y controladores de bus que realizan las operaciones de E/S reales.

Referencias y bibliografía de la sección.

- Silberschatz 2.7.

Otras estructuras del núcleo.

Aunque no son los únicos, los seis diseños de núcleo más representativos son: sistemas monolíticos, sistemas de capas, microkernels, sistemas cliente-servidor, máquinas virtuales y exokernels.

Sistemas monolíticos.

En esta arquitectura, todas las funciones del kernel se ejecutan en un solo espacio. Con este diseño, el sistema operativo se ejecuta como un solo programa en modo kernel.

El sistema operativo se escribe como una colección de procedimientos, enlazados entre sí en un solo programa binario ejecutable extenso.

Cuando se utiliza esta técnica, cada procedimiento en el sistema tiene la libertad de llamar a cualquier otro, si éste proporciona cierto cómputo útil que el primero necesita. Al tener miles de procedimientos que se pueden llamar entre sí sin restricción, con frecuencia se produce un sistema poco manejable y difícil de comprender.

Para construir el programa objeto[40] actual del sistema operativo cuando se utiliza este diseño, primero se compilan todos los procedimientos individuales (o los archivos que contienen los procedimientos) y luego se vinculan en conjunto para formar un solo archivo ejecutable, usando el enlazador[41] del sistema.

Para solicitar los servicios (llamadas al sistema) que proporciona el sistema operativo, los parámetros se colocan en un lugar bien definido (por ejemplo, en una pila) y luego se ejecuta una instrucción de trap[42]. Esta instrucción cambia la máquina del modo usuario al modo kernel y transfiere el control al sistema operativo. Después el sistema operativo obtiene los parámetros y determina cuál es la llamada al sistema que se va a llevar a cabo. Después la indiza en una tabla que contiene un apuntador al procedimiento que lleva a cabo la llamada al sistema.

Esta organización sugiere una estructura básica para el sistema operativo:

- Un programa principal que invoca el procedimiento de servicio solicitado.
- Un conjunto de procedimientos de servicio que llevan a cabo las llamadas al sistema.
- Un conjunto de procedimientos utilitarios que ayudan a los procedimientos de servicio.

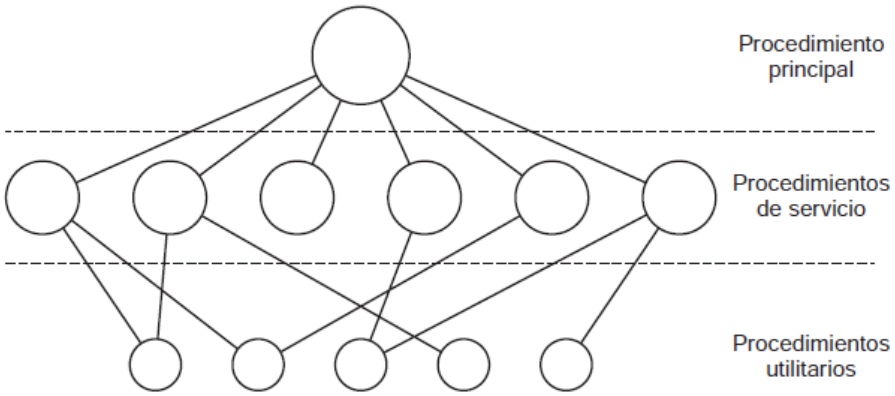


Ilustración 22 - División de los procedimientos en tres niveles.

En este modelo, para cada llamada al sistema hay un procedimiento de servicio que se encarga de la llamada y la ejecuta. Los procedimientos utilitarios hacen cosas que necesitan varios procedimientos de servicio, como obtener datos de los programas de usuario.

Sistemas de capas.

Otra opción de diseño es organizar el sistema operativo como una jerarquía de capas, cada una construida encima de la que tiene abajo. El primer sistema construido de esta forma fue el

[40] Programa objeto es aquel que se encuentra en lenguaje máquina y que ya es ejecutable por ésta. Es el programa fuente traducido (por el compilador) a código máquina.
[41] Se podría definir a un enlazador como un programa que toma los códigos objetos creados en el proceso de compilación de un programa, junto con los datos provenientes de subprogramas que se agrupan en bibliotecas, todo esto con el fin de producir un ejecutable o una biblioteca.
[42] Es una interrupción para buscar información en otros sitios, haciendo que se salte a otro programa. Hay una instrucción TRAP para intercambiar entre el modo usuario y el modo kernel.

sistema THE, desarrollado en Países Bajos por E. W. Dijkstra y sus estudiantes. Publicado en 1968, ***THE*** era un sistema simple de procesamiento por lotes para una computadora holandesa, la Electrologica X8[43].

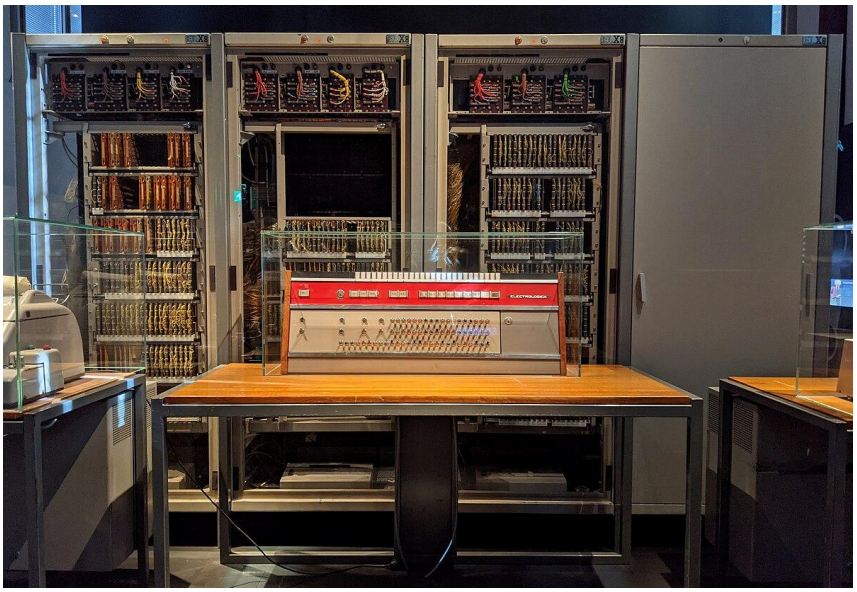


Ilustración 23 - Electrologica X8.

El sistema tenía seis capas. El nivel 0 se encargaba de la asignación del procesador, de cambiar entre un proceso y otro cuando ocurrían interrupciones o expiraban los temporizadores. Por encima del nivel 0, el sistema consistía en procesos secuenciales, cada uno de los cuales se podían programar sin necesidad de preocuparse por el hecho de que había varios procesos en ejecución en un solo procesador. En otras palabras, el nivel 0 proporcionaba la multiprogramación básica de la CPU.

La capa 1 se encargaba de la administración de la memoria. Asignaba espacio para los procesos en la memoria principal y en un tambor de palabras de 512 K que se utilizaba para contener partes de procesos (páginas), para los que no había espacio en la memoria principal. Por encima de la capa 1, los procesos no tenían que preocuparse acerca de si estaban en memoria o en el tambor; el software de la capa 1 se encargaba de asegurar que las páginas se llevaran a memoria cuando se requerían.

La capa 2 se encargaba de la comunicación entre cada proceso y la consola del operador (es decir, el usuario). Encima de esta capa, cada proceso tenía en efecto su propia consola de computadora.

La capa 3 se encargaba de administrar los dispositivos de E/S y de guardar en búferes los flujos de información dirigidos para y desde ellos. Encima de la capa 3, cada proceso podía trabajar con los dispositivos abstractos de E/S con excelentes propiedades, en vez de los dispositivos reales con muchas peculiaridades.

[43] La Electrologica X8 fue una computadora digital fabricada en los Países Bajos por Electrologica NV entre 1964 y 1968.

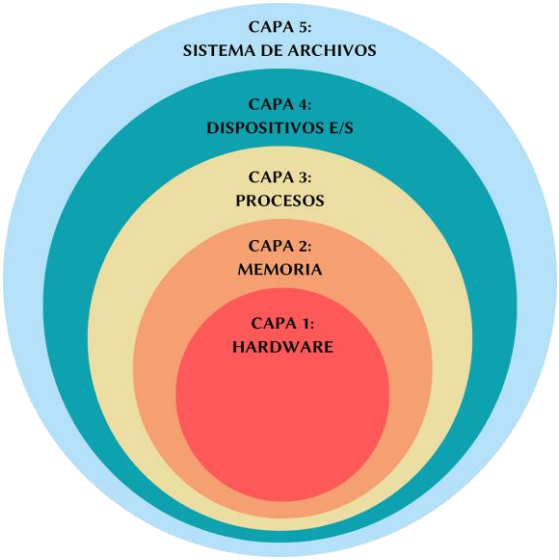
La capa 4 era en donde se encontraban los programas de usuario. No tenían que preocuparse por la administración de los procesos, la memoria, la consola o la E/S.

El proceso operador del sistema se encontraba en el nivel 5.

Capa	Función
5	El operador
4	Programas de usuario
3	Administración de la entrada/salida
2	Comunicación operador-proceso
1	Administración de memoria y tambor
0	Asignación del procesador y multiprogramación

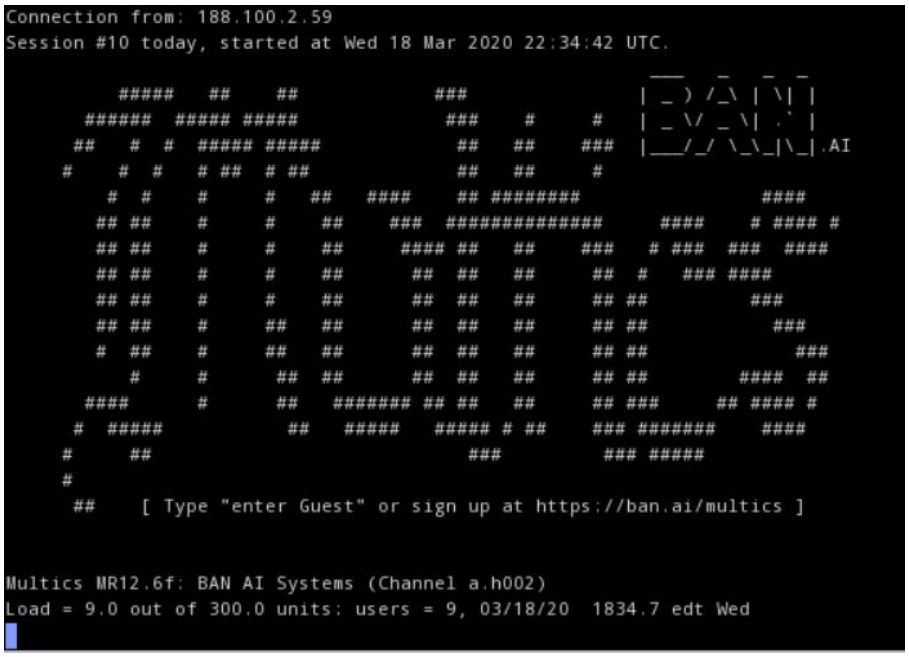
Ilustración 24 - Capas del sistema operativo THE.

Una mayor generalización del concepto de capas estaba presente en el sistema ***MULTICS***[44]. En vez de capa, MULTICS se describió como una serie de anillos concéntricos, en donde los interiores tenían más privilegios que los exteriores (que en efecto viene siendo lo mismo).



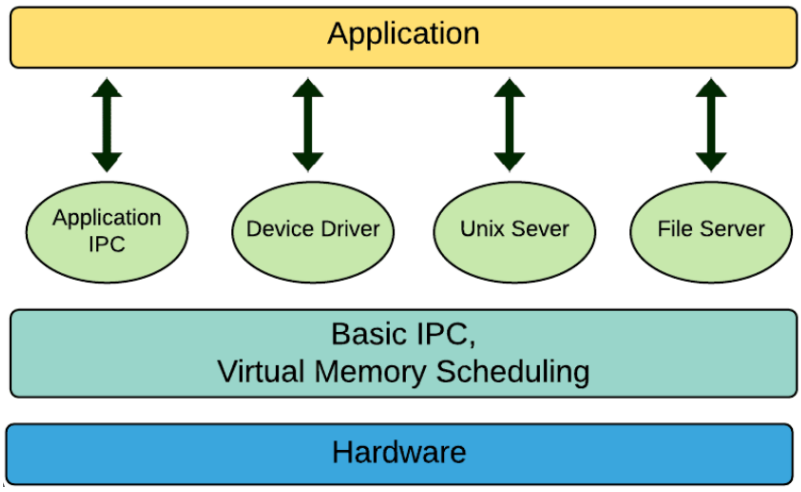
Cuando un procedimiento en un anillo exterior quería llamar a un procedimiento en un anillo interior, tenía que hacer el equivalente de una llamada al sistema; es decir, una instrucción TRAP cuyos parámetros se comprobaban para asegurarse de que fueran válidos antes de permitir que continuara la llamada.

[44] Multics (Multiplexed Information and Computing Service) fue uno de los primeros sistemas operativos de tiempo compartido y tuvo una gran influencia en el desarrollo de los posteriores sistemas operativos.



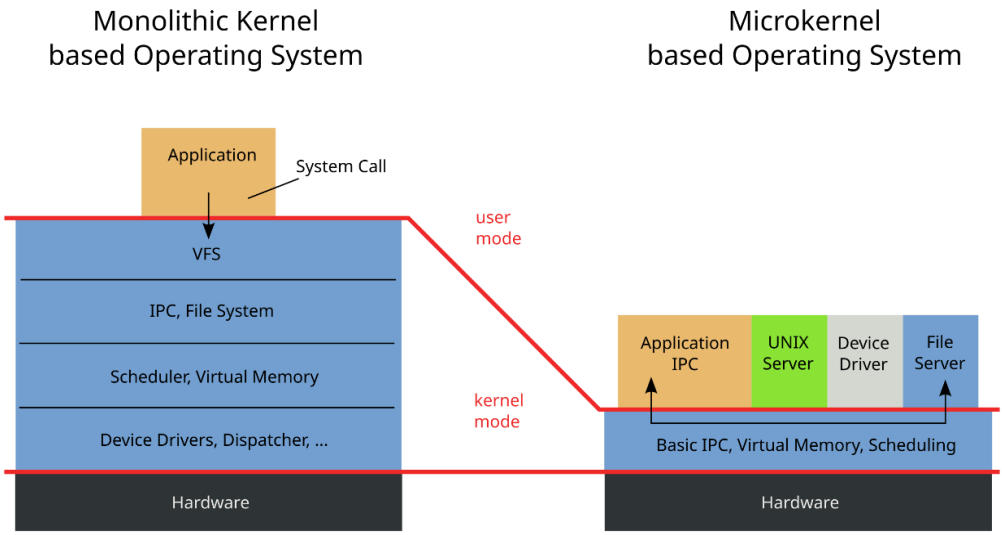
Microkernels.

Con el diseño de capas, los diseñadores podían elegir en dónde dibujar el límite entre kernel y usuario. Tradicionalmente todas las capas iban en el kernel, pero eso no es estrictamente necesario. De hecho, puede tener mucho sentido poner lo menos que sea posible en modo kernel, debido a que los errores en el kernel pueden paralizar el sistema de inmediato. En contraste, los procesos de usuario se pueden configurar para que tengan menos poder, por lo que un error en ellos tal vez no sería fatal.



La densidad de los errores depende del tamaño del módulo, su tiempo de vida y más, pero una cifra aproximada para los sistemas industriales formales es de diez errores por cada mil líneas

de código. Esto significa que es probable que un sistema operativo monolítico de cinco millones de líneas de código contenga cerca de 50,000 errores en el kernel.

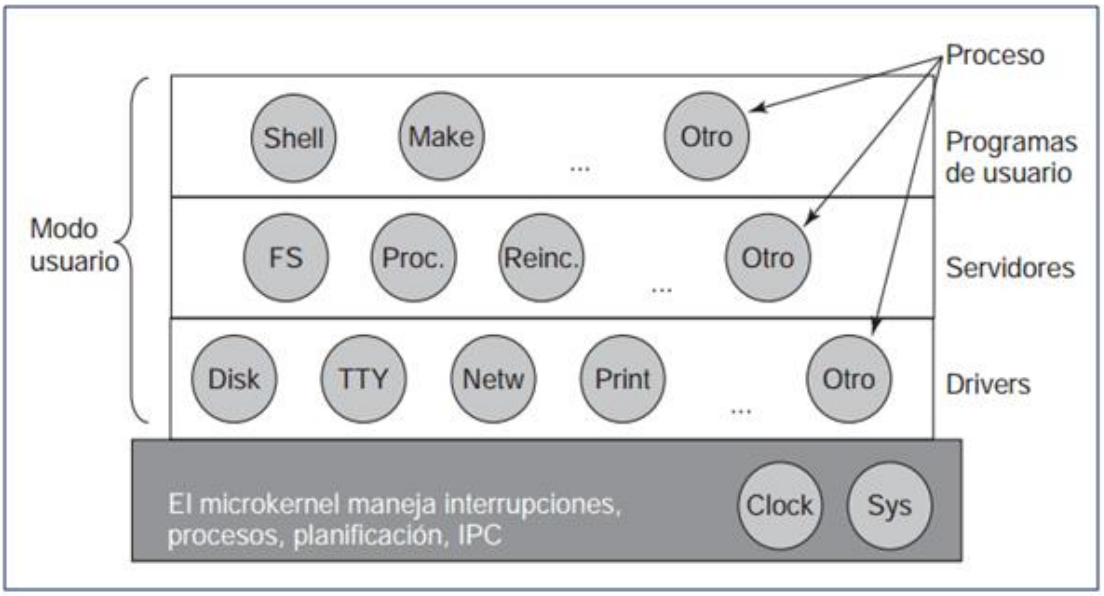


Desde luego que no todos los errores son fatales, ya que algunos pueden ser cosas tales como emitir un mensaje de error incorrecto en una situación que ocurre raras veces. Sin embargo, los sistemas operativos tienen tantos errores que los fabricantes de computadoras colocan botones de reinicio en ellas (a menudo en el panel frontal), algo que los fabricantes de televisiones, estéreos y autos no hacen, a pesar de la gran cantidad de software en estos dispositivos.

La idea básica detrás del diseño de microkernel es lograr una alta confiabilidad al dividir el sistema operativo en módulos pequeños y bien definidos, sólo uno de los cuales (el microkernel) se ejecuta en modo kernel y el resto se ejecuta como procesos de usuario ordinarios.

En especial, al ejecutar cada driver de dispositivo y sistema de archivos como un proceso de usuario separado, un error en alguno de estos procesos puede hacer que falle ese componente, pero no puede hacer que falle todo el sistema. Así, un error en el driver del dispositivo de audio hará que el sonido sea confuso o se detenga, pero la computadora no fallará. En contraste, en un sistema monolítico con todos los drivers en el kernel, un driver de audio con errores puede hacer fácilmente referencia a una dirección de memoria inválida y llevar a todo el sistema a un alto rotundo en un instante.

Los microkernels son especialmente comunes en las aplicaciones en tiempo real, industriales, aeronáuticas y militares que son de misión crítica y tienen requerimientos de confiabilidad muy altos. Un buen ejemplo de microkernel es el que podemos encontrar en el sistema operativo MINIX 3.



MINIX 3.

MINIX es un clon del sistema operativo Unix distribuido junto con su código fuente y desarrollado por el profesor Andrew S. Tanenbaum en 1987.

Debido al enfoque puramente educacional de MINIX, Tanenbaum no permitía que este fuera modificado demasiado ya que esto complicaría el sistema. Por estos motivos, Linus Torvalds decidió escribir su propio núcleo de sistema operativo (Linux) compatible con Unix.

MINIX 3 ha llevado la idea de modularidad hasta el límite, dividiendo la mayor parte del sistema operativo en varios procesos independientes en modo usuario.

MINIX 3 es un sistema conformidad con POSIX. Su microkernel sólo tiene cerca de 3200 líneas de C y 800 líneas de ensamblador para las funciones de muy bajo nivel, como las que se usan para atrapar interrupciones y conmutar procesos. El código de C administra y planifica los procesos, se encarga de la comunicación entre procesos (al pasar mensajes entre procesos) y ofrece un conjunto de aproximadamente 35 llamadas al kernel para permitir que el resto del sistema operativo realice su trabajo. Estas llamadas realizan funciones tales como asociar los drivers a las interrupciones, desplazar datos entre espacios de direcciones e instalar nuevos mapas de memoria para los procesos recién creados.

En MINIX 3, manejador de dispositivo para el reloj también está en el kernel, debido a que el planificador interactúa de cerca con él. Todos los demás dispositivos controladores se ejecutan como procesos de usuario separados.

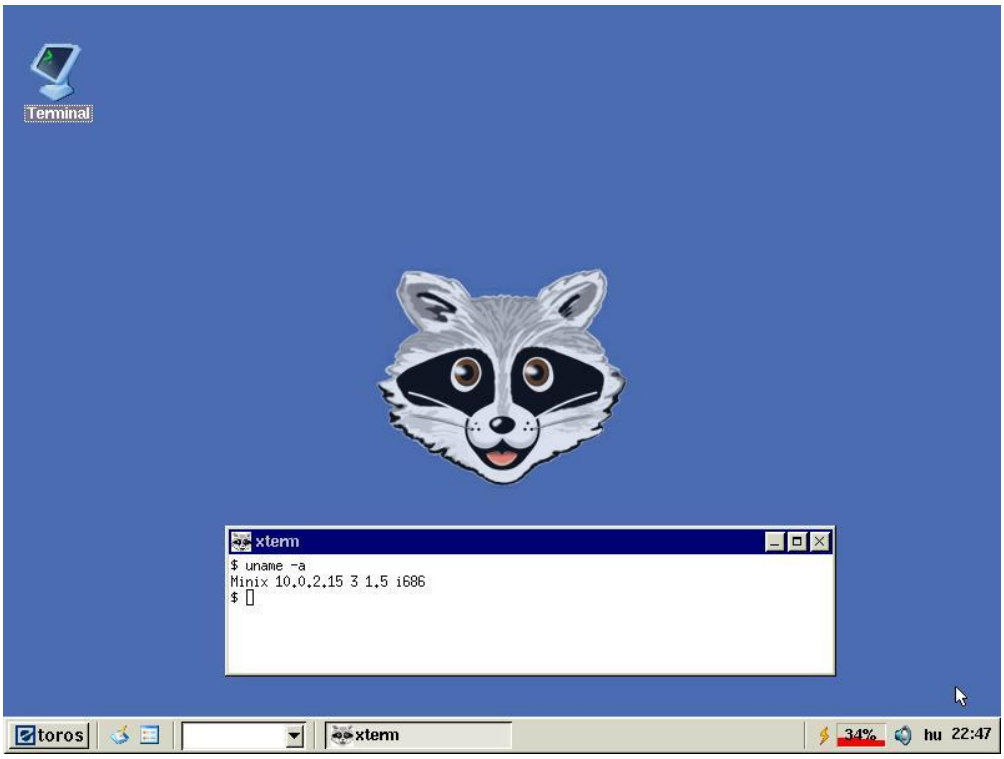
Fuera del kernel, el sistema se estructura en capas de procesos, y todos se ejecutan en modo usuario. La capa inferior contiene los drivers de dispositivos. Como todos se ejecutan en modo usuario, no tienen acceso físico al espacio de puertos de E/S y no pueden emitir comandos de E/S directamente. En consecuencia (y a diferencia de un diseño monolítico), un driver de audio defectuoso no puede escribir accidentalmente en el disco.

Encima de los drivers hay otra capa en modo usuario que contiene los servidores, que realizan la mayor parte del trabajo del sistema operativo. Uno o más servidores de archivos

administran el (los) sistema(s) de archivos, el administrador de procesos crea, destruye y administra los procesos y así sucesivamente.

Los programas de usuario obtienen servicios del sistema operativo mediante el envío de mensajes cortos a los servidores, pidiéndoles las llamadas al sistema POSIX. Por ejemplo, un proceso que necesite realizar una llamada read envía un mensaje a uno de los servidores de archivos para indicarle qué debe leer.

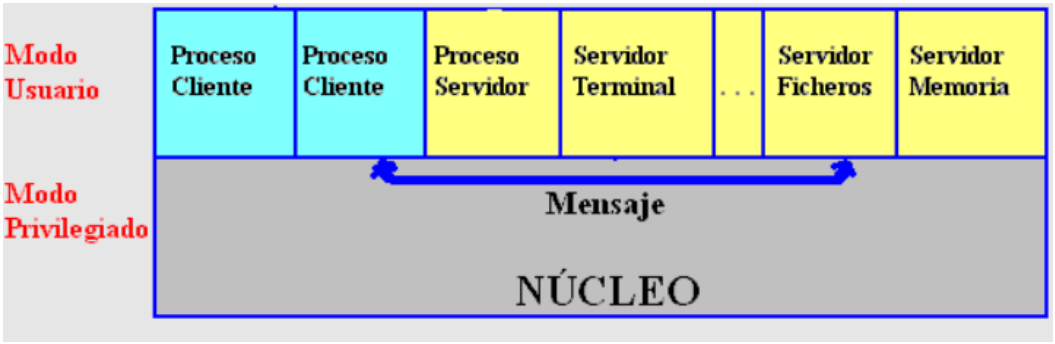
Un servidor interesante es el servidor de reencarnación, cuyo trabajo es comprobar si otros servidores y drivers están funcionando en forma correcta. En caso de que se detecte uno defectuoso, se reemplaza automáticamente sin intervención del usuario. De esta forma, el sistema es autocorregible y puede lograr una alta confiabilidad.



Modelo cliente-servidor.

Una ligera variación de la idea del microkernel es diferenciar dos clases de procesos: los servidores, cada uno de los cuales proporciona cierto servicio, y los clientes, que utilizan estos servicios.

Este modelo se conoce como cliente-servidor. A menudo la capa inferior es un microkernel, pero eso no es requerido. La esencia es la presencia de procesos cliente y procesos servidor.



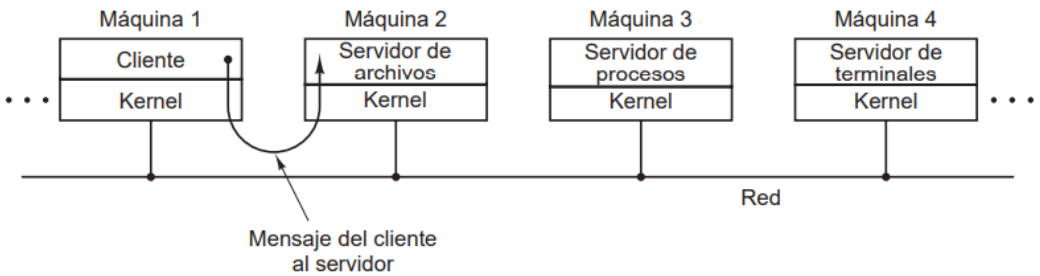
La comunicación entre clientes y servidores se lleva a cabo comúnmente mediante el paso de mensajes. Para obtener un servicio, un proceso cliente construye un mensaje indicando lo que desea y lo envía al servicio apropiado. Después el servicio hace el trabajo y envía de vuelta la respuesta.

Si el cliente y el servidor se ejecutan en el mismo equipo se pueden hacer ciertas optimizaciones, pero en concepto estamos hablando sobre el paso de mensajes.

Una generalización obvia de esta idea es hacer que los clientes y los servidores se ejecuten en distintas computadoras, conectadas mediante una red de área local o amplia. Como los clientes se comunican con los servidores mediante el envío de mensajes, no necesitan saber si los mensajes se manejan en forma local en sus propios equipos o si se envían a través de una red a servidores en un equipo remoto.

En cuanto a lo que al cliente concierne, lo mismo ocurre en ambos casos: se envían las peticiones y se regresan las respuestas. Por ende, el modelo cliente-servidor es una abstracción que se puede utilizar para un solo equipo o para una red de equipos.

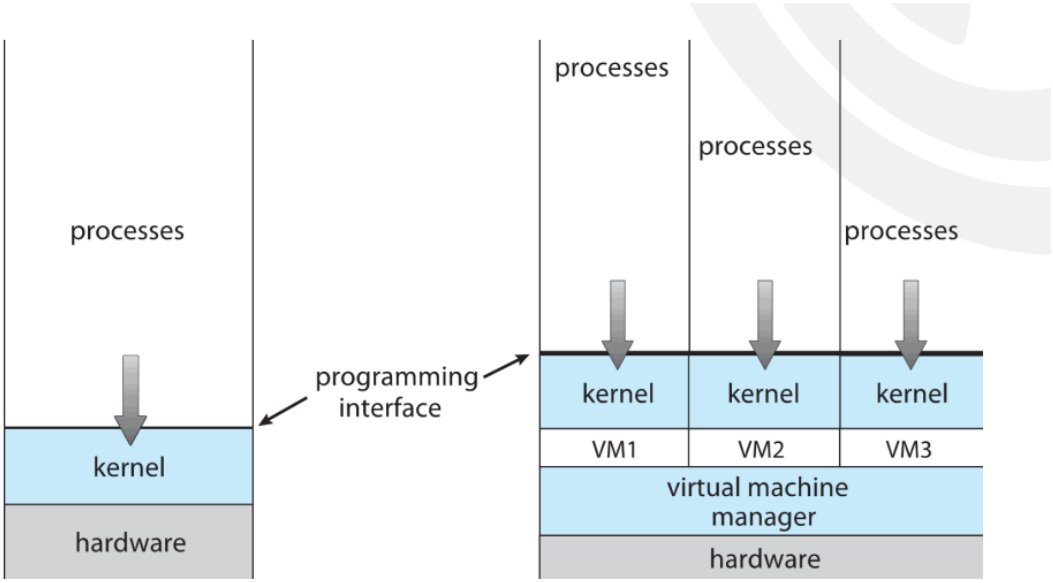
Existen muchos sistemas que involucran a los usuarios en sus PCs domésticas como clientes y equipos más grandes que operan en algún otro lado como servidores. De hecho, la mayor parte de la Web opera de esta forma. Una PC envía una petición de una página Web al servidor y la página Web se envía de vuelta. Éste es un uso común del modelo cliente-servidor en una red.



Máquinas virtuales.

La idea central de una máquina virtual es la abstracción del hardware de una computadora en varios ambientes de ejecución diferentes, creando la ilusión de que cada ambiente de ejecución está corriendo en su propia computadora privada.

Una máquina virtual provee una interfaz idéntica al hardware primitivo subyacente. Se crea la ilusión de múltiples procesos, cada uno ejecutando en su propio procesador con su propia memoria (virtual). Cada invitado es provisto con una copia (virtual) de la computadora.



Máquina no virtual

Máquina virtual

Muchas compañías han ejecutado tradicionalmente sus servidores de correo, servidores Web, servidores FTP y otros servidores en computadoras separadas, algunas veces con distintos sistemas operativos. Consideremos la virtualización como una forma de ejecutarlos todos en la misma máquina, sin que una falla de un servidor haga que falle el resto.

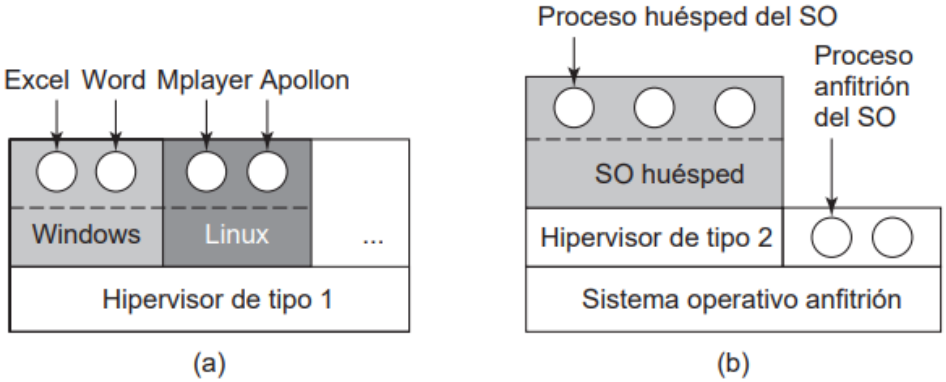
La virtualización también es popular en el mundo del hospedaje Web. Sin ella, los clientes de hospedaje Web se ven obligados a elegir entre el hospedaje compartido (que les ofrece sólo una cuenta de inicio de sesión en un servidor Web, pero ningún control sobre el software de servidor) y hospedaje dedicado (que les ofrece su propia máquina, lo cual es muy flexible pero no es costeable para los sitios Web de pequeños a medianos). Cuando una compañía de hospedaje Web ofrece la renta de máquinas virtuales, una sola máquina física puede ejecutar muchas máquinas virtuales, cada una de las cuales parece ser una máquina completa. Los clientes que rentan una máquina virtual pueden ejecutar cualesquier sistema operativo y software que deseen, pero a una fracción del costo de un servidor dedicado.

Otro uso de la virtualización es para los usuarios finales que desean poder ejecutar dos o más sistemas operativos al mismo tiempo, por ejemplo, Windows y Linux.

El problema de las máquinas virtuales está en su implementación. Para poder ejecutar software de máquina virtual en una computadora, su CPU debe ser virtualizable. Cuando un sistema operativo que funciona en una máquina virtual (en modo usuario) ejecuta una instrucción privilegiada, como realizar una operación de E/S, es esencial que el hardware la atrape para el monitor de la máquina virtual, de manera que la instrucción se pueda emular en el software. En algunas CPUs los intentos de ejecutar instrucciones privilegiadas en modo de usuario simplemente se ignoran.

Un administrador de máquinas virtuales (Virtual Machine Manager) o hipervisor, crea, administra y ejecuta las máquinas virtuales. Existen diferentes tipos:

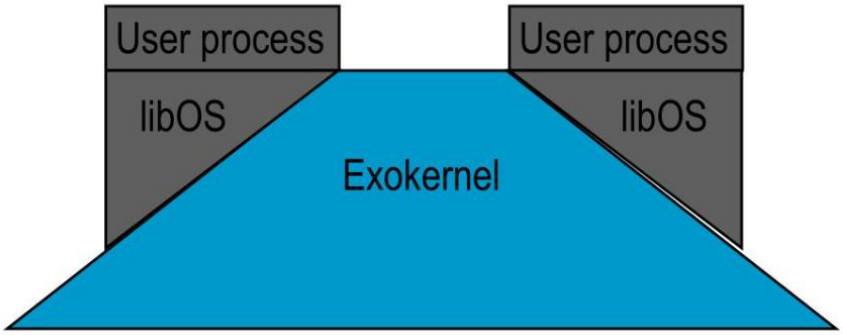
- Tipo 0: son soluciones basados en hardware, que proveen soporte para la creación y administración vía firmware.
- Tipo 1: se ejecutan directamente sobre el hardware de la máquina.
- Tipo 2: se ejecutan sobre el sistema operativo host que provee los servicios de virtualización.



Exokernels.

En vez de clonar la máquina actual, como se hace con las máquinas virtuales, otra estrategia es particionarla; en otras palabras, a cada usuario se le proporciona un subconjunto de los recursos. Así, una máquina virtual podría obtener los bloques de disco del 0 al 1023, la siguiente podría obtener los bloques de disco del 1024 al 2047 y así sucesivamente.

En la capa inferior, que se ejecuta en el modo kernel, hay un programa llamado exokernel. Su propósito es crear una especie de capa de software para otros sistemas virtuales.

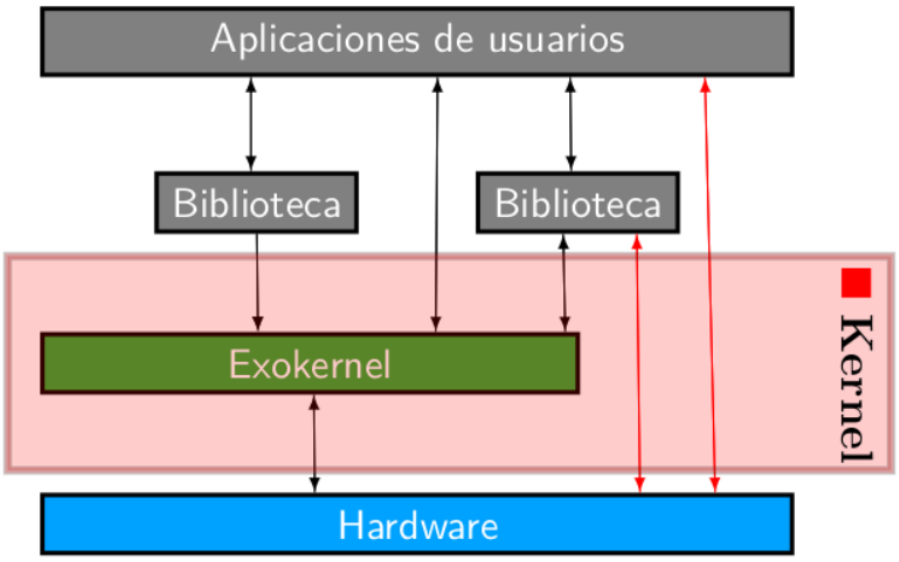


El exokernel se encarga de asignar recursos a las máquinas virtuales y después comprobar los intentos de utilizarlos, para asegurar que ninguna máquina trate de usar los recursos de otra. Cada máquina virtual de nivel de usuario puede ejecutar su propio sistema operativo y cada una está restringida a utilizar sólo los recursos que ha pedido y que le han sido asignados.

Entonces, el núcleo se encarga de destinar los recursos físicos de la máquina (por ejemplo, bloques de disco, páginas de memoria y el tiempo de CPU) a múltiples aplicaciones, y dejar a cada programa decidir qué hacer con esos recursos. El programa podría enlazar una librería de sistema operativo (***libOS***) que implementa abstracciones comunes, o podría implementar sus propias abstracciones.

Teóricamente un sistema basado en exokernel podría proveer varias libOS, así como diferentes tipos de sistema operativo podrían ejecutarse bajo el mismo exokernel.

La ventaja del esquema del exokernel es que ahorra una capa de asignación. En los otros diseños, cada máquina virtual piensa que tiene su propio disco, con bloques que van desde 0 hasta cierto valor máximo, por lo que el monitor de la máquina virtual debe mantener tablas para reasignar las direcciones del disco (y todos los demás recursos). Con el exokernel, esta reasignación no es necesaria. El exokernel sólo necesita llevar el registro para saber a cuál máquina virtual se le ha asignado cierto recurso.



Exokernel del MIT.

El MIT desarrolló dos sistemas operativos basados en exokernel, que empleaban dos núcleos: Aegis[45], una prueba del concepto con soporte limitado para almacenamiento, y XOK[46], que aplicaba el concepto de exokernel más directamente.

Una idea esencial del sistema exokernel del MIT era que el sistema operativo debería actuar como un ejecutador para pequeños programas suministrados por la aplicación, que sólo están sujetos al requisito de que el exokernel debe ser capaz de garantizar que emplean el hardware de forma segura.

Consta de 3 capas, la capa del exokernel, la capa libOS (núcleos de varios sistemas operativos corriendo sobre exokernel) y la capa de aplicaciones, corriendo sobre LibOS.

Referencias y bibliografía de la sección.

- Tanenbaum 1-7.
- KMC - Sistemas operativos - Máquinas virtuales.

Más sobre el modelo de capas.

A medida que se han añadido más características a los sistemas operativos y el hardware subyacente se ha vuelto más potente y versátil, han crecido el tamaño y la complejidad de estos sistemas.

CTSS[47], que fue puesto en marcha en el MIT en 1963, estaba compuesto aproximadamente por 32.000 palabras de almacenamiento de 36 bits. OS/360, presentado por IBM un año después, tenía más de un millón de instrucciones de máquina. En 1975, el sistema Multics, desarrollado por MIT y los laboratorios Bell, había superado los 20 millones de instrucciones.

El tamaño de un sistema operativo con un conjunto completo de características, ha derivado en cuatro comunes problemas.

En primer lugar, los sistemas operativos se entregan tarde de forma crónica. Esto implica la creación de nuevos sistemas operativos y frecuentes actualizaciones a viejos sistemas.

---

[45] MIT AEGIS pertenece al tipo de estructura de sistema operativo conocida como Microkernels. Este tipo de sistema operativo se caracteriza por tener un núcleo mínimo que gestiona tareas básicas, delegando la mayoría de las funciones del sistema a procesos independientes.

[46] XOK fue un sistema operativo desarrollado por el Instituto Tecnológico de Massachusetts (MIT) como parte de sus investigaciones sobre exonúcleo.

[47] CTSS, que significa Compatible Time-Sharing System (Sistema de Tiempo Compartido Compatible), fue uno de los primeros sistemas operativos de tiempo compartido; fue desarrollado en el Centro de Computación del MIT.

En segundo lugar, los sistemas tienen fallos latentes que deben ser planteados y resueltos.

En tercer lugar, el rendimiento no es frecuentemente el esperado.

En último lugar, se ha comprobado que es imposible construir un sistema operativo complejo que no sea vulnerable a una gran cantidad de ataques de seguridad, incluyendo virus, gusanos y accesos no autorizados.

Para gestionar la complejidad de los sistemas operativos y eliminar estos problemas, se ha puesto mucho énfasis en la estructura software del sistema operativo a lo largo de los años. El software debe ser modular. Esto ayudará a organizar el proceso de desarrollo de software y limitará el esfuerzo de diagnosticar y corregir errores.

Los módulos deben tener interfaces bien definidas, y estas interfaces deben ser tan sencillas como sea posible. Esto facilita la evolución del sistema. Con mínimas interfaces entre los módulos, se puede modificar un módulo con un mínimo impacto en otros.

Para sistemas operativos grandes, que ejecutan desde millones a decenas de millones de líneas de código, no es suficiente la programación modular. De hecho, ha habido un incremento en el uso de los conceptos de capas jerárquicas y abstracción de información.

La estructura jerárquica de un sistema operativo moderno separa sus funciones de acuerdo a las características de su escala de tiempo y su nivel de abstracción. Se puede ver al sistema como una serie de niveles. Cada nivel realiza un subconjunto relacionado de funciones requeridas por el sistema operativo. Dicho nivel confía en los niveles inmediatamente inferiores para realizar funciones más primitivas y ocultar los detalles de esas funciones.

Cada nivel proporciona servicios a la capa inmediatamente superior. Idealmente, estos niveles deben definirse de tal forma que los cambios en un nivel no requieran cambios en otros niveles. Por tanto, de esta forma se ha descompuesto un problema en un número de subproblemas más manejables.

En general, las capas inferiores tratan con una escala de tiempo menor. Algunas partes del sistema operativo deben interaccionar directamente con el hardware de la computadora, donde los eventos pueden tener una escala de tiempo tan ínfima como unas pocas mil millonésimas de segundo.

En el otro extremo del espectro, algunas partes del sistema operativo se comunican con el usuario, que invoca mandatos en una escala de tiempo mucho más larga, tal vez unos pocos segundos.

El uso de un conjunto de niveles se adapta adecuadamente a este entorno. La forma en que estos principios se aplican varía enormemente entre los sistemas operativos contemporáneos.

Nivel	Nombre	Objetos	Ejemplos de operaciones
13	Intérprete de mandatos	Entorno de programación de usuario	Sentencias en lenguaje del intérprete de mandatos
12	Procesos de usuario	Procesos de usuario	Salir, matar, suspender, continuar
11	Directorios	Directorios	Crear, destruir, insertar entrada, eliminar entrada, buscar, listar
10	Dispositivos	Dispositivos externos, como impresoras, pantallas y teclados	Abrir, cerrar, leer, escribir
9	Sistema de ficheros	Ficheros	Crear, destruir, abrir, cerrar, leer, escribir
8	Comunicaciones	Tuberías	Crear, destruir, abrir, cerrar, leer, escribir
7	Memoria virtual	Segmentos, páginas	Leer, escribir, cargar
6	Almacenamiento secundario local	Bloques de datos, canales de dispositivo	Leer, escribir, asignar, liberar
5	Procesos primitivos	Procesos primitivos, semáforos, lista de procesos listos	Suspender, continuar, esperar, señalizar
4	Interrupciones	Programas de gestión de interrupciones	Invocar, enmascarar, desenmascarar, reintentar
3	Procedimientos	Procedimientos, pila de llamadas, registro de activación	Marcar la pila, llamar, retornar
2	Conjunto de instrucciones	Pila de evaluación, intérprete de microprogramas, datos escalares y vectoriales	Cargar, almacenar, sumar, restar, saltar
1	Circuitos electrónicos	Registros, puertas, buses, etc.	Poner a 0, transferir, activar, complementar

El área sombreado en gris representa al hardware.

A modo de ejemplo, el modelo de la tabla anterior está compuesto por los siguientes niveles:

• **Nivel 1.** Está formado por circuitos electrónicos, donde los objetos tratados son registros, celdas de memoria, y puertas lógicas. Las operaciones definidas en estos objetos son acciones, como poner a cero un registro o leer una posición de memoria.

• **Nivel 2.** El conjunto de instrucciones del procesador. Las operaciones a este nivel son aquéllas permitidas en el conjunto de instrucciones de lenguaje máquina, como adición, resta, carga o almacenamiento.

• **Nivel 3.** Añade el concepto de **procedimiento** o **subrutina**, más las operaciones de llamada y retorno (call/return).

• **Nivel 4.** Introduce las interrupciones, que permiten al procesador guardar el contexto actual e invocar una rutina de tratamiento de interrupciones.

Estos cuatros primeros niveles no son parte del sistema operativo, sino que constituyen el hardware del procesador. Sin embargo, algunos elementos del sistema operativo se empiezan a mostrar en estos niveles, por ejemplo, las rutinas de tratamiento de interrupciones. Es el Nivel 5 el que corresponde con el sistema operativo propiamente dicho y en el que los conceptos asociados a la multiprogramación aparecen.

• **Nivel 5.** En este nivel se introduce la noción de un proceso como un programa en ejecución. Los requisitos fundamentales de los sistemas operativos para dar soporte a múltiples procesos incluyen la habilidad de suspender y continuar los procesos. Esto requiere guardar los registros

de hardware de forma que se pueda interrumpir la ejecución de un proceso e iniciar la de otro. Adicionalmente, si los procesos necesitan cooperar, se necesitan algunos métodos de sincronización.

• **Nivel 6.** Trata los dispositivos de almacenamiento secundario de la computadora. En este nivel, se dan las funciones para posicionar las cabezas de lectura/escritura y la transferencia real de bloques. El Nivel 6 delega al Nivel 5 la planificación de la operación y la notificación al proceso solicitante de la finalización de la misma. Niveles más altos se preocupan de la dirección en el disco de los datos requeridos y proporcionan una petición del bloque de datos apropiado a un controlador de dispositivo del Nivel 5.

• **Nivel 7.** Crea un espacio de direcciones lógicas para los procesos. Este nivel organiza el espacio de direcciones virtuales en bloques que pueden moverse entre memoria principal y memoria secundaria.

Hasta este punto, el sistema operativo trata con los recursos de un único procesador. Comenzando con el Nivel 8, el sistema operativo trata con objetos externos, como dispositivos periféricos y posiblemente redes y computadores conectados a la red. Los objetos de estos niveles superiores son objetos lógicos con nombre, que pueden compartirse entre procesos de la misma computadora o entre múltiples equipos.

• **Nivel 8.** Trata con la comunicación de información y mensajes entre los procesos. Mientras que el Nivel 5 proporciona un mecanismo de señalización primitivo que permite la sincronización de procesos, este nivel trata con una compartición de información más rica.

• **Nivel 9.** Da soporte al almacenamiento a largo plazo en archivos con nombre. En este nivel, los datos en el almacenamiento secundario se ven en términos de entidades abstractas y con longitud variable. Esto contrasta con la visión orientada a hardware del almacenamiento secundario en términos de pistas, sectores y bloques de tamaño fijo en el Nivel 6.

• **Nivel 10.** Proporciona acceso a los dispositivos externos utilizando interfaces estándar.

• **Nivel 11.** Es el nivel responsable para mantener la asociación entre los identificadores externos e internos de los recursos y objetos del sistema. El identificador externo es un nombre que puede utilizar una aplicación o usuario. El identificador interno es una dirección de otro identificador que puede utilizarse por parte de los niveles inferiores del sistema operativo para canalizar y controlar un objeto. Estas asociaciones se mantienen en un directorio. Las entradas no sólo incluyen la asociación entre identificadores externos/internos, sino también características como los derechos de acceso.

• **Nivel 12.** Proporciona una utilidad completa para dar soporte a los procesos. Este nivel va más allá del proporcionado por el Nivel 5. En el Nivel 5, sólo se mantienen los contenidos de los registros del procesador asociados con un proceso, más la lógica de los procesos de planificación. En el Nivel 12, se da soporte a toda la información necesaria para la gestión ordenada de los procesos. Esto incluye el espacio de direcciones virtuales de los procesos, una lista de objetos y procesos con la cual puede interactuar y las restricciones de esta interacción, parámetros pasados al proceso en la creación. También se incluye cualquier otra característica que pudiera utilizar el sistema operativo para controlar el proceso.

• **Nivel 13.** Proporciona una interfaz del sistema operativo al usuario. Se denomina **shell**. Separa al usuario de los detalles de los sistemas operativos y presenta al sistema simplemente como una colección de servicios. En este nivel, la interfaz podría implementarse de una

manera gráfica, proporcionando al usuario comandos a través de una lista presentada como un menú.

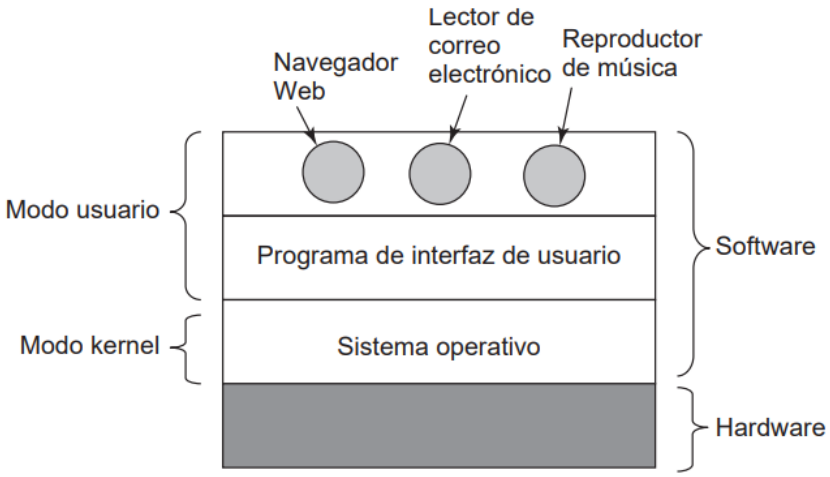
Este modelo hipotético de un sistema operativo proporciona una estructura descriptiva útil y sirve como una guía de implementación.

Referencias y bibliografía de la sección.

- Stallings pág. 76.

Definición de modo núcleo y usuario.

Sabemos que el sistema operativo es un software al igual que las aplicaciones de usuario. Si tratamos de ubicar este sistema con respecto al hardware, notaremos que se encuentra en la capa más cercana a los componentes físicos.



La mayoría de las computadoras tienen dos modos de operación: ***modo kernel*** y ***modo usuario***. El sistema operativo es la pieza fundamental del software y se ejecuta en modo kernel (también conocido como modo supervisor). En este modo, el sistema operativo tiene acceso completo a todo el hardware y puede acceder a cualquier instrucción que la máquina sea capaz de ejecutar.

El resto del software se ejecuta en modo usuario, en el cual sólo un subconjunto de las instrucciones de máquina es permitido. En particular, las instrucciones que afectan el control de la máquina o que se encargan de la E/S (entrada/salida) están prohibidas para los programas en modo usuario.

Una distinción importante entre el sistema operativo y el software que se ejecuta en modo usuario es que, si a un usuario no le gusta, por ejemplo, su lector de correo electrónico, es libre de conseguir otro o incluso escribir el propio si así lo desea; sin embargo, no es libre de

escribir su propio manejador de interrupciones de reloj, que forma parte del sistema operativo y está protegido por el hardware contra cualquier intento de modificación por parte de los usuarios.

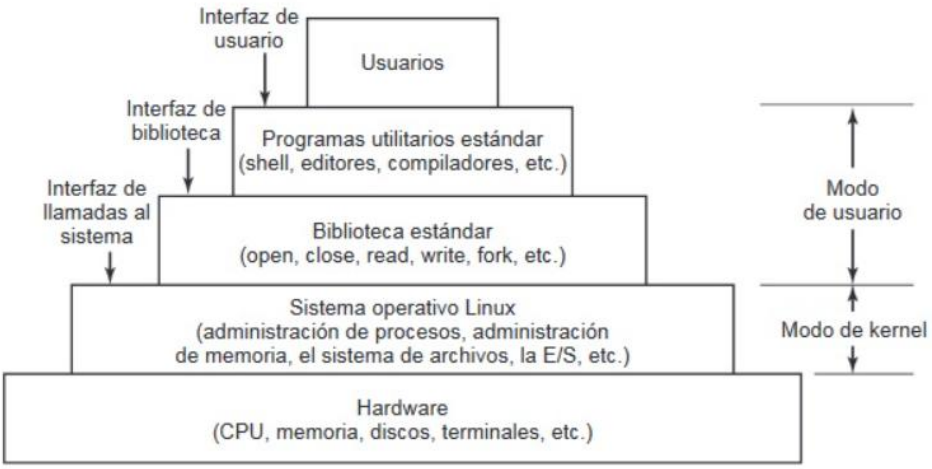
En ciertos sistemas, la idea se lleva hasta el extremo y partes de lo que tradicionalmente se considera el sistema operativo (por ejemplo, el sistema de archivos) se ejecutan en el espacio del usuario. En dichos sistemas es difícil trazar un límite claro. Todo lo que se ejecuta en modo kernel forma, sin duda, parte del sistema operativo, pero podría decirse que algunos programas que se ejecutan fuera de este modo también forman parte del mismo sistema, o por lo menos están estrechamente asociados a él.

Si tomamos de ejemplo a los sistemas operativos basados en Linux, notaremos que la mayoría de estos principios se cumplen de forma bastante precisa.

Podríamos decir que en Linux el kernel controla todas las funciones principales del hardware, ya sea un teléfono, una computadora portátil, un servidor o cualquier otro tipo de equipo.

En este tipo de sistemas el kernel cumple cuatro tareas:

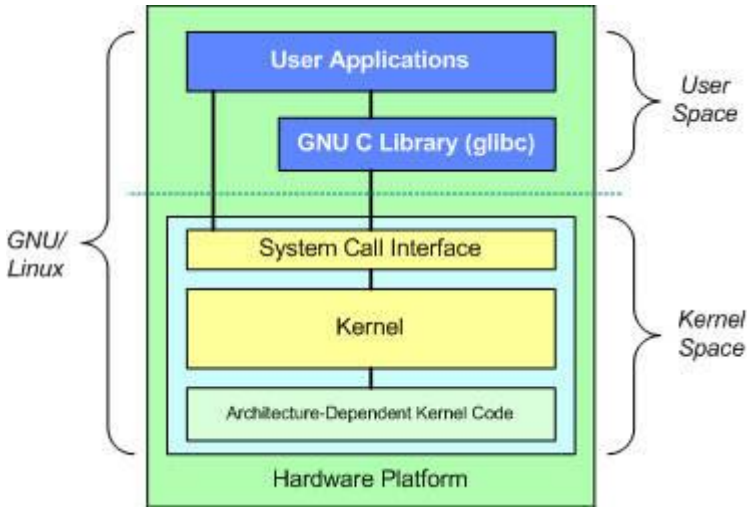
- Gestión de la memoria: supervisa cuánta memoria se utiliza para almacenar qué tipo de elementos, así como el lugar en que los guarda.
- Gestión de los procesos: determina qué procesos pueden usar la unidad central de procesamiento (CPU), cuándo y durante cuánto tiempo.
- Controladores de dispositivos: actúa como mediador o intérprete entre el hardware y los procesos.
- Seguridad y llamadas al sistema: recibe solicitudes de servicio por parte de los procesos.



La memoria se divide en espacio del kernel y del usuario. Esta división conforma la base de algunas operaciones complejas, como la separación de privilegios para la seguridad y las máquinas virtuales.

Si un proceso falla en el modo usuario, el daño es limitado, y el kernel puede recuperarlo. Por otro lado, debido al acceso del kernel a la memoria y al procesador, un fallo en uno de sus

procesos puede perjudicar a todo el sistema. Dado que hay mecanismos de protección y se requieren permisos para cruzar los límites, los fallos en los procesos del usuario no suelen causar demasiados problemas.

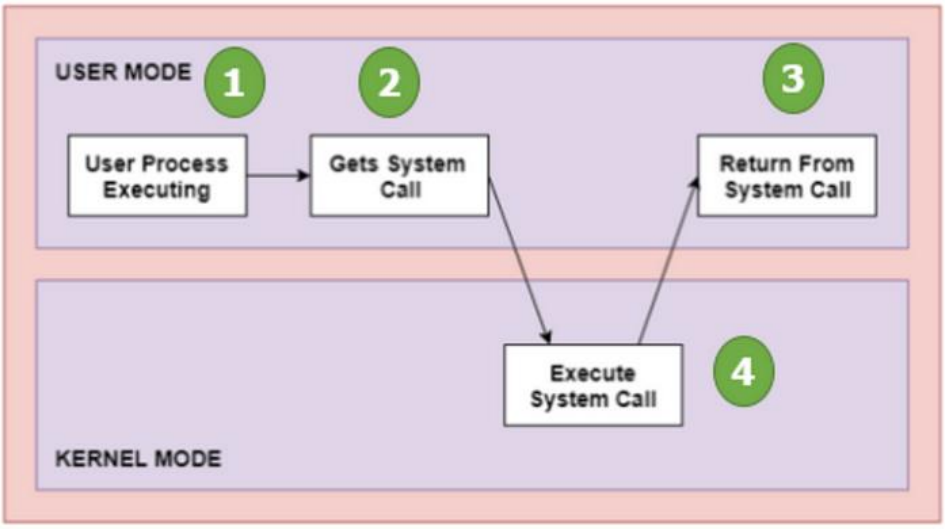


Referencias y bibliografía de la sección.

- Tanenbaum 1.
- Red Hat - ¿Qué es el kernel Linux y cuáles son sus funciones?

Llamadas al sistema.

Los sistemas operativos tienen dos funciones principales: proveer abstracciones a los programas de usuario y administrar los recursos de la computadora. En su mayor parte, la interacción entre los programas de usuario y el sistema operativo se relaciona con la primera función; por ejemplo, crear, escribir, leer y eliminar archivos.

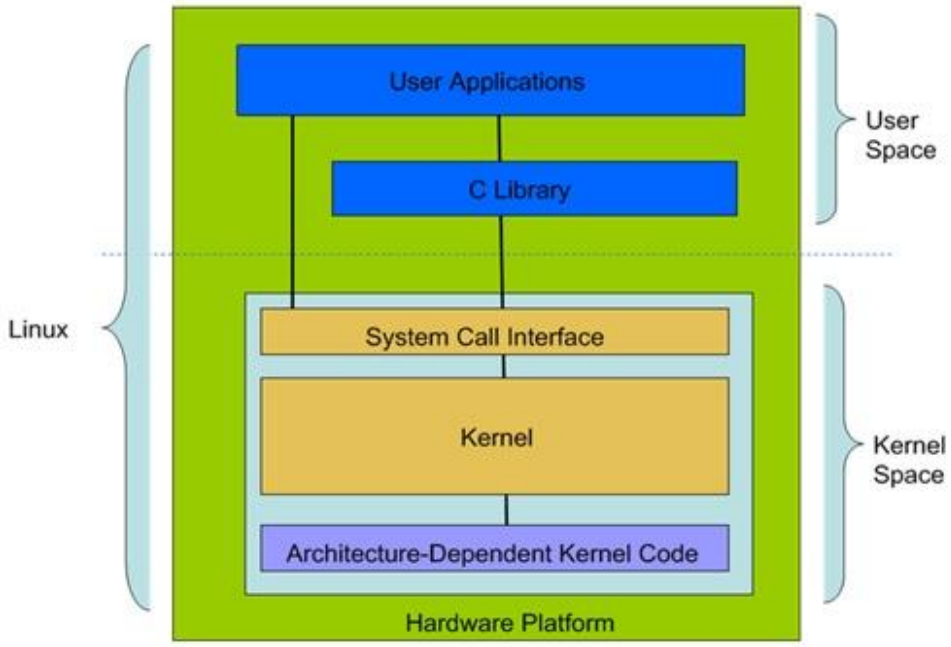


La parte de la administración de los recursos es en gran parte transparente para los usuarios y se realiza de manera automática. Por ende, la interfaz entre los programas de usuario y el sistema operativo trata principalmente acerca de cómo lidiar con las abstracciones.

Las llamadas al sistema disponibles varían de un sistema operativo a otro, aunque los conceptos subyacentes tienden a ser similares.

Cualquier computadora con una sola CPU puede ejecutar sólo una instrucción a la vez. Si un proceso está ejecutando un programa de usuario en modo usuario y necesita un servicio del sistema, como leer datos de un archivo, tiene que ejecutar una instrucción de TRAP para transferir el control al sistema operativo. Después, el sistema operativo averigua qué es lo que quiere el proceso llamador, para lo cual inspecciona los parámetros. Luego, lleva a cabo la llamada al sistema y devuelve el control a la instrucción que va después de la llamada.

En cierto sentido, realizar una llamada al sistema es como realizar un tipo especial de llamada a un procedimiento, sólo que las llamadas al sistema entran al kernel y las llamadas a procedimientos no.



Si bien podríamos decir que las llamadas al sistema están dentro del kernel en el sentido de que el código que las atiende vive en el núcleo, también podríamos entender a estas llamadas como la puerta de entrada al kernel desde un programa en modo usuario. Son la interfaz que el kernel expone para que las aplicaciones puedan pedirle servicios (abrir un archivo, crear un proceso, reservar memoria, etc.).

```
[Programa en modo usuario]
        |
        v
[Función de biblioteca: printf(), open(), etc.]
        |
        v
[Llamada al sistema -> transición a modo kernel]
        |
        v
[Servicio del kernel]
        |
        v
[Resultado de vuelta al modo usuario]
```

Para hacer más entendible el mecanismo de llamadas al sistema, vamos a dar un vistazo rápido a la llamada al sistema read, muy común en sistemas tipo UNIX.

Tiene tres parámetros: el primero especifica el archivo, el segundo apunta al búfer y el tercero proporciona el número de bytes a leer.

Al igual que casi todas las llamadas al sistema, se invoca desde programas en C mediante una llamada a un procedimiento de la biblioteca con el mismo nombre que la llamada al sistema, read.

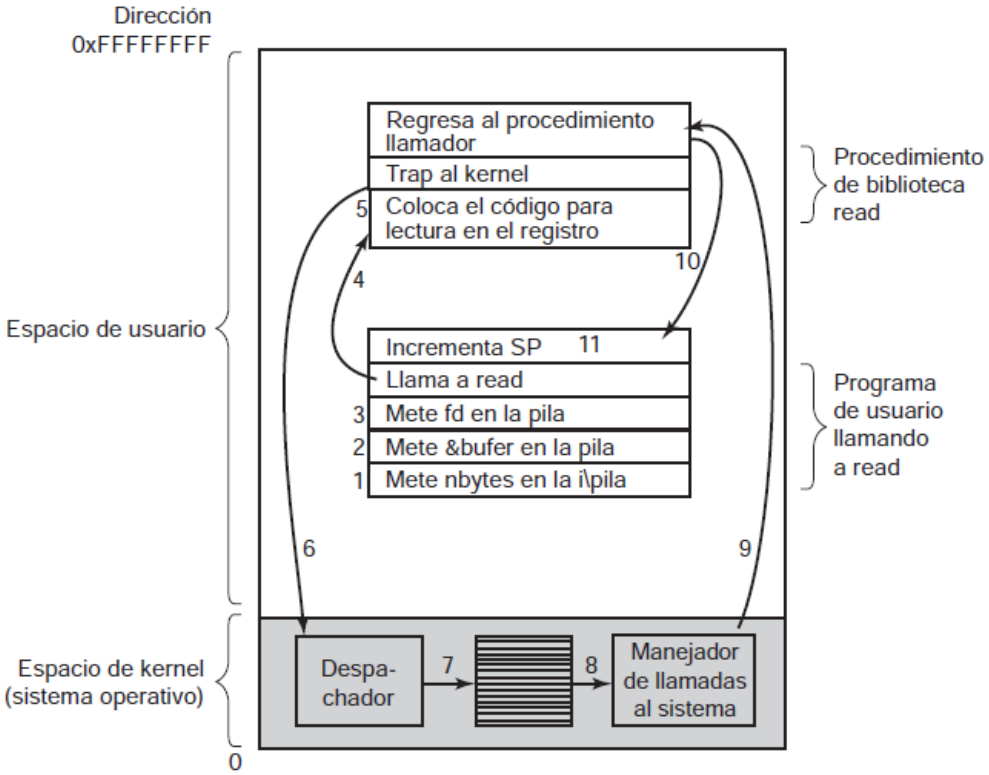
Una llamada desde un programa en C podría tener la siguiente apariencia:

```
cuenta=read(fd, bufer, nbytes);
```

La llamada al sistema (y el procedimiento de biblioteca) devuelve el número de bytes que se leen en cuenta. Por lo general este valor es el mismo que nbytes pero puede ser más pequeño si, por ejemplo, se encuentra el fin de archivo al estar leyendo.

Si la llamada al sistema no se puede llevar a cabo, ya sea debido a un parámetro inválido o a un error del disco, cuenta se establece a 1 y el número de error se coloca en una variable global llamada errno. Los programas siempre deben comprobar los resultados de una llamada al sistema para ver si ocurrió un error.

Las llamadas al sistema se llevan a cabo en una serie de pasos. En su preparación para llamar al procedimiento de biblioteca read, que es quien realmente hace la llamada al sistema read, el programa llamador primero mete los parámetros en la pila, como se muestra en los pasos 1 a 3 de la figura.



Los parámetros primero y tercero se pasan por valor, pero el segundo parámetro se pasa por referencia, lo cual significa que se pasa la dirección del búfer (lo cual se indica mediante &), no el contenido del mismo.

Después viene la llamada al procedimiento de biblioteca (paso 4). Esta instrucción es la de llamada a procedimiento normal utilizada para llamar a todos los procedimientos.

El procedimiento de biblioteca (probablemente escrito en lenguaje ensamblador) coloca por lo general el número de la llamada al sistema en un lugar en el que el sistema operativo lo espera, como en un registro (paso 5).

Después ejecuta una instrucción TRAP para cambiar del modo usuario al modo kernel y empezar la ejecución en una dirección fija dentro del núcleo (paso 6).

La instrucción TRAP en realidad es muy similar a la instrucción de llamada a procedimiento en el sentido en que la instrucción que le sigue se toma de una ubicación distante y la dirección de retorno se guarda en la pila para un uso posterior.

Sin embargo, la instrucción TRAP también difiere de la instrucción de llamada a un procedimiento en dos formas básicas. En primer lugar, como efecto secundario, cambia a modo kernel. La instrucción de llamada al procedimiento no cambia el modo. En segundo lugar, en vez de dar una dirección relativa o absoluta en donde se encuentra el procedimiento, la instrucción TRAP no puede saltar a una dirección arbitraria. Dependiendo de la arquitectura, salta a una ubicación fija.

El código de kernel que empieza después de la instrucción TRAP examina el número de llamada al sistema y después la pasa al manejador correspondiente de llamadas al sistema, por lo general a través de una tabla de apuntadores a manejadores de llamadas al sistema, indexados en base al número de llamada al sistema (paso 7).

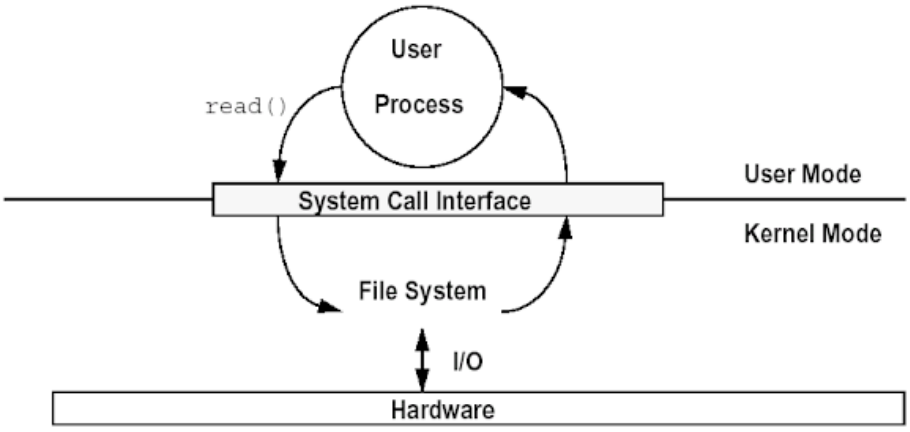
Luego, se ejecuta el manejador de llamadas al sistema (paso 8). Una vez que el manejador ha terminado su trabajo, el control se puede regresar al procedimiento de biblioteca que está en espacio de usuario, en la instrucción que va después de la instrucción TRAP (paso 9).

Seguidamente, este procedimiento regresa al programa de usuario en la forma usual en que regresan las llamadas a procedimientos (paso 10).

Para terminar el trabajo, el programa de usuario tiene que limpiar la pila, como lo hace después de cualquier llamada a un procedimiento (paso 11).

Ahora el programa es libre de hacer lo que quiera a continuación.

La llamada al sistema puede bloquear al procedimiento llamador, evitando que continúe. Por ejemplo, si trata de leer del teclado y no se ha escrito nada aún, el procedimiento llamador tiene que ser bloqueado. En este caso, el sistema operativo buscará a su alrededor para ver si se puede ejecutar algún otro proceso a continuación. Más adelante, cuando esté disponible la entrada deseada, este proceso recibirá la atención del sistema y se llevarán a cabo los pasos 9 a 11.

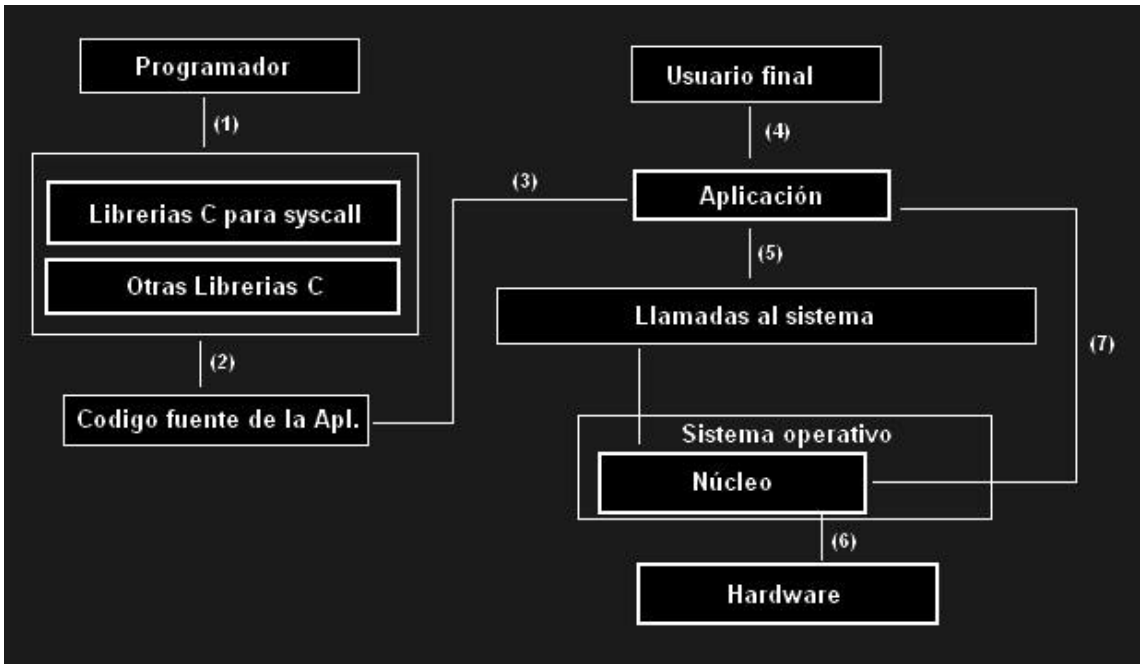


En la imagen anterior vemos claramente como una llamada al sistema es el mecanismo por el cual los procesos y aplicaciones de usuario acceden a los servicios del núcleo (***syscall read ()***).

El ***modo usuario*** necesita acceder al disco para leer, para ello utiliza la ***syscall read ()*** utilizando la interfaz de llamadas al sistema. El núcleo atiende la petición accediendo al hardware y devolviendo el resultado al proceso que inició la petición.

Para hacer uso de las llamadas al sistema desde el lenguaje de programación C, los sistemas operativos que trabajan con el núcleo Linux ponen a disposición del usuario varias ***funciones*** o ***procedimientos de librería*** que representan a las llamadas del sistema.

El sistema operativo de núcleo Linux cuenta con aproximadamente 200 funciones relacionadas con cada llamada al sistema, algunos de los cuales pueden agruparse en ciertas categorías que permiten el manejo o control de: procesos, señales, archivos, tiempo, etc.



Referencias y bibliografía de la sección.

- Tanenbaum 1-6.
- Silberschatz 2-3.

Clasificación de llamadas al sistema.

Administración de procesos

Llamada	Descripción
pid = fork()	Crea un proceso hijo, idéntico al padre
pid = waitpid(pid, &statloc, opciones)	Espera a que un hijo termine
s = execve(nombre, argv, entornp)	Reemplaza la imagen del núcleo de un proceso
exit(estado)	Termina la ejecución de un proceso y devuelve el estado

Administración de archivos

Llamada	Descripción
fd = open(archivo, como, ...)	Abre un archivo para lectura, escritura o ambas
s = close(fd)	Cierra un archivo abierto
n = read(fd, bufer, nbytes)	Lee datos de un archivo y los coloca en un búfer
n = write(fd, bufer, nbytes)	Escribe datos de un búfer a un archivo
posicion = lseek(fd, desplazamiento, dedonde)	Desplaza el apuntador del archivo
s = stat(nombre, &buf)	Obtiene la información de estado de un archivo

Administración del sistema de directorios y archivos

Llamada	Descripción
s = mkdir(nombre, modo)	Crea un nuevo directorio
s = rmdir(nombre)	Elimina un directorio vacío
s = link(nombre1, nombre2)	Crea una nueva entrada llamada nombre2, que apunta a nombre1
s = unlink(nombre)	Elimina una entrada de directorio
s = mount(especial, nombre, bandera)	Monta un sistema de archivos
s = umount(especial)	Desmonta un sistema de archivos

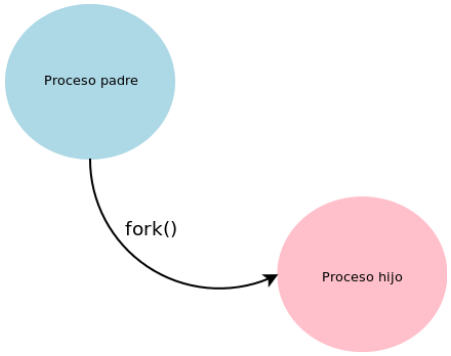
Llamadas varias

Llamada	Descripción
s = chdir(nombredir)	Cambia el directorio de trabajo
s = chmod(nombre, modo)	Cambia los bits de protección de un archivo
s = kill(pid, senial)	Envía una señal a un proceso
segundos = tiempo(&segundos)	Obtiene el tiempo transcurrido desde Ene 1, 1970

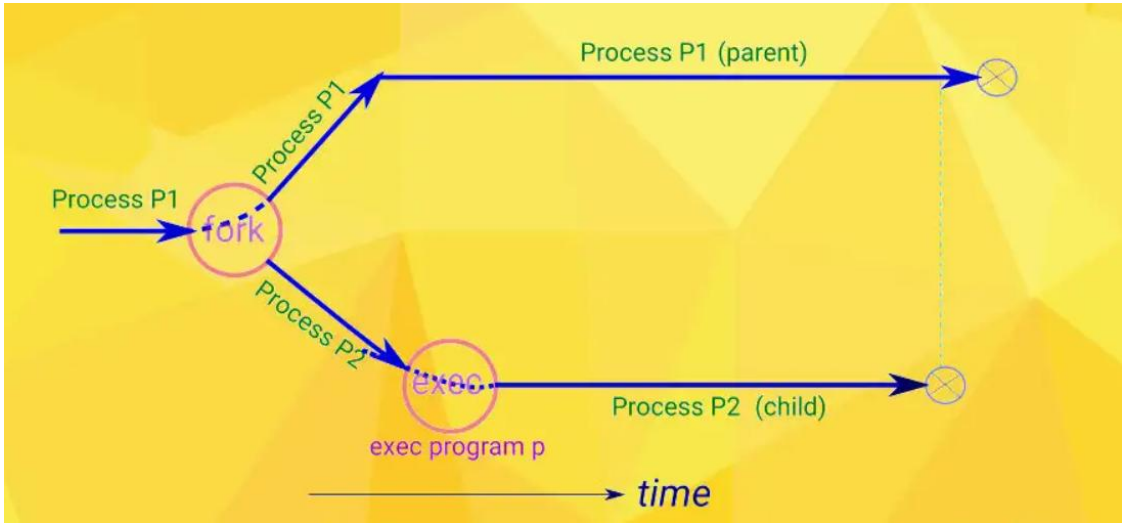
Llamadas al sistema para la administración de procesos.

Fork es el método principal de creación de procesos en sistemas operativos similares a Unix.

Podríamos definir a la llamada al sistema ***fork*** como una operación mediante la cual un proceso crea una copia de sí mismo. Es la única manera de crear un nuevo proceso en POSIX.



La llamada fork crea un duplicado exacto del proceso original, incluyendo todos los descriptores de archivos, registros y todo lo demás. Después de fork, el proceso original y la copia (el padre y el hijo) se van por caminos separados. Todas las variables tienen valores idénticos al momento de la llamada a fork, pero como los datos del padre se copian para crear al hijo, los posteriores cambios en uno de ellos no afectarán al otro.



La llamada a fork devuelve un valor, que es cero en el hijo e igual al identificador del proceso (PID) hijo en el padre. Mediante el uso del PID devuelto, los dos procesos pueden ver cuál es el proceso padre y cuál es el proceso hijo.

En la mayoría de los casos, después de una llamada a fork el hijo tendrá que ejecutar código distinto al del padre.

Un ejemplo basado en el shell: se lee un comando de la terminal, se llama a fork para crear un proceso hijo, se espera a que el hijo ejecute el comando y después, se lee el siguiente comando cuando el hijo termina.

Para esperar a que el hijo termine, el padre ejecuta una llamada al sistema ***waitpid***, la cual sólo espera hasta que el hijo termine (cualquier hijo, si existe más de uno). Cuando waitpid se completa, la dirección a la que apunta el segundo parámetro (statloc) se establece al estado de salida del hijo (terminación normal o anormal, con el valor de exit). También se proporcionan varias opciones, especificadas por el tercer parámetro.

Cuando se escribe un comando, el shell crea un nuevo proceso usando fork. Este proceso hijo debe ejecutar el comando de usuario. Para ello, utiliza la llamada al sistema ***execve***, la cual hace que toda su imagen de núcleo completa se sustituya por el archivo nombrado en su primer parámetro. En otras palabras, execve ejecuta el programa indicado por el primer parámetro.

```
#define TRUE 1

while (TRUE) {                          /* se repite en forma indefinida */
        type_prompt();                  /* muestra el indicador de comando en la pantalla */
        read_command(command, parameters);      /* lee la entrada de la terminal */

        if (fork() !=0) {               /* usa fork para el proceso hijo */
                /* Codigo del padre. */
                waitpid(-1, &status, 0);        /* espera a que el hijo termine */
        } else {
                /* Codigo del hijo. */
                execve(command, parameters, 0);     /* ejecuta el comando */
        }
}
```

Ilustración 25 - Ejemplo de código C de una versión simplificada del shell. En este caso se asume que TRUE es 1.

Vale aclarar que execve pertenece a la familia de funciones de ***exec.*** Se proporcionan varias rutinas de biblioteca incluyendo a execl, execv, execle y execve, para permitir la omisión de los parámetros o para especificarlos de varias formas.

Consideremos el caso de un comando tal como:

```
cp archivo1 archivo2
```

Se utiliza para copiar archivo1 a archivo2. Una vez que el shell se ha bifurcado mediante fork, el proceso hijo localiza y ejecuta el archivo cp y le pasa los nombres de los archivos de origen y destino.

Cuando los procesos terminan su ejecución deben usar la llamada ***exit***. Esta tiene un solo parámetro, el estado de exit (0 a 255), que se devuelve al padre mediante statloc en la llamada al sistema waitpid.

Llamadas al sistema para la administración de archivos.

Muchas llamadas al sistema se relacionan con el sistema de archivos. Para leer o escribir en un archivo, éste debe primero abrirse mediante ***open***. Esta llamada especifica el nombre del archivo que se va a abrir, ya sea como un nombre de ruta absoluto o relativo al directorio de trabajo, y un código de O_RDONLY, O_WRONLY o O_RDWR, que significa abrir para lectura, escritura o ambos. Para crear un nuevo archivo se utiliza el parámetro O_CREAT.

Después, se puede utilizar el descriptor de archivo devuelto para leer o escribir.

Al terminar, el archivo se puede cerrar mediante ***close***, que hace que el descriptor de archivo esté disponible para reutilizarlo en una llamada a open posterior.

Las llamadas de uso más frecuente son sin duda ***read*** y ***write***. Estas llamadas tienen los mismos parámetros.

La función read es una llamada del sistema en Linux que se utiliza para leer datos de un descriptor de archivo[48] abierto en un búfer.

La llamada al sistema write se utiliza para escribir datos en un archivo abierto. Toma el descriptor de archivo, el búfer de datos y la cantidad de bytes que se escribirán como argumentos.

Para cada archivo, UNIX lleva el registro del modo del archivo (archivo regular, especial, directorio, etcétera), su tamaño, la hora de la última modificación y demás información. Para ver esta información, los programas pueden utilizar la llamada al sistema ***stat***. El primer parámetro especifica el archivo que se va a inspeccionar; el segundo, es un apuntador a una estructura en donde se va a colocar la información.

Llamadas al sistema para la administración de directorios.

Las llamadas ***mkdir*** y ***rmdir*** crean y eliminan directorios vacíos, respectivamente. Por su parte, la llamada ***link*** permite que aparezca el mismo archivo bajo dos o más nombres, a menudo en distintos directorios. Un uso común es para permitir que varios miembros del mismo equipo compartan un archivo común, en donde cada uno de ellos puede ver el archivo en su propio directorio, posiblemente bajo distintos nombres.

```
link ("/usr/jim/memo", "/usr/ast/nota");
```

En el ejemplo anterior el archivo memo en el directorio de jim se introduce en el directorio de ast bajo el nombre nota. De acá en adelante, /usr/jim/memo y /usr/ast/nota harán referencia al mismo archivo.

Todo archivo en UNIX tiene un número único, su número -i, que lo identifica. Este número -i es un índice en una tabla de nodos -i, uno por archivo, que indican quién es propietario del archivo, en dónde están sus bloques de disco, etc. Un directorio es simplemente un archivo que contiene un conjunto de pares (número -i, nombre ASCII).

[48] Un descriptor de archivo es un entero positivo que debe ser exclusivo en cada trabajo. El trabajo utiliza un descriptor de archivo para identificar un archivo abierto al realizar operaciones sobre el archivo.

Lo que link hace es tan sólo crear una nueva entrada de directorio con un nombre (posiblemente nuevo), usando el número -i de un archivo existente. Si más adelante se elimina una de las dos mediante la llamada al sistema ***unlink***, la otra sigue vigente. Si se eliminan ambas, UNIX ve que no existen entradas para el archivo (un campo en el nodo-i lleva la cuenta del número de entradas de directorio que apuntan al archivo), por lo que el archivo se elimina del disco.

La llamada al sistema ***mount*** permite combinar dos sistemas de archivos en uno. Al ejecutar la llamada al sistema mount, el sistema de archivos de, por ejemplo, un pendrive, se puede adjuntar al sistema de archivos raíz. Una instrucción común en C para realizar el montaje es:

```
mount ("/dev/fd0", "/mnt", 0);
```

El primer parámetro es el nombre de un archivo especial de bloque para la unidad 0, el segundo parámetro es la posición en el árbol en donde se va a montar y el tercer parámetro indica que el sistema de archivo se va a montar en modo de lectura-escritura o de sólo escritura.

Después de la llamada a mount, se puede tener acceso a un archivo en la unidad 0 con sólo utilizar su ruta del directorio raíz o del directorio de trabajo, sin importar en cuál unidad se encuentre.

Cuando ya no se necesita un sistema de archivos, se puede desmontar mediante la llamada al sistema ***umount***.

Más llamadas al sistema

También existe una variedad de otras llamadas al sistema. Por ejemplo, la llamada a ***chdir*** cambia el directorio de trabajo actual.

```
chdir("/usr/ast/prueba");
```

La llamada al sistema ***chmod*** hace posible modificar el modo de un archivo. Para que un archivo sea de sólo lectura para todos excepto el propietario, podríamos ejecutar:

```
chmod ("archivo", 0644);
```

La llamada al sistema ***kill*** es la forma en que los usuarios y los procesos de usuario envían señales. Si un proceso está preparado para atrapar una señal específica, y luego ésta llega, se ejecuta un manejador de señales. Si el proceso no está preparado para manejar una señal, entonces su llegada mata el proceso (por esto es que se utiliza ese nombre para la llamada).

POSIX define varios procedimientos para manejar el tiempo. Por ejemplo, ***time*** sólo devuelve la hora actual en segundos, en donde 0 corresponde a Enero 1, 1970 a medianoche (justo cuando el día empezaba y no terminaba).

La API Win32 de Windows

Hasta ahora nos hemos enfocado principalmente en UNIX. Windows y UNIX difieren de una manera fundamental en sus respectivos modelos de programación. Un programa de UNIX

consiste en código que realiza una cosa u otra, haciendo llamadas al sistema para realizar ciertos servicios. En contraste, un programa de Windows es por lo general manejado por eventos. El programa principal espera a que ocurra cierto evento y después llama a un procedimiento para manejarlo. Los eventos comunes son las teclas que se oprimen, el mouse que se desplaza, un botón de mouse que se oprime o un pendrive que se inserta. Después, los manejadores se llaman para procesar el evento, actualizar la pantalla y actualizar el estado interno del programa.

Desde luego que Windows también tiene llamadas al sistema. Con UNIX, hay casi una relación de uno a uno entre las llamadas al sistema (por ejemplo, read) y los procedimientos de biblioteca (por ejemplo, read) que se utilizan para invocar las llamadas al sistema.

Con Windows, la situación es bastante distinta. Para empezar, las llamadas a la biblioteca y las llamadas al sistema están muy desacopladas. Microsoft ha definido un conjunto de procedimientos conocidos como API Win32 (Application Program Interface, Interfaz de programación de aplicaciones) que los programadores deben utilizar para obtener los servicios del sistema operativo.

Esta interfaz se proporciona (parcialmente) en todas las versiones de Windows, desde Windows 95. Al desacoplar la interfaz de las llamadas actuales al sistema, Microsoft retiene la habilidad de modificar las llamadas al sistema en el tiempo (incluso de versión en versión) sin invalidar los programas existentes.

El número de llamadas a la API Win32 es muy grande, alrededor de los miles. Aunque muchas de ellas invocan llamadas al sistema, un número considerable de las mismas se lleva a cabo completamente en espacio de usuario. Como consecuencia, con Windows es imposible ver lo que es una llamada al sistema (el kernel se encarga de ello) y qué es simplemente una llamada a la biblioteca en espacio de usuario.

La API Win32 tiene un gran número de llamadas para administrar ventanas, figuras geométricas, texto, tipos de letras, barras de desplazamiento, cuadros de diálogo, menús y otras características de la GUI.

Windows no tiene una jerarquía de procesos como UNIX, por lo que no hay un concepto de un proceso padre y un proceso hijo. Una vez que se crea un proceso, el creador y el creado son iguales. ***WaitForSingleObject*** se utiliza para esperar un evento.

Se pueden esperar muchos eventos posibles. Si el parámetro especifica un proceso, entonces el proceso llamador espera a que el proceso especificado termine, lo cual se hace mediante ***Exit-Process***.

Windows tiene directorios, que se crean y eliminan con las llamadas a la API ***CreateDirectory***

y ***RemoveDirectory***, respectivamente. También hay una noción de un directorio actual, el cual se establece mediante ***SetCurrentDirectory***.

Desde luego que Win32 tiene un gran número de otras llamadas que UNIX no tiene, en especial para administrar la GUI.

UNIX	Win32	Descripción
fork	CreateProcess	Crea un nuevo proceso
waitpid	WaitForSingleObject	Puede esperar a que un proceso termine
execve	(ninguno)	CreateProces = fork + execve
exit	ExitProcess	Termina la ejecución
open	CreateFile	Crea un archivo o abre uno existente
close	CloseHandle	Cierra un archivo
read	ReadFile	Lee datos de un archivo
write	WriteFile	Escribe datos en un archivo
lseek	SetFilePointer	Desplaza el apuntador del archivo
stat	GetFileAttributesEx	Obtiene varios atributos de un archivo
mkdir	CreateDirectory	Crea un nuevo directorio
rmdir	RemoveDirectory	Elimina un directorio vacío
link	(ninguno)	Win32 no soporta los enlaces
unlink	DeleteFile	Destruye un archivo existente
mount	(ninguno)	Win32 no soporta el montaje
umount	(ninguno)	Win32 no soporta el montaje
chdir	SetCurrentDirectory	Cambia el directorio de trabajo actual
chmod	(ninguno)	Win32 no soporta la seguridad (aunque NT sí)
kill	(ninguno)	Win32 no soporta las señales
time	GetLocalTime	Obtiene la hora actual

Referencias y bibliografía de la sección.

- Silberschatz 2-4.
- Tanenbaum 1-6.

Interfaz con las aplicaciones.

Dentro de los sistemas operativos conviven varias categorías o tipos de interfaces. La interfaz con las aplicaciones es la capa del sistema operativo que permite que los programas del usuario (como editores de texto, navegadores, juegos, etc.) puedan interactuar con el sistema operativo y, a través de él, con los recursos del hardware (CPU, memoria, archivos, etc.).

Podríamos pensar en las ***llamadas al sistema*** como una interfaz no muy amigable para los programadores, que permite realizar solicitudes al sistema por parte de los programas

siguiendo unos convenios sobre el nombre de la petición y sus parámetros. El conjunto de las llamadas constituye la ***SCI***[49] (System Call Interface).

Las interfaces para el usuario se implementan con software del sistema, mediante programas intérpretes. Es común utilizar una interfaz gráfica (***GUI***: Graphical User Interface) o una interfaz de línea de comandos (***CLI***: Command Line Interface) implementada con un programa intérprete de órdenes, o shell. El shell interpreta directamente algunas órdenes (builtin commands), pero la mayoría de los comandos son en realidad nombres de programas del sistema que el shell invoca.

Es importante remarcar que la GUI y el shell comúnmente no forman parte del kernel del sistema operativo. Por lo tanto, técnicamente, la interfaz con las aplicaciones es la API más las llamadas al sistema que el kernel exponga.



Los programas pueden hacer uso directamente de la SCI, pero las facilidades que ofrece son muy básicas y su uso engorroso. Por eso, los programas de aplicación, que normalmente se escriben en algún lenguaje de alto nivel[50], no hacen un uso directo de las llamadas al sistema, sino por medio de un conjunto de programas de utilidad que proporcionan funciones de un nivel similar al de estos lenguajes. Este conjunto se llama biblioteca (o librería), y ofrece otra interfaz a las aplicaciones, la ***API*** (Application Programming Interface). Una llamada a una función de la API puede generar una o varias llamadas al sistema.

En sistemas Unix la biblioteca que ofrece la API se implementa en lenguaje C y se llama ***libc*** (o ***glibc***). En realidad, si nos ponemos estrictos, la API nativa del Kernel de Linux es el conjunto de sus llamadas al sistema en combinación con la libc que actúa como un ***wrapper***[51] (envoltorio).

POSIX (Portable Operating System Interface) es un estándar del IEEE que define los nombres y convenios de las funciones de la API para Unix.

[49] La capa llamada System Call Interface permite a las aplicaciones realizar llamadas al sistema para acceder a recursos privilegiados.
[50] Un lenguaje de programación de alto nivel es un lenguaje que se asemeja más al lenguaje humano que un lenguaje de bajo nivel, lo que facilita la escritura y comprensión del código.
[51] Un wrapper de C es una capa de código (función, biblioteca o estructura) que "envuelve" otro código C existente para modificar, extender o facilitar su uso sin cambiar el original. Actúa como intermediario para simplificar interfaces complejas, mejorar la compatibilidad o añadir manejo de errores, permitiendo reutilizar código.

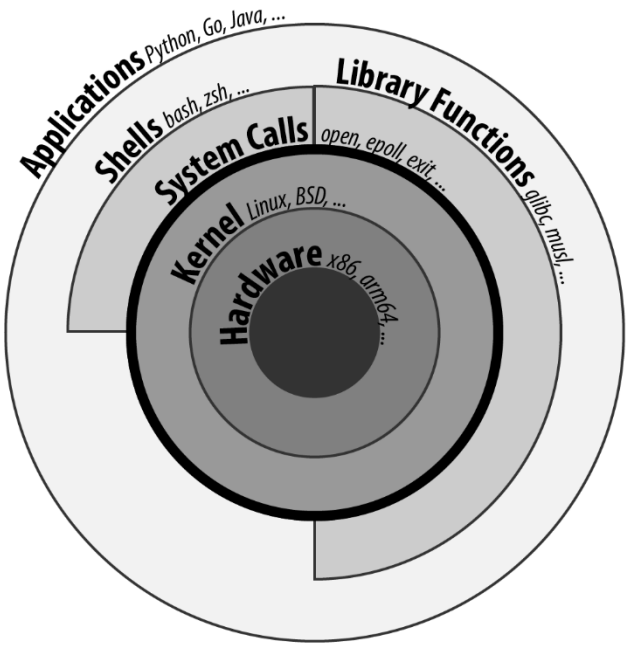
La ventaja de utilizar la API en lugar de la SCI, aparte de la mayor facilidad de programación, es que proporciona una abstracción del sistema operativo, lo que permite desarrollar programas más portables. Hay versiones de sistemas operativos que no son Unix pero que siguen el estándar POSIX; un mismo programa fuente podría servir para cualquiera de estos sistemas.

También podríamos definir una API como un conjunto de definiciones y protocolos que se usa para diseñar e integrar el software de las aplicaciones.

Las API permiten que productos y servicios se comuniquen con otros, sin necesidad de saber cómo están implementados. Esto simplifica el desarrollo de las aplicaciones y permite ahorrar tiempo y dinero. Las API otorgan flexibilidad; simplifican el diseño, la administración y el uso de las aplicaciones.

Libc y glibc.

La biblioteca estándar de C, o ***libc***, es una recopilación de archivos de cabecera y bibliotecas con rutinas que implementan operaciones comunes, tales como las de entrada y salida o el manejo de cadenas. Prácticamente todo programa implementado en C se basa en la biblioteca estándar para funcionar.



La biblioteca de C de GNU (del inglés GNU C library), comúnmente conocida como glibc es la biblioteca de tiempo de ejecución estándar del lenguaje C de GNU.

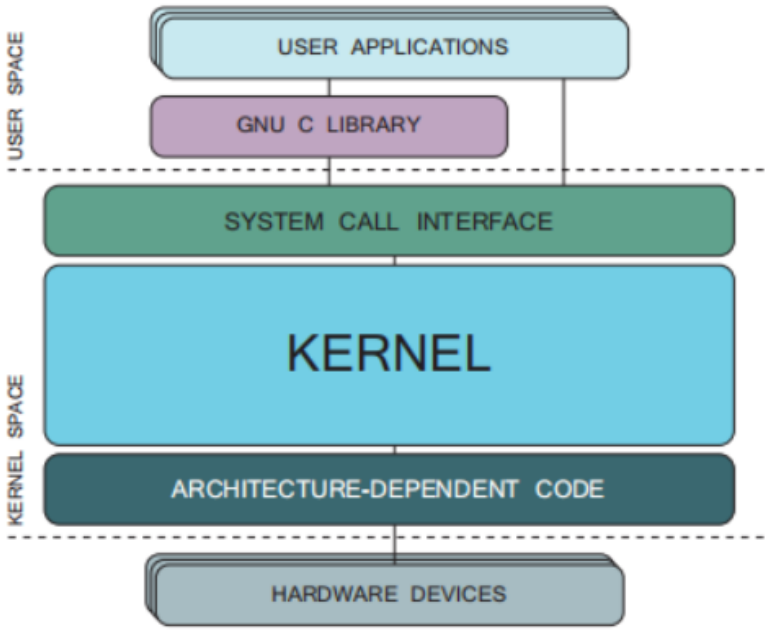
En los sistemas en los que se usa, esta biblioteca de C que proporciona y define las llamadas al sistema y otras funciones básicas, es utilizada por casi todos los programas. Es muy empleada en los sistemas GNU y sistemas basados en el núcleo Linux. Como consecuencia, la mayoría de los programadores de Linux no llaman directamente al Kernel, en su lugar, llaman a funciones de la libc (como open()o read()).

Es importante aclarar que las funciones de esta librería ***no son syscalls.*** De hecho, hay funciones en la libc que ni siquiera tocan el kernel porque no necesitan hardware. Por ejemplo:

- abs(): Calcula el valor absoluto de un número.
- strcpy(): Copia texto de un lado a otro.

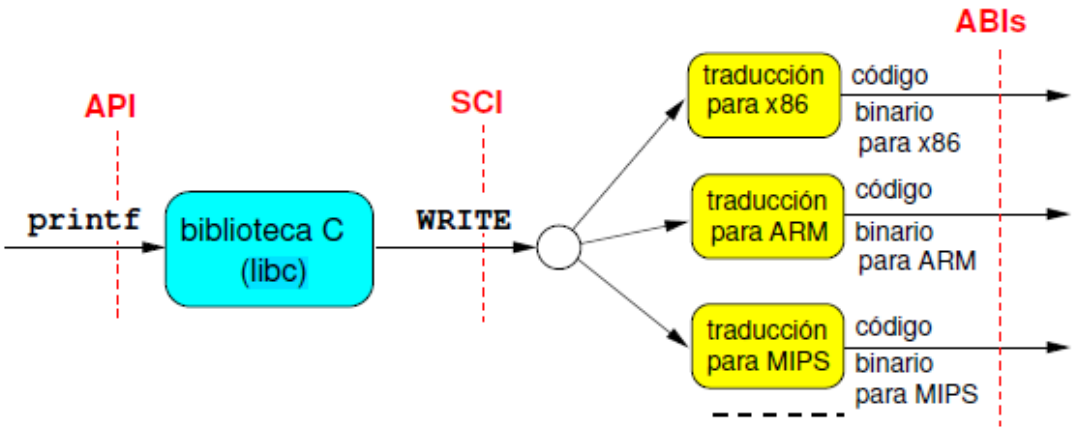
Estas funciones viven y mueren en el espacio de usuario. El kernel ni se entera de ellas.

Otra característica de la librería estándar de C es que resulta muy portable y soporta gran cantidad de plataformas de hardware.



POSIX especifica una serie de rutinas que deben estar disponibles más allá de la propia biblioteca estándar de C. A menudo éstas se implementan conjuntamente con la biblioteca de C estándar, con distintos niveles de proximidad. Por ejemplo, glibc implementa funciones como fork() dentro de libc.so.

Para entender las ventajas de las librerías estándar de C podríamos imaginar un programa tan sencillo como el de sumar dos números enteros. Este programa se complicaría enormemente si quisiéramos convertirlo en interactivo, es decir, que lea los operandos del teclado y escriba el resultado en la pantalla. El motivo es que hay que acompañarlo de un subprograma que convierta las cadenas de caracteres leídas en números y otro que convierta el resultado numérico en una cadena de caracteres, con el fin de poder utilizar las llamadas read y write. Sin embargo, en la biblioteca libc hay dos funciones, scanf y printf, que, llamándolas con los parámetros adecuados, leen y escriben en una variedad de formatos, y se encargan también de las llamadas necesarias al sistema.



Wrappers.

A las funciones de la libc que sirven para invocar una syscall se las conoce como ***Wrappers*** (o funciones envoltorio). Su trabajo es "envolver" la complejidad del procesador y el kernel en una función de C que sea fácil de usar para los programadores.

Casi todas las funciones en la libc que interactúan con el mundo exterior (archivos, red, memoria, procesos) son wrappers. Algunos de los ejemplos más comunes:

Nombre en la `libc`	Lo que hace el Wrapper	Syscall que invoca
`open()`	Abre un archivo.	`sys_open`
`read()`	Lee datos de un descriptor.	`sys_read`
`write()`	Escribe datos.	`sys_write`
`fork()`	Crea un proceso hijo.	`sys_clone` (en Linux moderno)
`exit()`	Termina el programa.	`sys_exit_group`
`getpid()`	Obtiene el ID del proceso.	`sys_getpid`

POSIX.

Debido a el código fuente de UNIX estaba disponible ampliamente, varias organizaciones desarrollaron sus propias versiones (incompatibles entre sí), lo cual produjo caos. Se desarrollaron dos versiones principales: System V de AT&T y BSD (Berkeley Software Distribution, Distribución de Software de Berkeley) de la Universidad de California en Berkeley. Estas versiones tenían también variantes menores.

Para que fuera posible escribir programas que pudieran ejecutarse en cualquier sistema UNIX, el IEEE[52] desarrolló un estándar para UNIX conocido como ***POSIX***, con el que la mayoría de las versiones de UNIX actuales cumplen.

POSIX define una interfaz mínima de llamadas al sistema a la que los sistemas UNIX deben conformarse. De hecho, algunos de los otros sistemas operativos (no UNIX) también admiten ahora la interfaz POSIX.

POSIX tiene aproximadamente 100 ***llamadas a procedimientos***. En mayor grado, los servicios ofrecidos por estas llamadas determinan la mayor parte de la labor del sistema operativo, ya que la administración de recursos en las computadoras personales es una actividad mínima (por lo menos si se le compara con los equipos grandes que tienen muchos usuarios). Los ***servicios*** incluyen acciones tales como crear y terminar procesos, crear, eliminar, leer y escribir en archivos, administrar directorios y realizar operaciones de entrada y salida.

Al margen, vale la pena mencionar que la asignación de las llamadas a procedimientos POSIX a llamadas al sistema no es de uno a uno. El estándar POSIX especifica varios procedimientos que debe suministrar un sistema que se conforme a este estándar, pero no especifica si deben ser llamadas al sistema, llamadas a una biblioteca, o algo más.

Si un procedimiento puede llevarse a cabo sin necesidad de invocar una llamada al sistema, por lo general se realizará en espacio de usuario por cuestión de rendimiento. Sin embargo, la mayoría de los procedimientos POSIX invocan llamadas al sistema, en donde por lo general un procedimiento se asigna directamente a una llamada al sistema.

Referencias y bibliografía de la sección.

- Gregorio Fernández Fernández - Elementos de sistemas operativos 2-4.
- Tanenbaum 1-2-3.

Interfaz con la maquina real.

La ***interfaz con la máquina real*** es la parte del sistema operativo que se encarga de interactuar directamente con el hardware de la computadora. Es decir, es la capa más cercana a la máquina física, que traduce las órdenes del sistema operativo (y, por extensión, de las aplicaciones) en instrucciones que el hardware pueda ejecutar. Luego, recibe señales (respuestas) desde el hardware para pasarlas al sistema.

La interfaz con la máquina real es la sección del kernel que trabaja al nivel más bajo, usando ***instrucciones privilegiadas del procesador*** y ***controladores de dispositivos*** (drivers) para comunicarse con el hardware.

[52] El Instituto de Ingenieros Eléctricos y Electrónicos es la sociedad técnico-profesional más grande y prestigiosa del mundo, dedicada a promover y divulgar los avances científicos en las áreas de Ingeniería Eléctrica, Electrónica, Energética, Informática y afines.

Estas instrucciones privilegiadas y el acceso a registros específicos son distintos en cada arquitectura.

El ***software dependiente de la arquitectura*** incluye código que interactúa directamente con el hardware, y ese hardware varía según la arquitectura (por ejemplo, x86, ARM, RISC-V). Este código es específico para manejar la CPU, controladores, interrupciones y modos de operación de un tipo de procesador o plataforma.

El ***software controlador de dispositivos*** (drivers) está diseñado específicamente para permitir al sistema operativo comunicarse con dispositivos como discos, impresoras, tarjetas de red, etc.

Podríamos decir que la interfaz con la maquina real es la parte más profunda del kernel, ya que es la sección del sistema operativo que más se acerca al lenguaje binario que entiende la máquina real.

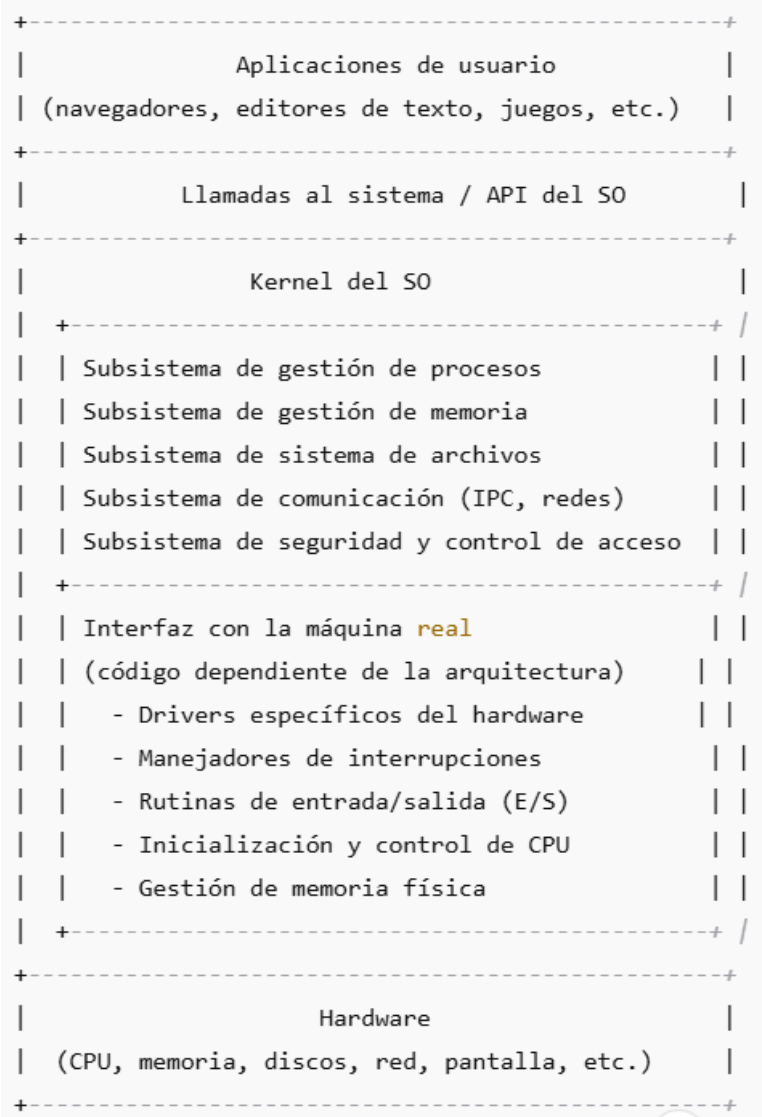
En ***Linux***, por ejemplo, el directorio ***arch/*** contiene la parte del kernel que es dependiente de la arquitectura. Ahí está gran parte de la interfaz con la máquina real, como el manejo de interrupciones, inicialización del procesador, gestión de memoria física, etc.

A modo de ejemplo, analicemos de una forma muy superficial como podríamos mover (portar) un sistema operativo tipo UNIX a una nueva máquina.

Para portar un sistema operativo de este tipo primero hay que escribir un compilador en C para la nueva máquina.

Después, hay que escribir ***drivers*** para los nuevos dispositivos de E/S de la máquina, como monitores, impresoras y discos. Aunque el código de los drivers probablemente está escrito en C, no es posible moverlo a otra máquina, compilarlo y ejecutarlo debido a que no hay dos discos que funcionen de la misma manera.

Por último, se debe reescribir una pequeña cantidad de ***código dependiente de la máquina***, como los ***manejadores de interrupciones*** y las ***rutinas de administración de memoria***, que por lo general están en lenguaje ***ensamblador***.



E/S.

Gran parte del código del sistema operativo se dedica a gestionar la entrada y la salida, debido a su importancia para la fiabilidad y rendimiento del sistema y debido también a la variada naturaleza de los dispositivos.

Una computadora de propósito general consta de una o más CPU y de múltiples controladoras de dispositivo que se conectan a través de un bus común. Cada controladora de dispositivo se encarga de un tipo específico de dispositivo.

Dependiendo de la controladora, puede haber más de un dispositivo conectado.

Una controladora de dispositivo mantiene algunos búferes locales y un conjunto de registros de propósito especial. La controladora del dispositivo es responsable de transferir los datos entre los dispositivos periféricos que controla y su búfer local.

Normalmente, los sistemas operativos tienen un controlador (driver) de dispositivo para cada controladora (controller) de dispositivo. Este software controlador del dispositivo es capaz de

entenderse con la controladora hardware y presenta al resto del sistema operativo una interfaz uniforme mediante la cual comunicarse con el dispositivo.

Al iniciar una operación de E/S, el controlador del dispositivo carga los registros apropiados de la controladora hardware. Ésta, a su vez, examina el contenido de estos registros para determinar qué acción realizar (como, por ejemplo, “leer un carácter del teclado”).

La controladora inicia entonces la transferencia de datos desde el dispositivo a su búfer local. Una vez completada la transferencia de datos, la controladora hardware informa al controlador de dispositivo, a través de una interrupción, de que ha terminado la operación. El controlador devuelve entonces el control al sistema operativo, devolviendo posiblemente los datos, o un puntero a los datos, si la operación ha sido una lectura. Para otras operaciones, el controlador del dispositivo devuelve información de estado.

Esta forma de E/S controlada por interrupción resulta adecuada para transferir cantidades pequeñas de datos, pero representa un desperdicio de capacidad de proceso cuando se usa para movimientos masivos de datos, como en la E/S de disco.

Para resolver este problema, se usa el acceso directo a memoria (DMA, direct memory access). Después de configurar búferes, punteros y contadores para el dispositivo de E/S, la controladora hardware transfiere un bloque entero de datos entre su propio búfer y la memoria, sin que intervenga la CPU. Sólo se genera una interrupción por cada bloque, para decir al controlador software del dispositivo que la operación se ha completado, en lugar de la interrupción por byte generada en los dispositivos de baja velocidad.

De esta forma, mientras la controladora hardware realiza estas operaciones, la CPU está disponible para llevar a cabo otros trabajos.

Referencias y bibliografía de la sección.

- Silberschatz 1-2-3.
- Tanenbaum 10-1-3.

El Intérprete de mandatos.

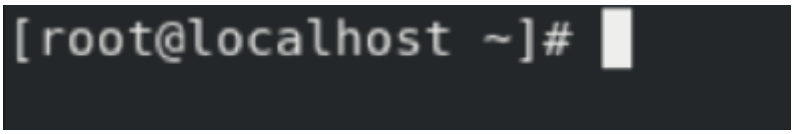
El intérprete de ordénenos, comandos o mandatos es un programa que permite a los usuarios interactuar con el sistema operativo. Es la interfaz principal entre un usuario sentado en su terminal y el sistema operativo, a menos que se utilice una GUI.

Estos interpretes no forman parte del sistema operativo, sin embargo, son importantes y útiles. Por este motivo, vale la pena repasar generalidades de los intérpretes de comandos. Nos centraremos en el intérprete de comandos de UNIX, conocido como shell.

Como ya dijimos, aunque no forma parte del sistema operativo, utiliza con frecuencia muchas características del mismo y, por ende, sirve como un buen ejemplo de la forma en que se pueden utilizar las llamadas al sistema. Existen muchos shells, incluyendo csh, ksh y bash. Todos ellos derivan del shell original (sh).



En muchos sistemas Linux, cuando cualquier usuario inicia sesión, se inicia un shell. El shell tiene la terminal como entrada estándar y salida estándar. Empieza por escribir el indicador de comandos (***prompt***[53]), un carácter tal como un signo de dólar o numeral, que indica al usuario que el shell está esperando aceptar un comando.



Por ejemplo, si el usuario escribe:

```
date
```

El shell crea un proceso hijo y ejecuta el programa date[54]. Mientras se ejecuta el proceso hijo, el shell espera a que termine. Cuando el hijo termina, el shell escribe de nuevo el indicador y trata de leer la siguiente línea de entrada.

El usuario puede especificar que la salida estándar sea redirigida a un archivo, por ejemplo:

```
date >archivo
```

[53] El prompt es la información o símbolo que aparece antes del cursor en la terminal de Linux, donde se empieza a escribir los comandos.

[54] El comando date nos permite imprimir la fecha de hoy en muchos formatos. También nos permite extraer solo el día, mes, día de la semana, etc.

La salida de un programa se puede utilizar como entrada para otro, si se conectan mediante un canal. Así:

```
cat archivo1 archivo2 archivo3 | sort >/dev/lp
```

Se invoca al programa cat[55] para concatenar tres archivos y enviar la salida a sort[56] para ordenar todas las líneas en orden alfabético. La salida de sort se redirige al archivo /dev/lp, que por lo general es la impresora.

Si un usuario coloca el signo ***&*** después de un comando, el shell no espera a que se complete. En vez de ello, proporciona un indicador de comandos de inmediato. En consecuencia:

```
cat archivo1 archivo2 archivo3 | sort >/dev/lp &
```

Inicia el comando sort como un trabajo en segundo plano y permite al usuario continuar su trabajo de manera normal mientras el ordenamiento se lleva a cabo.

Referencias y bibliografía de la sección.

- Tanenbaum 1-5-6.
- Silberschatz 2-2-1.

La interfaz gráfica de usuario.

Actualmente, muchas computadoras personales utilizan una GUI. Podríamos decir que la GUI es sólo un programa que se ejecuta encima del sistema operativo, como un shell. En los sistemas Linux, este hecho se hace obvio debido a que el usuario puede elegir entre varias GUIs como Gnome[57] o KDE Plasma[58]. En Windows, la separación entre la interfaz gráfica de usuario y el sistema operativo no es tan obvia como en Linux.

[55] El comando cat de Linux permite mostrar el contenido de archivos, unirlos y crear nuevos.
[56] El comando sort de Linux ordena las líneas de un archivo de texto o la salida de un comando.
[57] GNOME es un entorno de escritorio e infraestructura de desarrollo para sistemas operativos GNU/Linux, Unix y derivados Unix como BSD o Solaris; compuesto enteramente de software libre.
[58] KDE Plasma es el entorno de escritorio creado por KDE principalmente para distribuciones Linux.

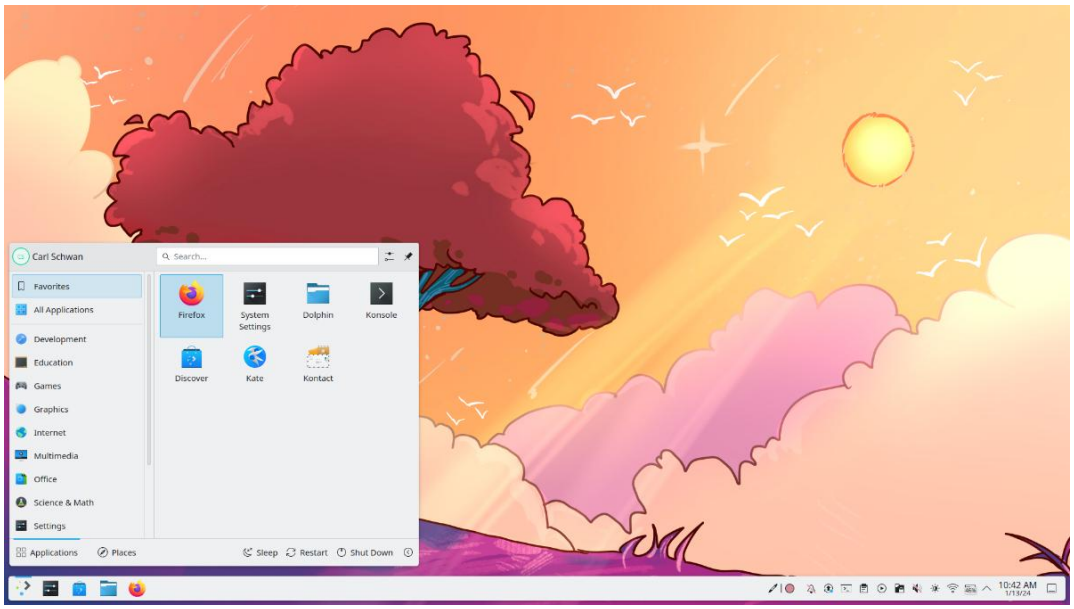


Ilustración 26 - KDE Plasma 6.

Con una GUI, en lugar de tener que introducir ordenes directamente a través de la línea de comandos, los usuarios emplean un sistema de ventanas y menús controlables mediante el mouse. Es decir, que una GUI proporciona una especie de escritorio en el que el usuario mueve el mouse para colocar su puntero sobre imágenes, o iconos, que se muestran en la pantalla (el escritorio) y que representan programas, archivos, carpetas y funciones del sistema.

Dependiendo de la ubicación del puntero, pulsar el botón del ratón puede invocar un programa, seleccionar un archivo o carpeta o desplegar un menú que contenga comandos ejecutables.

Las primeras interfaces gráficas de usuario aparecieron debido, en parte, a las investigaciones realizadas en el departamento de investigación de Xerox Pare a principios de los años 70. La primera GUI se incorporó en la computadora Xerox Alto[59] en 1973.

[59] El Xerox Alto, desarrollada en 1973, fue uno de las primeras computadoras personales de la historia y la que utilizó por primera vez la metáfora de escritorio y una interfaz gráfica de usuario, así como un mouse.

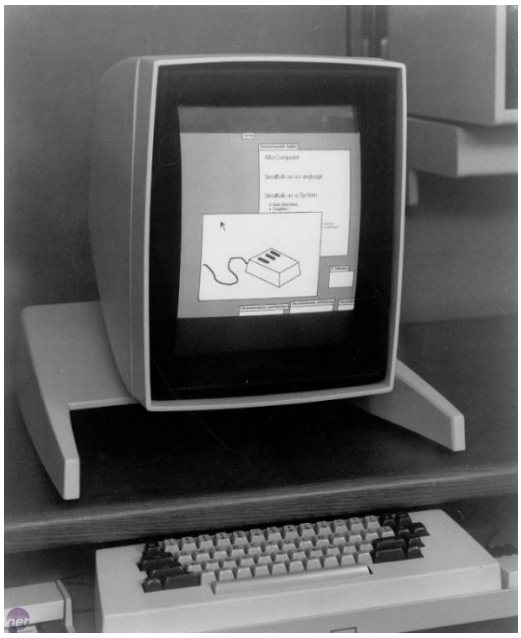


Ilustración 27 - Xerox PARC Alto.

Sin embargo, las interfaces gráficas se popularizaron con la introducción de las computadoras Apple Macintosh en los años 80. La Macintosh 128K fue una de las primeras computadoras personales con una GUI que tuvo éxito en el mercado.

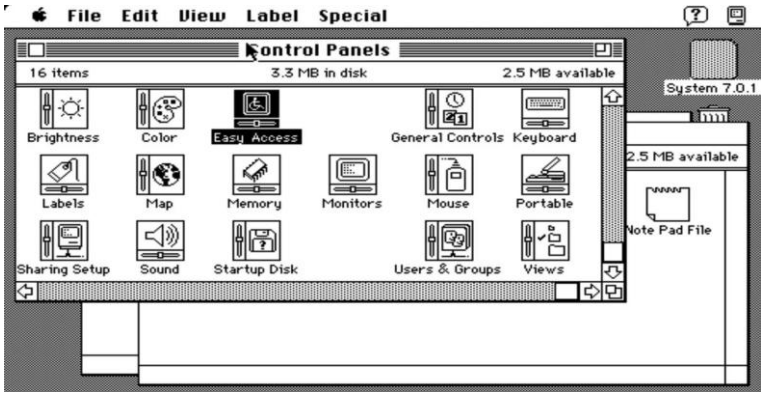


Ilustración 28 - System 7 de Apple.

La primera versión de Microsoft Windows (versión 1.0) estaba basada en una interfaz GUI que permitía interactuar con el sistema operativo MS-DOS. Las distintas evoluciones de los sistemas Windows proceden de esta versión inicial, a la que se le han aplicado cambios cosméticos en cuanto su apariencia y diversas mejoras de funcionalidad.

Debido a lanzamientos de posteriores versiones de Windows y finalmente con la llegada de Windows 95, Microsoft se convirtió en una empresa líder en el mercado por el gran éxito comercial de su sistema operativo. En la década de 1990, las interfaces gráficas de Apple y Microsoft eran dueñas del mercado.

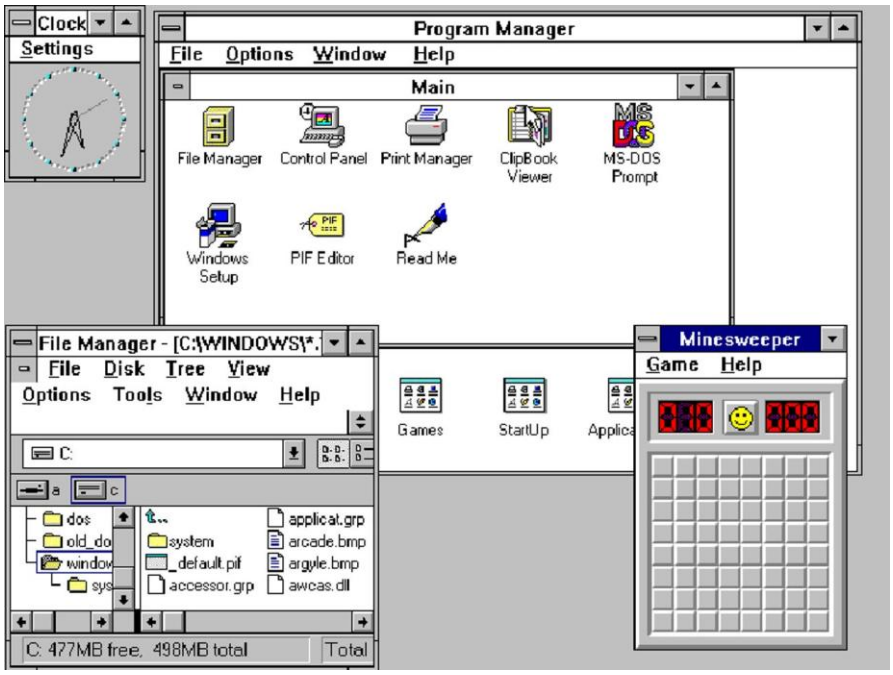


Ilustración 29 - Windows 3.1.

Tradicionalmente, en los sistemas UNIX han predominado las interfaces de línea de comandos. Sin embargo, con el paso del tiempo, se fueron desarrollando diversas interfaces de tipo GUI en diferentes proyectos de código fuente abierto, como por ejemplo el entorno de escritorio GNOME.

Este tipo de escritorios se ejecutan sobre Linux y otros varios sistemas UNIX, y generalmente están disponibles con licencias de código abierto, lo que quiere decir que su código fuente es del dominio público.

La decisión de usar una interfaz de línea de comandos o GUI es, principalmente, una opción personal. La interfaz de usuario puede variar de un sistema operativo a otro e incluso de un usuario a otro dentro de un sistema; por esto, la interfaz de usuario se suele eliminar de la propia estructura del sistema. El diseño de una interfaz de usuario útil y amigable no es, por lo tanto, una función directa del sistema operativo.

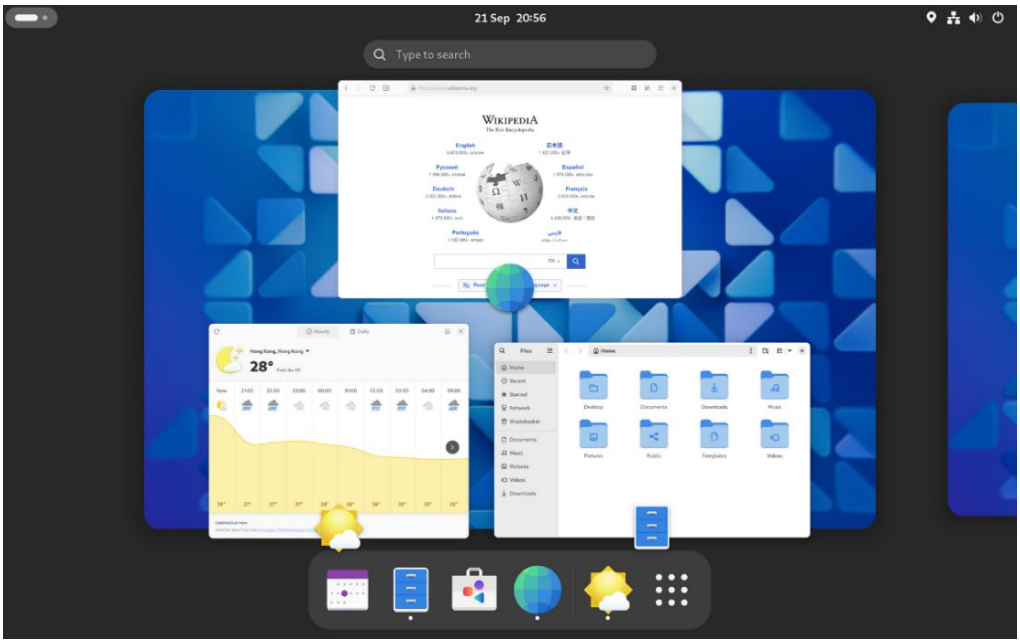


Ilustración 30 - Gnome.

Referencias y bibliografía de la sección.

- Silberschatz 2-2-2.